

© 2026 Antonios Psistakis

Reliable Consistency and Persistency Support in Distributed and Shared Memory Systems in Datacenters

by

Antonios Psistakis

Dissertation

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois Urbana-Champaign, 2026

Urbana, Illinois

Doctoral Committee:

Professor Josep Torrellas, Chair and Director of Research
Associate Professor Jian Huang
Assistant Professor Ramnatthan Alagappan
Associate Professor Alexandros Daglis, Georgia Institute of Technology
Dr. Fabien Chaix, ICS-FORTH

Abstract

This thesis presents modern solutions for achieving high-performant and resilient distributed and shared-memory systems through the implementation of consistency and persistency models and novel recovery mechanisms for datacenters.

We first introduce MINOS, a system designed to support leaderless distributed systems by offering Linearizable consistency with five distinct types of persistency. MINOS is evaluated in two configurations: MINOS-Baseline (MINOS-B) and MINOS-Offload (MINOS-O). MINOS-B provides the foundational algorithms for the real-system implementation of the models, while MINOS-O leverages a new SmartNIC architecture to optimize performance through selective data coherence, message batching, and broadcasting. Our evaluations on a 5-node distributed machine demonstrate that MINOS-O significantly reduces request latency and increases throughput in microbenchmarks. Compared to MINOS-B, MINOS-O reduces the average end-to-end latency of two microservice functions by 35%.

We enhance MINOS with resilience support. The result is IASO, a general recovery approach tailored for leaderless distributed systems. IASO performs optimized online incremental recovery, ensuring minimal disruption to live-system throughput. Implemented on a 6-node Intel cluster, IASO recovers from node failures in just 120 milliseconds on average, with a modest 8.7% reduction in throughput. This marks a substantial improvement over state-of-the-art systems, which require 360 seconds for recovery and incur a 16.2% throughput reduction. Together, MINOS and IASO provide a comprehensive framework for building high-performant and resilient distributed systems.

Finally, we introduce DIKTAMO, a hardware–software co-designed architecture that brings resilience to Compute Express Link (CXL) 3.0-based (and newer) distributed shared memory systems. While CXL enables cache-coherent data sharing across nodes at

cacheline granularity, it introduces a critical resilience challenge: node failures can lead to the loss of dirty cacheline state, corrupting application execution, and the CXL specification neither addresses processor failures nor restores a consistent shared-memory state in the directory. DIKTAMO enables fault-tolerant write-back caching by extending the coherence protocol with lightweight, cacheline-granular replication of writes to a small set of peer nodes, where updates are temporarily stored in hardware-based Logging Units and periodically copied to remote memory, enabling recovery by reconstructing correct directory and memory state without reverting to costly write-through policies. It further incorporates protocol optimizations that overlap replication with store execution to hide latency and preserve performance. Evaluation on a simulated multi-node CXL cluster shows that DIKTAMO achieves fault-tolerant shared memory semantics with only a 30% slowdown (26% with two replicas) over a system without fault tolerance, and is $2.82\times$ faster than a write-through baseline.

*Dedicated to my parents, sister, and friends for their steadfast support and enduring love,
and to those who face adversity with courage and resilience.*

Acknowledgements

I concluded my PhD defense with a poem by Constantinos Kavafis, *Ithaka*, whose message emphasizes the value of the journey and keeping one’s “thought raised high” despite challenges. This PhD has indeed been a long and demanding journey, made possible by the support of many.

I am deeply grateful to my advisor, Prof. Josep Torrellas, for his trust, guidance, and continuous support, and for the opportunity to grow as a researcher and explore paths in academia and industry.

I sincerely thank my committee members—Prof. Josep Torrellas, Prof. Jian Huang, Prof. Ramnatthan (Ram) Alagappan, Prof. Alexandros (Alex) Daglis, and Dr. Fabien Chaix—for their time, feedback, and insightful discussions, which strengthened this work.

During my PhD studies at the University of Illinois at Urbana-Champaign (UIUC), I was fortunate to collaborate with outstanding researchers, academics, and colleagues, including Prof. Josep Torrellas, Prof. Ramnatthan (Ram) Alagappan, Dr. Fabien Chaix, Dr. Chloe Alverti, Burak Ocalan, Rishabh Ravi, Shashwat Jaiswal, Dr. David Koufaty, Dr. Apostolos Kokolis, Prof. Jian Huang, and Benjamin (Ben) Reidys. I have learned a great deal from each of them.

I am grateful for my industry collaborations, including at Microsoft with Prof. Daniel S. Berger, Dr. Pantea Zardoshti, Samir Rajadnya, Navneet Singh, Narayan Ranganathan, Yuhong Zhong, and Ashwin Poduval, as well as at NVIDIA with Arthur Handras and his team. These collaborations have provided me with invaluable insights and experiences.

I wholeheartedly thank the i-acoma group and all my labmates, my academic family—listed here in alphabetical order—Abraham (Abe), Antonio, Apostolos, Burak, Charles (Chase), Chloe, Dimitrios M., Dimitrios S. (Prof. Skarlatos), Dingyuan, Filippou, Gerasimos (Makis), Jakob, Jovan (Prof. Stojkovic), Konstantinos (Kostas), Nam, Neil (Prof. Zhao), Nikoleta, Pedro, Raúl, Rishabh, Serif, Shashwat, Victor, and Yao. This

group was not only a research environment but a true community. I am very thankful to Prof. Skarlatos for his support in the early years of my studies, and to Dr. Alverti for her support and collaboration later on. Special thanks as well to my Lumon squad for the great memories.

Beyond the lab, I am deeply grateful to my friends on campus, including Anna, Jovan, and Makis (The President). We arrived the same year, lived as roommates or neighbors, and grew together—creating indelible memories that I will always cherish. They hold a special place in my heart. Later, I met and became friends with Mubashir, Zois, Argyris M., George, Panagiotis, and Polyvios, with whom I shared many memorable moments—through time spent together, as well as travels, concerts, and gatherings. The same holds for Nikoleta, Simoni, Chaitanya, and Chloe—so many great memories. While all my friends have been caring, Mubashir has always looked out for me; I am fortunate to call him a friend. I also thank Selin, Silvia, Dimitris B., Argyris G., Ben, Antonis, Thimios, Sofia, Xinying, Bakshree, Meilu, Tobias, Manav, Keshav, and Konstantinos. I am grateful for everyone’s friendship, which made this journey all the more meaningful. More broadly—and hoping I am not forgetting anyone—I am very grateful to the Greek community on campus, especially the graduate students with whom I interacted most, for keeping the Greek spirit alive. Special thanks as well to Manousos and the entire Cretan Youth of Chicago team for the cherished moments, and to Jake, Michael, and the entire Brazilian Jiu-Jitsu student organization on campus for the enriching time I spent as a member.

During my internships, I had the pleasure of meeting and forming new friendships with many wonderful people—listed here in alphabetical order: Aditya, Arthur, Ashwin, Athinagoras, Christos, Dimitra, Elizabeth, Emily, Jordi, Lei, Manos, Naz, Pei-Huan, Seokjin, Tanay, Yuhong, and Zeying. I also thank my cousin Theodoros, my friend (and former desk mate) Angelos, and my longtime friend Landon for their friendship, both from afar and in person.

I am profoundly grateful to my parents, Rania and George, and my sister Marianna for their unconditional love and unwavering support. I also thank my relatives in Chicago—my uncle George, aunt Theodora, and cousin John—for their warmth and kindness, which brought me comfort and joy whenever I had the chance to visit. I am equally grateful to all my relatives back home, who always kept me in their thoughts and supported me from afar. I am also grateful to my friends back home—Manos, Stratos, Maria, Emmy, Chara, Georgia, Despoina, Marios, Fabien, Vassilis, George, Simos, Elina, and Dimitra—for keeping me grounded and connected despite the distance. Fabien has been a long-time collaborator and friend, as well as a constant source of support—keeping me both humble and deeply curious about research. I also thank the University of Crete and ICS-FORTH for helping shape my path prior to my PhD at UIUC.

I thank the teaching, administrative, and advising staff at UIUC and the Siebel School for their support over the years. I extend special thanks to Kara, Jennifer C., Jennifer F., Samantha, and Anthony for their consistent help, support, and kindness. I would also like to express my appreciation to the cleaning staff, whose kindness and positivity in our interactions—often during late nights at Siebel School—made somewhat difficult moments easier to endure.

I am immensely grateful to everyone I have met throughout my PhD journey who has shown kindness and support, and who, whether to a small or large degree, has helped make this experience better. I do not take any of it for granted.

Contents

Chapter 1	Thesis Overview	11
1.1	Performance and Programmability	12
1.2	Fault Tolerance in Distributed Systems	13
1.3	Fault Tolerance in Shared Memory Systems	14
1.4	Summary of Contributions	15
Chapter 2	Background	17
2.1	Consistency and Persistency Models	17
2.2	SmartNICs	20
2.3	Leader-Based Online Incremental Recovery	20
2.4	CXL Distributed Shared Memory	20
2.5	CXL Reliability	22
Chapter 3	MINOS: Distributed Consistency and Persistency Protocol Implemen- tation & Offloading to SmartNICs	23
3.1	Introduction	23
3.2	MINOS-Baseline Algorithms	25
3.3	MINOS-Baseline Performance	35
3.4	MINOS-Offload Design	36
3.5	Protocol Verification	43
3.6	Methodology	44
3.7	Evaluation	47
3.8	Related Work	54
3.9	Conclusion	55
Chapter 4	IASO: Online Incremental Recovery in Leaderless Distributed Systems with Different Persistency Models	56
4.1	Introduction	56
4.2	Motivation	59
4.3	Failure Model	60
4.4	IASO: Recovery in Leaderless Systems	62
4.5	Concurrent Failures	72
4.6	Protocol Verification	74
4.7	Methodology	75

4.8	Evaluation	77
4.9	Related Work	82
4.10	Discussion	83
4.11	Conclusion	84
Chapter 5	DIKTAMO: Extending CXL for Resilience to CPU Failures	85
5.1	Introduction	85
5.2	Motivation	87
5.3	Overview of DIKTAMO	88
5.4	Augmenting Ordinary Execution	91
5.5	DIKTAMO and Failures	100
5.6	Methodology	104
5.7	Evaluation	108
5.8	Related Work	116
5.9	Discussion	118
5.10	Conclusion	119
Chapter 6	Future Work	120
Chapter 7	Conclusion	123
References		125

Chapter 1 Thesis Overview

Modern cloud infrastructures rely heavily on distributed systems such as key-value stores, file systems, and databases to deliver high performance, availability, and resilience to failures [1], [2], [3]. These systems replicate data across multiple nodes [4], [5], [6], [7], enabling continued operation even when some machines become unavailable. Ensuring that replicated data remains correct and durable in the presence of failures is a fundamental challenge in the design of distributed systems.

Traditionally, many distributed storage systems follow a *leader-based* architecture, where a single node coordinates updates and enforces ordering across replicas [4], [7]. In such systems, the leader determines the serialization order of operations and ensures that updates are replicated consistently across the cluster. While leader-based designs simplify reasoning about correctness, they often introduce performance bottlenecks and limit scalability due to the centralization of coordination.

To overcome these limitations, *leaderless distributed systems* have gained increasing attention [8], [9]. In leaderless architectures, any node may initiate updates without relying on a centralized coordinator. By eliminating the leader, these systems can reduce latency, improve scalability, and avoid single points of performance contention. However, leaderless designs introduce additional complexity in ensuring that replicated state remains correct and durable, particularly in the presence of failures.

A central aspect of these systems is the definition of *consistency models*, which specify when updates become visible across replicas. Popular models include Sequential consistency [10], Linearizability [11], [12], Eventual consistency [13], and Causal consistency [14]. These models define the ordering guarantees provided to applications and represent different trade-offs between performance and correctness.

In addition to consistency, distributed systems must ensure that updates are reliably persisted so that the system can recover from failures. *Persistency models* define when

updates must reach non-volatile storage and become durable [15], [16]. Examples include Synchronous persistency, Strict persistency, Read-Enforced persistency [17], Eventual persistency [16], and Scope persistency [18]. These models provide different durability guarantees and interact closely with consistency requirements.

Recent work has proposed *Distributed Data Persistency (DDP)* models [16], which decouple consistency and persistency guarantees for distributed systems. DDP models are particularly attractive for leaderless architectures because they define clear semantics for when updates become both visible and durable across nodes. However, while these models provide a useful theoretical framework, practical implementations and system-level evaluations remain limited.

In the era of Compute Express Link (CXL), multiple hosts can be interconnected through shared CXL memory with hardware-supported coherence [19]. This capability allows hosts to cache data locally, including modified (dirty) cachelines.

The vision of this thesis is to enable high-performance and fault-tolerant leaderless systems across both distributed and shared-memory architectures. Specifically, this thesis addresses challenges in three key dimensions: performance and programmability of leaderless protocols, fault tolerance in distributed systems, and resilience in emerging shared-memory architectures.

1.1 Performance and Programmability

One of the key challenges of leaderless distributed systems is realizing them efficiently in real systems. While leaderless designs eliminate centralized coordination, they often require additional communication among nodes to ensure consistency and persistence guarantees. As a result, naïve implementations can incur significant latency and coordination overhead. Another important concern is *programmability*. System designers and programmers must reason about how operations propagate across nodes and how durability guarantees interact with consistency models. Supporting multiple combinations of consistency and

persistence models further complicates system implementation.

To address these challenges, this thesis introduces *MINOS* (Chapter 3), a system that provides concrete implementations of consistency and persistence model combinations for leaderless distributed systems.

We first design a set of distributed algorithms implementing Linearizable consistency combined with several persistence models, including Synchronous, Strict, Read-Enforced, Eventual, and Scope persistence. These algorithms are implemented in a baseline system called *MINOS-Baseline* (MINOS-B), which we evaluate in a multi-node distributed system.

To further improve performance, we introduce *MINOS-Offload* (MINOS-O), a SmartNIC-based architecture that offloads key components of the distributed protocol to programmable network interface cards (NICs). By leveraging SmartNIC capabilities, MINOS-O reduces CPU overhead and accelerates communication-intensive operations. The design incorporates optimizations such as selective data coherence, message batching, and broadcast communication to further improve system performance.

1.2 Fault Tolerance in Distributed Systems

Fault tolerance is a fundamental requirement in distributed systems. When nodes fail, the system must reconstruct lost state while ensuring that consistency and persistence guarantees remain intact.

In leader-based systems, recovery is typically orchestrated by a leader node, which maintains global knowledge of the system state. In contrast, leaderless systems lack a centralized authority responsible for coordinating recovery. As a result, recovery can become significantly more complex. Questions such as who initiates the recovery process, how replicas synchronize their state, and how the system reconstructs the correct version of data must be addressed without relying on a central coordinator.

Existing recovery mechanisms often rely on transferring large portions of the database to rebuild a failed node’s state. Such approaches can introduce significant recovery latency

and degrade system throughput during the recovery process.

To address these limitations, this thesis introduces *IASO* (Chapter 4), a system that enables fast and efficient recovery in leaderless distributed systems. IASO provides an *incremental online recovery* mechanism that allows a failed node to reconstruct its state by gathering information from live replicas while the system continues operating. Rather than performing full database transfers, IASO collects only the necessary state information and rebuilds the failed node incrementally. This approach significantly reduces recovery latency and minimizes disruption to the running system.

1.3 Fault Tolerance in Shared Memory Systems

Recent advances in interconnect technologies such as Compute Express Link (CXL) enable the construction of distributed shared-memory systems in which multiple compute nodes access and cache remote memory. These architectures are inherently leaderless: any node may read or modify shared memory locations.

However, such systems introduce new reliability challenges. In particular, nodes may hold *dirty cached data* corresponding to modified memory lines that have not yet been written back to memory. If a node holding dirty data crashes, the system risks losing the only up-to-date copy of that data.

This problem raises fundamental questions for shared-memory systems: how can the system ensure forward progress when nodes fail, and how can it recover data that was temporarily stored in volatile caches?

To address these challenges, this thesis introduces *DIKTAMO* (Chapter 5), a hardware–software co-designed architecture that enables fault-tolerant write-back caching in CXL-based distributed shared-memory systems. DIKTAMO extends the coherence protocol to replicate modified cachelines across peer compute nodes. Each remote store is replicated to other nodes and temporarily logged in hardware, allowing the system to recover dirty cachelines after permanent node failures. The design incorporates protocol

optimizations that overlap replication with store execution, hiding replication latency and preserving performance.

1.4 Summary of Contributions

This thesis makes the following contributions:

- Introduces, MINOS a novel architecture for real-system implementation of leaderless consistency and persistency models. More specifically,
 - the design and implementation of MINOS-B, a set of leaderless distributed algorithms implementing various combinations of consistency and persistency models.
 - the MINOS-O SmartNIC-offloaded architecture and its evaluation demonstrating improved performance compared to software-only implementations.
- The IASO protocol for fast, online incremental recovery in leaderless distributed systems. This contribution includes the design and evaluation of IASO across multiple combinations of consistency and persistency models in a distributed cluster.
- The DIKTAMO architecture, which enables fault-tolerant write-back caching in CXL-based distributed shared-memory systems.

Together, MINOS, IASO, and DIKTAMO advance the design of leaderless distributed and shared-memory systems by improving performance, enabling efficient recovery, and providing resilience against node failures in datacenters.

Thesis Organization. The remainder of this thesis is organized as follows. Chapter 2 presents background on consistency and persistency models, SmartNICs, and CXL-based distributed shared-memory systems. Chapter 3 introduces MINOS, an architecture for

real-system implementation of various combinations of consistency and persistency models for leaderless distributed systems and explores SmartNIC offloading to improve performance. Chapter 4 presents IASO, which enables fast online incremental recovery in leaderless distributed systems. Chapter 5 introduces DIKTAMO, a hardware–software architecture that provides fault tolerance for write-back caching in CXL-based distributed shared-memory systems. Chapter 6 discusses directions for future research, and Chapter 7 concludes the thesis.

Chapter 2 Background

This chapter provides background on the key concepts used in this thesis. We first review various consistency and persistency models for leaderless distributed systems, and describe the message patterns of their theoretical design. We then summarize relevant system technologies, including SmartNIC offloading, recovery mechanisms in existing systems, and Compute Express Link (CXL)-based distributed shared memory.

2.1 Consistency and Persistency Models

Distributed storage systems maintain multiple copies of a record (i.e., replicas) across nodes to improve performance, availability, and resilience. The *consistency* model defines how updates are propagated to the replicas in volatile memory, while the *persistency* model defines when those updates become durable in each node’s storage. In general, strong consistency models update replicas eagerly, whereas weaker ones may allow stale reads; similarly, strong persistency models make updates durable eagerly, whereas weaker ones may delay persistence and risk data loss on failure.

A Distributed Data Persistency (DDP) model [16], [20] combines a consistency model and a persistency model. It is characterized by two key concepts: the *Visibility Point*, when an update becomes observable, and the *Durability Point*, when the update becomes durable and cannot be lost due to failure. The former is dictated by the consistency model, and the latter by the persistency model. The DDP framework supports multiple $\langle \text{consistency}, \text{persistency} \rangle$ combinations.

Prior work describes DDP models at a relatively high level [16], but does not provide the detailed machine-level algorithms needed to support them in practice. These protocols assume an environment where, for simplicity, a record is replicated in all nodes rather than in a subset, and where both reads and writes can be initiated from any node rather than from a single leader. This latter feature makes the protocols *leaderless*, which improves

generality, performance, and scalability, although at the cost of greater implementation complexity.

In this thesis, we focus on DDP models that combine Linearizable consistency [8], [11], [12], [21] with one of five persistency models: Synchronous [16], [20], Strict [15], [22], Read-Enforced [17], Eventual [16], [20], and Scope [16], [18], [20]. Space constraints prevent considering additional models.

Messages in DDP Models. Following Hermes [8], the node that initiates a write is called the *Coordinator*, while the rest are called *Followers*. On a write, the Coordinator sends an invalidation (INV) message to all Followers, carrying the new data and invalidating the old version. Followers respond with acknowledgment messages (ACKs), confirming that the update has been applied for consistency, persistency, or both. Once the required ACKs are received and the Coordinator has updated its local state, it sends validation (VAL) messages to the Followers to signal completion of the transaction. Depending on the DDP model, ACKs and VALs may have separate consistency and persistency forms, such as ACK_C/ACK_P and VAL_C/VAL_P. In Scope persistency, messages are additionally tagged with a scope identifier *sc*, e.g., [INV]*sc*.

Brief Model Definitions. *Linearizable Consistency (Lin)* imposes a total order on operations to volatile state across all nodes. In a write, the client receives a response only after the volatile replicas have been updated system-wide.

Synchronous Persistency (Synch) requires a write to be persisted in a node when the local volatile replica is updated. Under $\langle \text{Lin}, \text{Synch} \rangle$, the write response is returned to the client when all replicas have been updated and persisted. In practice, this occurs when the Coordinator has received all ACKs and persisted locally.

Read-Enforced Persistency (REnf) requires that all updated replicas be persisted before any of them can be read. Under $\langle \text{Lin}, \text{REnf} \rangle$, the write response is returned once all volatile replicas have been updated, i.e., when the Coordinator has received all ACK_Cs. Once the Coordinator has received all ACK_Cs and ACK_Ps, it knows that all replicas

are both updated and persisted; it then sends a VAL to all Followers, which may then allow reads to the record.

Eventual Persistency (Event) requires replicas to be persisted eventually, without delaying reads or writes waiting for persistence. Under $\langle \text{Lin}, \text{Event} \rangle$, the write response is returned once all volatile replicas have been updated.

Scope Persistency (Scope) groups reads and writes into scopes. A client marks the end of a scope with a [PERSIST]sc command. The model requires that the response to [PERSIST]sc be returned only after all writes in that scope have been persisted. Under $\langle \text{Lin}, \text{Scope} \rangle$, each write inside a scope returns once it has updated all volatile replicas, while the [PERSIST]sc returns only after all writes in the scope have updated both volatile and non-volatile state in all replicas. When the Coordinator receives all [ACK_P]sc messages, it informs the client and sends [VAL_P]sc messages to the Followers.

Strict Persistency (Strict) is the strongest persistency model considered. It requires a write to be persisted in all replica nodes before the client receives the write response, potentially even before all volatile replicas are updated. Unlike Synch, it decouples consistency and persistency using separate ACK_C/ACK_P and VAL_C/VAL_P messages. Under $\langle \text{Lin}, \text{Strict} \rangle$, the response is returned only when all replicas have been both updated and persisted.

The chosen $\langle \text{consistency}, \text{persistency} \rangle$ combination determines when a local read may observe the effect of a prior update. Because all models considered here use Lin consistency, an update must first complete across all nodes consistency-wise. This is known at the Coordinator when all consistency ACKs arrive, and at a Follower when the corresponding consistency VAL arrives. In addition, $\langle \text{Lin}, \text{Synch} \rangle$ and $\langle \text{Lin}, \text{REnf} \rangle$ impose persistency-related conditions before reads are allowed. In $\langle \text{Lin}, \text{Synch} \rangle$, the persistency requirement coincides with the consistency requirement. In $\langle \text{Lin}, \text{REnf} \rangle$, reads additionally require that all persistency ACKs have been received and the final VAL has been delivered.

2.2 SmartNICs

SmartNICs are network interface cards with programmable logic that offload processing from the CPU. They have been introduced to improve performance, and typically include low-end energy-efficient compute units with caches and on-board DRAM. SmartNICs commonly offload network-related processing [23], [24], but can also offload more complex tasks such as replication and persistency [25].

2.3 Leader-Based Online Incremental Recovery

ZooKeeper uses a leader-based protocol [26]. During a write, a quorum of nodes logs the update in local persistent storage. For recovery, ZooKeeper combines checkpointing with incremental replay. When a node fails, the leader uses its own log to recover a replacement node.

This recovery is online and incremental, but has three limitations. First, recovery is *reactive*: the recovering node sends the ID of the last record in its log (its *zxID*) to the leader, and the leader must then search its own log and extract all updates after that point. Second, the extracted updates may include multiple writes to the same record, even though only the most recent one is ultimately needed; nevertheless, all such updates are sent and replayed. Third, while recovering, the replacement node does not participate in ongoing transactions. New updates destined for it are buffered by the leader and sent in later batches, potentially repeating multiple times.

2.4 CXL Distributed Shared Memory

Compute Express Link (CXL) [19], [27], [28] is an emerging open standard high-performance interconnect. The latest versions of its *CXL.mem* protocol (CXL 3.0 and beyond) enable the compute nodes (CNs) of a cluster to directly access data in remote memory nodes (MNs) in a byte-addressable and cache-coherent manner. This unique capability enables CXL-enabled Distributed Shared Memory (CXL-DSM) systems, where

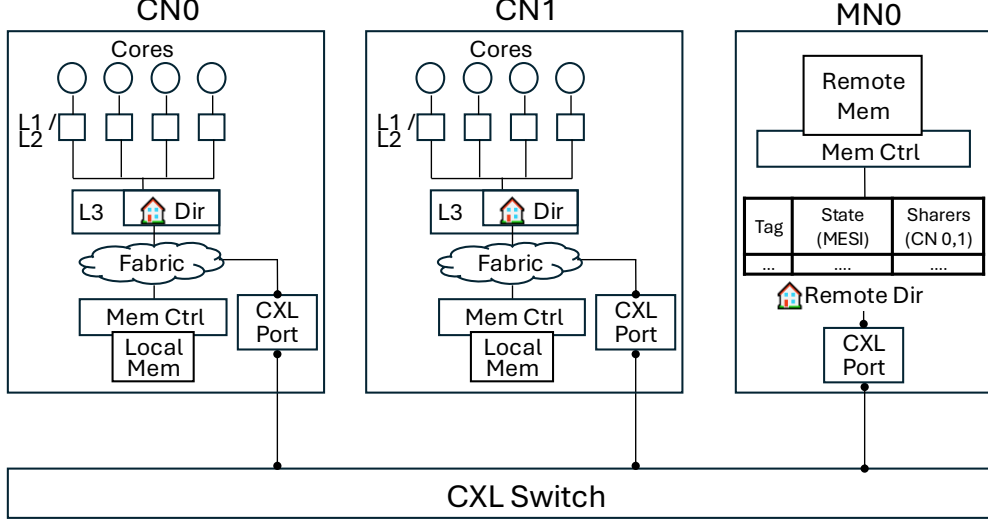


Figure 1: CXL-enabled distributed shared-memory system.

coherent data sharing across nodes is supported transparently by hardware, and at cache line granularity.

Figure 1 shows such a system with two CNs, one MN, and a CXL switch interconnecting them. The CNs have their local (private) memory hierarchies and all CNs are connected through the CXL switch to the (remote) memory in the MN. Routing hardware on the LLC miss path identifies if a miss is local or remote based on its target physical address. Remote accesses are routed to the CXL port. CXL uses write-back caching for both local and remote accesses, and a conventional cache coherence protocol like MESI [28]. The protocol is enforced by a hierarchical directory. Each CN has a directory that enforces cache coherence locally. Cached lines of remote memory are kept coherent across the CNs via a second-level directory that lives in the remote MN(s), i.e., the remote directory.

CXL is organized in three main protocol layers: a) the *Transaction Layer*, which defines the protocol transaction types and their ordering rules, b) the *Link Layer*, which ensures reliable transmission of the transaction packets, and c) the *Physical Layer*, which transmits data over the underlying interconnect.

2.5 CXL Reliability

In CXL-DSM clusters, faulty hardware can crash the execution of distributed shared-memory applications [29], [30], [31]. Thus, the CXL specification adds support for reliability, availability, and serviceability (RAS) [19], [32], [33]. It considers link, switch, and device (e.g., memory) failures. It detects link failures via timeouts, and retries packet transmissions to handle them. To detect data corruption, it relies on established techniques such as parity.

To contain errors, the CXL specification defines three complementary mechanisms. One is *Data Poisoning*, which allows devices to tag messages with corrupted data. A second one is *Isolation*, which allows the protocol to mark a faulty device and mask traffic from/to the device, keeping the rest of the system operational. The final one is *Viral Handling* which, when a fatal error is detected, sets a `Viral_Status` bit, propagating a “viral” state across all connected ports and tagging all traffic across ports. The specification [19] also introduces hardware commands to reset (re-initialize) CXL devices into a clean state after a viral handling.

This CXL RAS model has two limitations. The first one is that it does not consider processor (i.e., host) failures. In practice, handling processor failures is crucial, since CPUs and GPUs are reported to be failing with increased frequency in datacenters [34]. When a host fails, the dirty shared data it stores in its cache hierarchy is permanently lost [35].

The second limitation is that the CXL RAS model focuses only on containing errors. When a component fails, the specification tries to isolate it. There is no attempt to recover lost or corrupted data and bring the workload to a consistent state [35]. In reality, when a node participating in a shared-memory program fails and its dirty cached data is lost, continuing execution with an inconsistent state is not acceptable. The workload state must be corrected.

Chapter 3 MINOS: Distributed Consistency and Persistency Protocol Implementation & Offloading to SmartNICs

3.1 Introduction

A key component of modern cloud infrastructure is distributed storage systems, including key value stores, file systems, and databases [1], [2], [3]. To satisfy user needs, these systems must provide high performance, availability, and resilience to failures [4], [5], [6], [7]. To achieve these goals, they replicate data across multiple nodes. Such data replicas improve performance and allow the system to continue operation even if some of the machines become unavailable. However, supporting replicas requires the system to ensure their consistency when updates occur. For this purpose, different distributed *data consistency* models exist that describe when updates need to become visible to the replica nodes. Still, consistency models by themselves do not guarantee the resilience of the system, as they are not concerned with system recovery on a fault. To support recovery, data need to be persisted to non-volatile media. *Memory persistency* models describe when updates need to be persisted [15], [16].

Over the years, several distributed consistency models have been proposed, including Sequential [10], Linearizable [8], [10], [11], [12], [21], Eventual [13], Causal [14], [36], [37], and Transactional [4], [5], [38]. These models differ in the guarantees of replica consistency that they provide to the user, and offer different trade-offs between performance and programmer intuition. Similarly, several persistency models have been proposed, including Synchronous [16], Strict [15], Read-Enforced [17], Eventual [16], and Scope [16], [18]. Persistency models differ in the data persistency guarantees that they provide to the user, and offer different trade-offs between performance and durability. Finally, Distributed Data Persistency (DDP) models [16] combine consistency and persistency models, by introducing a unified framework for data consistency and persistency.

The operation of different DDP models was described by Kokolis et al. [16] at a

relatively high level. An important contribution of the DDP models is that they target *Leaderless* distributed systems [8]. These are systems where *any node in the system* can initiate read or write requests. Compared to leader-based systems, where all write requests need to be initiated by one leader node, leaderless systems deliver higher performance and are scalable. However, they are conceptually more involved. The DDP models have not been fleshed out in detailed algorithms.

In distributed computing systems, network interface cards (NICs) are responsible for the communication between servers and the network [24], [39], [40], [41], [42]. Traditionally, they are implemented as a PCIe card, and come with limited processing capabilities. However, recently, a new class of NICs with substantial processing and storage capabilities called SmartNICs have become available. SmartNICs have been used to offload operations such as packet processing, encryption, and compression from the CPU [25], [43], [44]. They could potentially offload DDP model protocols.

To improve the performance, availability, and durability of distributed systems, this chapter develops detailed distributed algorithms for DDP models and, additionally, offloads their operation from the CPUs to SmartNICs. Specifically, we take the DDP models outlined in Kokolis et al. [16] and develop detailed leaderless algorithms. We focus on Linearizable consistency with Synchronous, Strict, Read-Enforced, Eventual, or Scope persistency. We call these algorithms *MINOS-Baseline* (MINOS-B) and evaluate them on a 5-node distributed machine.

Then, we redesign the algorithms to offload them to a new SmartNIC architecture that we introduce. The SmartNIC architecture introduces several optimizations, including selective data coherence in hardware between host and SmartNIC, message batching, and message broadcasting. The resulting DDP algorithms and SmartNIC architecture are called *MINOS-Offload* (MINOS-O).

An evaluation of MINOS-O using simulations shows that offloading is very beneficial. It substantially reduces request latency and increases request throughput for various

workloads and number of nodes. For example, compared to MINOS-B, MINOS-O reduces the average end-to-end latency of two microservice functions by 35%. We also evaluate the impact of individual optimizations.

The contributions of this chapter are:

- The leaderless MINOS-B algorithms of several DDP models and their evaluation.
- The MINOS-O NIC-offloaded algorithms and SmartNIC.
- A comparative evaluation of the MINOS-B and MINOS-O algorithms.

3.2 MINOS-Baseline Algorithms

In this section, we introduce detailed algorithms for the efficient implementation of the DDP model protocols outlined by Kokolis et al. [16]. We call these algorithms *MINOS-Baseline* (*MINOS-B*). As already mentioned, we focus on models that combine Lin consistency with one of five persistency models. Next, we first introduce some definitions and then describe the protocol algorithms for writes and for reads.

3.2.1 Definitions

Our algorithms use *Locks* and *Logical Timestamps*. In this section, we define them and describe their use. In our description, we refer to two types of writes. A write transaction (i.e., operation) initiated by a client is called a “client-write”. During the transaction, there are substeps that involve updating the local memory subsystem of a node (e.g., the last-level cache (LLC)); we call such writes “local-writes”.

Locks. Our algorithms use two types of locks for the two types of writes: read locks (*RDLock_Owner* or *RDLock* for short) for client-writes and write locks (*WRLock*) for local-writes.

Figure 2(a) shows the metadata associated with a data record. *RDLock_Owner* tells whether the read lock for the record is taken and, if so, who the read lock owner is; *WRLock* tells whether the write lock for the record is taken. The other three fields relate

to the timestamps.

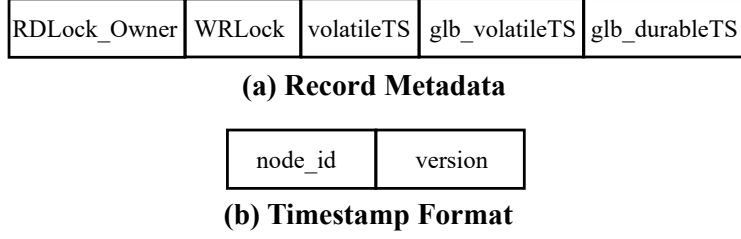


Figure 2: Record metadata and timestamp format.

The locks of a record are used as follows. In a given node, if one or more threads are attempting to perform a client-write (initiated locally or remotely) on a given record concurrently, one of them holds RDLock_Owner for the record. A taken RDLock_Owner prevents concurrent read transactions from accessing the record.

Assume that there is a single thread that is performing a client-write on the record. Such thread holds RDLock_Owner. During a client-write transaction, there is point when the thread, say $T1$, updates the local record (i.e., it performs a local-write). During this time, $T1$ must grab and hold the WRLock to ensure that no other client-write transaction to the same record tries to write concurrently. Once $T1$ finishes the update of the local record, it releases the WRLock and continues with the client-write operation. Once $T1$ completes the client-write operation, if it still holds RDLock_Owner, it releases it.

It is possible that a second thread, say $T2$, wants to perform a concurrent client-write on the same record while $T1$ is executing its client-write. If $T2$ has a higher timestamp (i.e., comes later in time), our algorithm allows $T2$ to “snatch” RDLock_Owner from $T1$, by updating RDLock_Owner. $T1$ continues execution and can even attempt and grab the WRLock. However, as we will see later, $T1$ will not be able to update the local record to a stale value. The benefit of snatching RDLock_Owner is that it will ensure that $T2$ ’s completion will not be delayed by $T1$ ’s completion. $T2$ is the only thread that can release RDLock_Owner. When it does, the record can be read by other threads.

Logical Timestamps. In our design, each data record in the volatile local memory of a

node keeps three logical timestamps [45]: *volatileTS*, *glb_volatileTS*, and *glb_durableTS* (Figure 2(a)). *volatileTS* describes the record’s version in the local volatile memory; *glb_volatileTS* describes the record’s global version in the machine-wide volatile memory as determined by the consistency model; and *glb_durableTS* describes the global version in the machine-wide non-volatile memory as determined by the persistency model. As shown in Figure 2(b), each timestamp is a tuple with a node identifier (*node_id*) and a version number.

When a node initiates a client-write, it generates a new timestamp for the write. The timestamp’s *node_id* is set to the Coordinator node ID, while the version is set by reading the *volatileTS* version of the record in the Coordinator node and adding one (without updating *volatileTS*). The resulting timestamp is included in a field called TS_{WR} in all the messages sent for this client-write. Moreover, TS_{WR} is eventually applied to the timestamp metadata of the record in all the nodes (including the Coordinator). Specifically, in any node, as soon as the volatile record is updated, the *volatileTS* is set to TS_{WR} . Similarly, once the volatile record is updated across all replicas, the *glb_volatileTS* is set to TS_{WR} , and when the durable record is updated across all replicas, the *glb_durableTS* is set to TS_{WR} . Note that TS_{WR} is unique for a write transaction.

Writes to the same record are ordered from older to newer based on their timestamp. Given two writes, the newer one is the one that has the higher version field or, if the versions are the same, the one with the higher *node_id*.

In our implementation, *RDLock_Owner* has the same format as a timestamp. Hence, it is a tuple of $\langle \text{node_id}, \text{version} \rangle$. When *RDLock_Owner* is acquired, it is atomically set to the client-write’s TS_{WR} . When *RDLock_Owner* is released, it is atomically set to $\langle -1, -1 \rangle$.

Outdated Writes. It is possible that, when a Coordinator or a Follower node is about to process a write transaction $WR1$ with timestamp τ_1 , the *volatileTS* timestamp of the local record already has a newer timestamp τ_2 , where $\tau_2 > \tau_1$. In this case, we would like to cut

WR1 short: return to the client right away and not inform other nodes (if the local node is the Coordinator) or return an ACK to the Coordinator right away (if the local node is a Follower). In either case, we want to skip updating the record’s volatile and non-volatile data.

Before we can do this, however, we need to ensure that the new transaction *WR2* that renders *WR1* obsolete has reached a correct state. There are both consistency and persistency concerns. Consider consistency first, recalling that we are only concerned with Lin consistency. Before cutting *WR1* short, we need to make sure that *WR2* has updated the volatile record in all the nodes and, therefore, no read will see the value before *WR2*. To ensure this, the Coordinator node for *WR2* must have received consistency ACKs from all the nodes in the system, and the Follower nodes for *WR2* must have received a consistency VAL. Note that *WR2* has already updated the local volatile record (as reflected by the volatileTS). Consequently, if the local node is the Coordinator for *WR2*, before cutting *WR1* short, the thread processing *WR1* must spin until all the *WR2* consistency ACKs are received from all the nodes. Similarly, if the local node is a Follower for *WR2*, before cutting *WR1* short, the thread processing *WR1* must spin until the *WR2* consistency VAL has been received. In other words, in both cases, the thread must spin until `glb_volatileTS` in the local record is updated. We define a primitive to perform this operation called *ConsistencySpin*.

A similar analysis is performed for persistency issues, and we define a primitive called *PersistencySpin*. For brevity, we do not discuss the details.

We also define the primitive *Obsolete*, which compares the timestamp of a client-write to the volatileTS timestamp of the local record. If the client-write has an older timestamp, the primitive returns true; otherwise, it returns false.

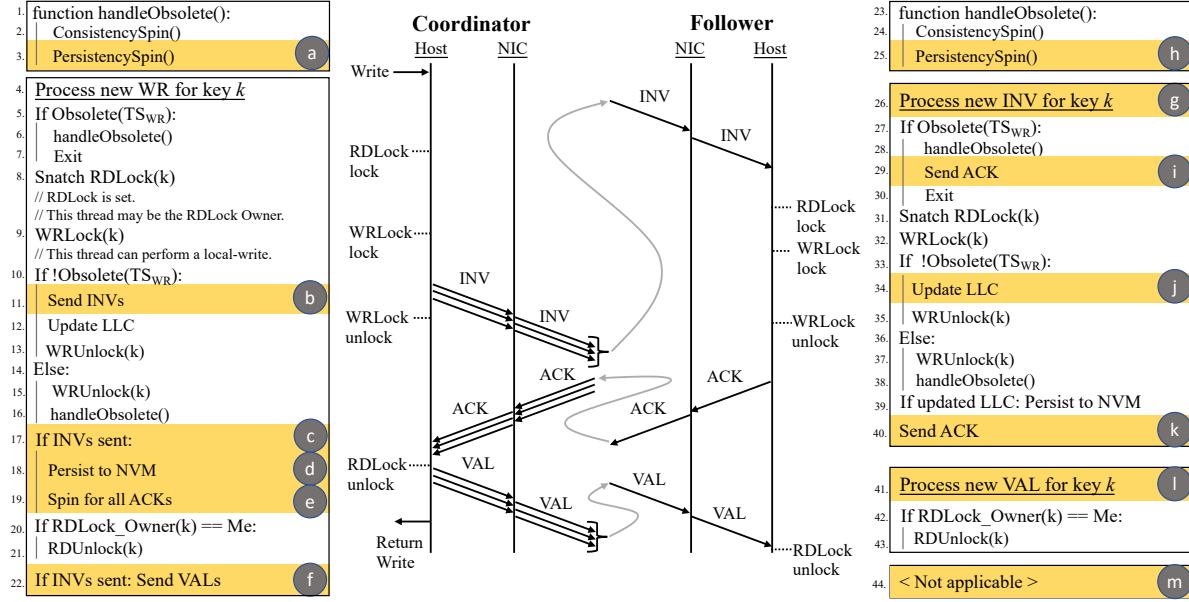


Figure 3: Detailed algorithm for the <Lin, Synch> model, with the Coordinator on the left side and the Follower on the right side. The steps in the algorithm that may change for different persistency models are highlighted.

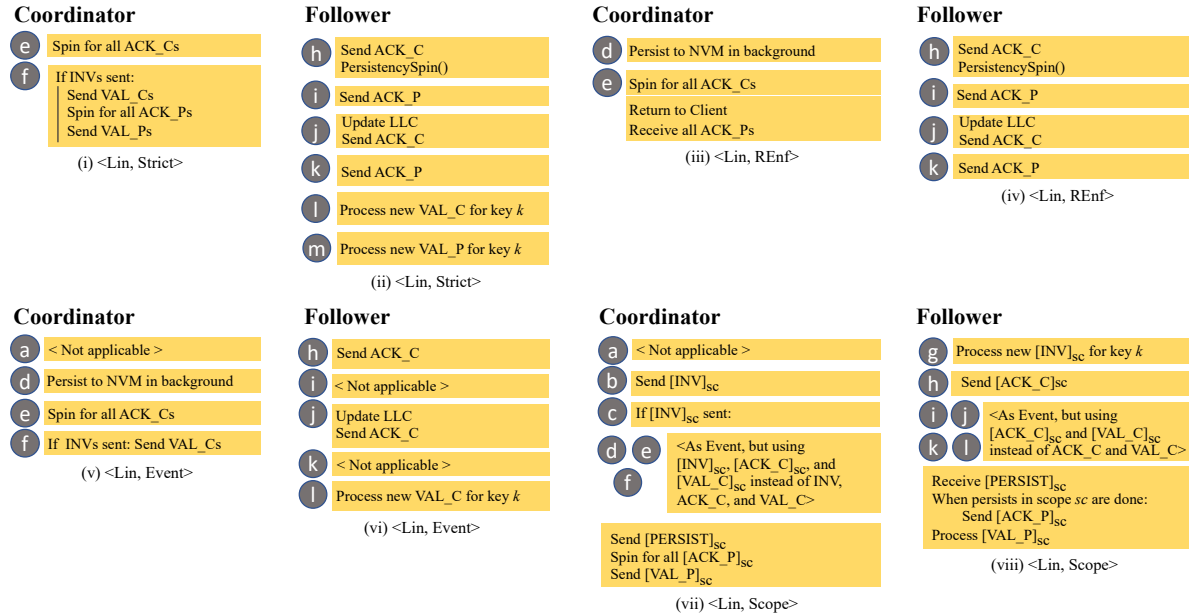


Figure 4: Differences of Strict, REnf, Event, and Scope persistency over Synch persistency (Figure 3). If a letter is not present, it means that the step does not change from Figure 3.

3.2.2 Protocol Algorithm for Writes

The write algorithm is similar in different persistency models. Consequently, we describe the algorithm for <Lin, Synch> in detail and then the differences for the rest of the

persistence models. Figure 3 shows the algorithmic steps (Coordinator on the left side and Follower on the right side) as well as the messages exchanged in the protocol for $\langle \text{Lin}, \text{Synch} \rangle$. We use a helper function called “handleObsolete” (Lines 1-3, 23-25) to run the ConsistencySpin and the PersistenceSpin. We start by describing the Coordinator steps, and then the Follower steps.

Coordinator. When the Coordinator receives a write request WR1 for key (i.e., record) k (Line 4), a timestamp TS_{WR} is generated as explained before. The algorithm first checks if WR1 is *Obsolete* (Line 5). If the check returns true, it means that there is another outstanding write WR2 to the same record that is more recent. Hence, WR1 should be skipped and execution should return to the client. Before doing so, however, the algorithm calls *handleObsolete* (Line 6), which performs the *ConsistencySpin* (Line 2) and *PersistenceSpin* (Line 3) operations.

If WR1 is not obsolete, it proceeds to perform a “Snatch RDLock” operation on key k (Line 8), which consists of the following: (i) if RDLock is free, WR1 grabs the lock; (ii) if RDLock is taken by an older write, WR1 snatches the RDLock from it; and (iii) if RDLock is taken by a younger thread, WR1 simply continues without grabbing the RDLock. This third case is fine because, in the presence of a younger write, we will see that WR1 cannot create an inconsistent state. What is important is that the RDLock be held by the youngest concurrent WR transaction to the record at a time, and that the owner of the RDLock is the one that later releases it. With such arrangement, read transactions to the record are prevented from proceeding concurrently with LLC updates, and LLC updates always produce a consistent state. Note that if WR1 successfully grabs the RDLock, a younger client-write WR2 may later snatch the RDLock from WR1; as we will see, correctness is guaranteed in all cases.

At this point, WR1 spins until it is able to grab the WRLock of key k (Line 9). Obtaining WRLock is essential because WR1 is about to perform a local-write to the record. Before performing the local-write, WR1 performs a final timestamp check (Line 10) to see if, since

Line 5, WR1 has become obsolete. If WR1 remains the latest write, the algorithm sends the INV messages (Line 11) to all Followers and updates the local, volatile state of the record in the LLC (Line 12). When the local volatile state is updated, the local *volatileTS* (not shown) is also updated. After these steps, the WRLock is released (Line 13). Instead, if WR1 has been made obsolete by another client-write, WR1 first releases the WRLock (Line 15) and then calls *handleObsolete* (Line 16). Releasing WRLock as soon as possible improves performance, as a concurrent write can grab it while WR1 performs additional operations. Holding the WRLock is not needed when *handleObsolete()* is invoked.

If INVs were sent (Line 17), the update is persisted to non-volatile memory (NVM) (Line 18). Moreover, the thread spins until all ACKs are received (Line 19). Note that, while the volatile state is always updated in increasing order of write TS_{WR} , the NVM can be updated by writes out of order. This is acceptable because we use a log structure for the persists.

When all ACKs are received, we know that the consistency and persistency requirements are satisfied across all nodes, hence the *glb_volatileTS* and *glb_durableTS* are also updated. Therefore, if WR1 remains the RDLock_Owner (which means that there is no other more recent write to the same record in the node), the read lock can be released (Lines 20-21). Finally, if INVs were sent earlier (in Line 11), VALs are now sent to all Followers (Line 22), which mark the completion of the write transaction. After this step, execution returns to the client.

Follower. When a Follower receives the INV for a write request WR1 to key k and timestamp TS_{WR} (Line 26), the algorithm performs the following steps. First, it checks if WR1 is obsolete (Line 27). If so, it calls *handleObsolete* (Line 28 and Lines 23-25), and then responds to the Coordinator with an ACK (Line 29) as if the write was done. This completes the transaction for an obsolete WR1 (Line 30). A VAL will be received later on but will be discarded.

If WR1 was not obsolete, the algorithm executes similar steps as in the Coordinator

(Lines 31-38), namely, WR1: (i) performs a “Snatch RDLock” operation on key k , (ii) grabs the WRLock, (iii) checks again if WR1 is obsolete and it either updates the local state in the LLC (and the *volatileTS*) and releases the WRLock, or releases the WRLock and invokes *handleObsolete*. Then, if the LLC was updated (Line 34), WR1 persists the update to NVM (Line 39). Next, WR1 responds to the Coordinator with an ACK, signifying that the write is performed consistency- and persistency-wise (Line 40).

Read transactions in the Follower node cannot yet see WR1’s update. Only after the Follower receives the VAL message from the Coordinator (Line 41) can the owner of the RDLock release it (Lines 42-43). Upon the release of the RDLock, any read transaction can read the key. Furthermore, at this point, the *glb_volatileTS* and *glb_durableTS* are also updated to reflect that the write is performed in all the replica nodes consistency- and persistency-wise.

3.2.3 Algorithm for Writes for Other Persistency Models

In Figure 3, we highlight with a shade and a letter some steps of the $\langle \text{Lin}, \text{Synch} \rangle$ algorithm. These are the steps that may change as we keep Lin consistency but move to other persistency models. All other steps remain unchanged. As explained in Chapter 2, some models have different INV, ACK, and VAL messages for consistency and persistency enforcement.

Figure 4 shows the changes to the algorithm for the combinations of Lin consistency and other persistency models. The text next to a letter in Figure 4 replaces the text next to the same letter in Figure 3.

For $\langle \text{Lin}, \text{Strict} \rangle$ Coordinator (Figure 4(i)), instead of using ACKs and VALs, the Coordinator spins for ACK_Cs in Step *e* and then, in Step *f*, sends VAL_Cs, spins for ACK_Ps, and sends VAL_Ps. Similarly, at the Follower side (Figure 4(ii)), instead of ACKs, the Follower sends ACK_Cs and ACK_Ps. First, if the received client-write is obsolete (Line 27), the algorithm performs *ConsistencySpin()*, sends ACK_C, and then

performs the `PersistencySpin()` and sends `ACK_P` (Line 29). If the client-write is not obsolete, after the LLC is updated, `ACK_C` is sent, and after the update is persisted to NVM, `ACK_P` is sent. Later, after `VAL_C` is received, the *RDLock* is released. Finally, after `VAL_P` is received in Step *m*, the write operation completes.

For the rest of the models, persisting the update to NVM is performed outside of the critical path. Consequently, Step *d* in the Coordinator (Line 18) is changed to persist NVM in the background, but line Line 39 in the Follower needs no change because the operation is not in the critical path: `ACK_C` has already been sent.

In $\langle \text{Lin}, \text{REnf} \rangle$ (Figure 4(iii)), Step *e* of the Coordinator involves waiting for all `ACK_C`s and then returning to the client. In the meantime, `ACK_P`s will arrive. When all `ACK_P`s are received, the *RDLock* is released and the `VAL`s are sent. At the Follower side (Figure 4(iv)), the only difference compared to *Strict* is that, since there is only one type of `VAL`, there is no change for Step *l* and there is no Step *m*.

In the rest of the models, the algorithm does not need to perform `PersistencySpin()` in Step *a* or Step *h*. This is because, in these weak models, accesses do not need to stall for the persist of prior writes that are still outstanding. However, `ConsistencySpin()` remains because of the Linearizable consistency.

In $\langle \text{Lin}, \text{Event} \rangle$ (Figure 4(v)), Step *e* of the Coordinator involves waiting for all `ACK_C`s. After that, the *RDLock* is released and then, in Step *f*, `VAL_C`s are sent before returning to the client. In this model, persistency will happen *eventually* and, therefore, there is no message exchange to track persistency. At the Follower side (Figure 4(vi)), in all cases, the Follower sends `ACK_C`s. Later, it receives a `VAL_C`. No persistency-related messages are exchanged.

In $\langle \text{Lin}, \text{Scope} \rangle$, the protocol steps are similar to $\langle \text{Lin}, \text{Event} \rangle$, although the message names are different. In addition, there is the new `[PERSIST]sc` transaction, shown in boxes without letters in Figures 4(vii) and (viii). The `[PERSIST]sc` transaction for scope *sc* consists of the following messages. The Coordinator sends the `[PERSIST]sc` to its

Followers, spins until it receives all the $[\text{ACK_P}]_{sc}$, and finally sends the $[\text{VAL_P}]_{sc}$ to its Followers, marking the end of the $[\text{PERSIST}]_{sc}$ transaction. A Follower, when it receives the $[\text{PERSIST}]_{sc}$, it completes persisting all the WR operations inside scope sc and the $[\text{PERSIST}]_{sc}$ request itself, and then responds with an $[\text{ACK_P}]_{sc}$ to the Coordinator. Later, it receives $[\text{VAL_P}]_{sc}$, which terminates the $[\text{PERSIST}]_{sc}$ transaction.

3.2.4 Protocol Algorithm for Reads

Like write operations, read operations can be initiated from any node. Since, in our environment, all records are replicated in all nodes, all read operations are satisfied locally. A read operation to a record is only stalled when the record's RDLock is taken by a write. Once the RDLock is free, the read operation can proceed. The handling of read operations is the same across all models.

3.2.5 Failure Detection and Recovery

While the DDP designs [16] do not discuss failure detection or recovery, MINOS-B has protocol extensions for failure detection and recovery that are similar for all the models considered. We assume that nodes can fail due to a crash or network disconnection and that, eventually, such nodes are re-inserted back into the cluster.

Failure detection is attained with timeout mechanisms [7], [8] that identify the non-responding node(s) (F) and alert all the other nodes. Later, when F is re-inserted back into the cluster, we need to bring F 's logs up-to-date. This is done by having a designated node send to F a message with the log of all the updates that have been committed since the time when F stopped responding. F then applies the updates to its local persistent and volatile state. More details of the recovery are left for future work.

3.3 MINOS-Baseline Performance

To understand the bottlenecks in the MINOS-B algorithms, we implement and run them on a cluster and measure their performance. We run them on a 5-node cluster where each node has a Xeon E5-2450 processor running at 2.1 GHz, and keep 5 cores busy per node. We use the eRPC [46] library for communication. More details are discussed in §3.6.

We measure the average latency of a write transaction (Figure 3). This is because, in our environment, write operations are much costlier than reads. We divide the latency into *communication* and *computation* times. Communication is the time taken by all the messages between hosts in a write transaction. For a given message, communication time starts after the sender host has deposited the message in its host send queue in memory, and finishes when the message has been deposited in the host receive queue in the memory of the receiver node. The communication time then includes the transfer of the message from the host send queue through the PCIe bus to the sender NIC, the NIC actions to send the message, the transfer of the message through the network, and the equivalent operations at the receiving NIC. The rest of the transaction is computation time, and can include LLC and NVM accesses.

Roughly speaking, the communication time in a write transaction is seen in Figure 3 as the time from when the first *INV* is sent (Line 11) until when the last *ACK* is received (Line 19), subtracting the average time it takes for a Follower to handle an *INV* message (Lines 26-40). For other models, the communication time may be accounted for slightly differently.

Figure 5 shows the average latency of a write transaction broken down into communication and computation times for the different <consistency, persistency> models. We see that the models with the more conservative persistency enforcement have higher write latencies. This is mostly because of their higher computation times—which are greatly affected by the overhead of persisting a record in the critical path of a write operation in the conservative persistency models. In addition, as multiple writes are

sometimes trying to access the same record concurrently, locks add overhead.

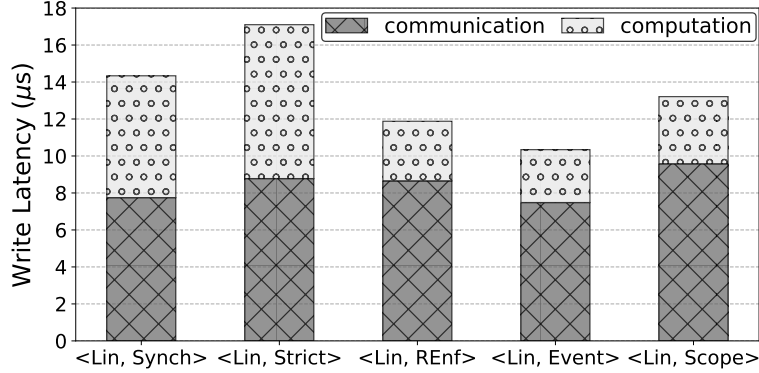


Figure 5: Average latency of a write transaction for different $\langle$ consistency, persistency $\rangle$ models.

The communication time varies less across models. However, it is the highest contributor to the write latency, contributing 51–73% to each model’s total write time. A major reason why communication time is so high is that the multiple INV messages in a transaction are sent one at a time. Indeed, they are taken one at a time from the send queue, transferred along the slow PCIe bus, and then sent out to the network. The same happens for the VAL messages. Current NICs lack the support to process batched messages or to use (true) broadcasting [47] of messages in the network. As we scale the protocols to many nodes, this bottleneck is likely to get worse.

3.4 MINOS-Offload Design

To reduce the latency of write transactions in MINOS-B, we propose to offload supporting the consistency and persistency model protocols from the host CPU to a SmartNIC. Based on the characterization of the latency of write transactions from Section 3.3, we propose modified algorithms to enforce consistency and persistency models and a SmartNIC architecture to support them. We call the algorithms and the SmartNIC architecture MINOS-Offload (*MINOS-O*). MINOS-O substantially reduces both the communication and the computation time of write transactions. In this section, we outline the SmartNIC architecture, present mechanisms to minimize communication and computation time, and

describe the overall MINOS-O algorithms to support $\langle \text{consistency, persistency} \rangle$ models. The same SmartNIC design supports all the models discussed in this chapter.

3.4.1 MINOS-O SmartNIC Architecture

The MINOS-O SmartNIC architecture is broadly based on the Mellanox Bluefield Data Processing Unit (DPU) [48]. Figure 6(a) shows the top level diagram. Like other SmartNICs, MINOS-O includes multiple cores, three levels of caches, a DRAM module, and interfaces to the network and the host. The new components that MINOS-O adds are: (i) the host interface is augmented with a module that provides some selective cache coherence between the L3 cache of MINOS-O and the L3 cache of the host; (ii) the network interface includes a module that supports message broadcast; (iii) the NIC has an NVM module that includes a durable FIFO queue (dFIFO) for the persistency protocol; and (iv) the DRAM module includes a volatile FIFO queue (vFIFO) for the consistency models.

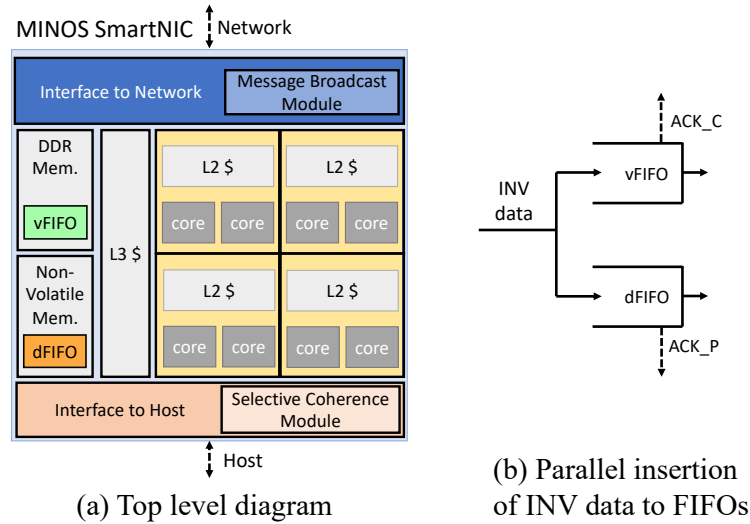
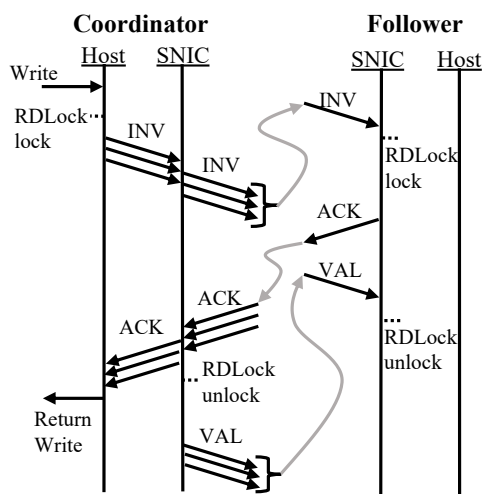


Figure 6: MINOS-O SmartNIC architecture.

3.4.2 Mechanisms to Improve Performance

We modify the distributed consistency and persistency algorithms of §3.2 to leverage the SmartNIC architecture. In this section, we describe the new mechanisms proposed to

1. Offloading Operations to the SmartNIC. We offload the execution of most of the Coordinator and Follower write algorithms of Figure 3 from the host to the SmartNIC (SNIC). Figure 7 shows the high-level operation. The algorithm changes little from that in Figure 3. On the Coordinator side, the host starts the write transaction (Lines 1-8 in Figure 3). After snatching the RDLock, it sends the multiple INVs to the SmartNIC. The SmartNIC takes over starting at Line 9 and executes the rest of the algorithm. Every time an ACK is received, it is passed to the host. Once the SmartNIC has received all of the ACKs, it proceeds to release the RDLock and send the VALs.



In the Follower, all the operations (Lines 23-44) are performed in the SmartNIC. The host is not invoked. Note that a SmartNIC can reject a request from its local host or from the network if it runs out of resources.

2. Coherence between Host and SmartNIC.

When we offload some operations of the

algorithm to the SmartNIC, we need to ensure that data which can be accessed concurrently by host and SmartNIC is kept coherent. MINOS-O enables fast sharing of some data structures by keeping them coherent between host and SmartNIC in hardware.

There are only four types of data structures that need to be kept coherent. They are four of the metadata fields of data records shown in Figure 2(a): `RDLock_Owner`, `volatileTS`, `glb_volatileTS`, and `glb_durableTS`. The `WRLock` is not included because, as we will see, MINOS-O does not use it.

Recall that `RDLock_Owner` (also called `RDLock` for short) for a record in a node is set when there is at least one ongoing write transaction to the record locally. As shown in Figure 7, there are four places in a write transaction where either the host or the SmartNIC can update `RDLock`. In addition, when a host receives a read transaction, it needs to check `RDLock`. MINOS-O enables fast access by providing coherence in hardware.

The `volatileTS`, `glb_volatileTS`, and `glb_durableTS` timestamps of a record are set in the Coordinator and Follower algorithms when the record in local volatile storage is updated, when the consistency of the record is enforced across all replicas, and when the persistency of the record is enforced across all replicas, respectively (§3.2.2). In MINOS-O, the code that updates these timestamps is executed by the SmartNIC.

These timestamps are read when the algorithm checks for the obsolescence of a write (i.e., $Obsolete(TS_{WR})$) and when performing the consistency and persistency spinning (i.e., $handleObsolete()$) (§3.2.2). In MINOS-O, the code that reads these timestamps is executed by the host and by the SmartNIC.

To provide hardware coherence for these four data structures, MINOS-O places them in a special range of addresses mapped to a small on-chip memory in both host and SmartNIC. A dedicated bus supports MSI snoop coherence between these two memories. This hardware is logically placed in the Selective Coherence Module of Figure 6(a).

3. Message Batching and Broadcasting. A source of overhead in distributed protocols that replicate data in multiple nodes is that, on a write operation, messages have

to be sent to update all the replicas. MINOS-O minimizes this overhead by supporting two mechanisms that have been proposed in other contexts. The first one focuses on minimizing the many messages that the Coordinator’s host and SmartNIC exchange through the slow PCIe bus. Indeed, as shown in Figure 7, the host sends as many INV messages as Followers, and the SNIC responds with as many ACKs as Followers. To reduce this overhead, MINOS-O performs *batching* [24], [49], [50], whereby the host sends a single INV message with information about which nodes should receive it, and the SmartNIC responds with a single ACK message when it has received all the ACKs. Later, the SmartNIC sends the VALs without host involvement.

The second mechanism addresses the SmartNIC overhead of preparing and sending the same INV and VAL messages to multiple Followers. Rather than considering these messages as completely different messages, MINOS-O provides special hardware and an RDMA verb [47] that broadcasts a message. Specifically, the SmartNIC deposits an INV or VAL message into the Send Buffer only once, and fills a Destination Map register. Then, an FSM broadcasts the message to all the destinations. This hardware is logically placed in the Message Broadcast Module of Figure 6(a). In contrast, the baseline approach requires depositing the same message on the Send Buffer multiple times.

4. Eliminating Write Locks. In the Coordinator and Follower algorithms of Figure 3, a thread must grab the WRLock for a record before it can update the local version of the record in local volatile memory. Grabbing the WRLock is needed for two reasons. First, it prevents record corruption when more than one thread are trying to update the same record concurrently. Second, locking is also needed to safely identify an obsolete write to the local volatile memory (thanks to the $Obsolete(TS_{WR})$ check), and avoid it.

Note that the update to the local non-volatile log is outside the WRLock critical section. The reason is that updates are deposited into the log in an atomic fashion. It is possible that entries are inserted into the log in an out-of-order manner, therefore creating obsolete entries. However, correctness is maintained because, before the log entries are

applied to the non-volatile database, they are checked for obsolescence.

Spinning on, grabbing, and releasing the WRLock adds overhead. To eliminate this overhead, MINOS-O adds special hardware that serializes the updates to the same record in the local volatile memory and skips obsolete updates to the local volatile memory. This hardware is the vFIFO queue. There is also the dFIFO queue that persists the updates locally in the SmartNIC, avoiding the need to push them to the host in the critical path. Both vFIFO and dFIFO queues are shown in Figure 6(b).

When a thread executing the Coordinator algorithm wants to write to a record, rather than grabbing the WRLock, it directly checks whether the write is obsolete. If it is not, the SmartNIC sends the INVs and writes to the vFIFO queue atomically and to the dFIFO queue atomically. When the hardware dequeues an entry from the vFIFO queue in the background, it checks for obsolescence before updating the LLC. If the entry is not obsolete, a DMA operation pushes the update to the host's LLC. Dequeueing can be done in parallel for updates to different records. When the hardware dequeues an entry from the dFIFO, it pushes it to the host NVM log. A thread cannot proceed to unlocking the RDLock until (1) the update has drained from the vFIFO to the LLC, and (2) all the ACKs have been received. There is no need to wait for the update to drain from the dFIFO because the update is already durable.

When a thread executing the Follower algorithm in the SmartNIC wants to write to a record, the process is similar. First, it checks whether the update is obsolete. If it is not, it writes to the vFIFO queue atomically and to the dFIFO queue atomically. Finally, it sends the ACK. When the hardware dequeues the entry from the vFIFO queue, it checks for obsolescence before updating the LLC; when the hardware dequeues an entry from the dFIFO, it pushes it to the host NVM log. After the update has drained from the vFIFO and the VAL has been received, the thread can proceed to unlocking the RDLock.

3.4.3 Overall MINOS-O Algorithms

When all the mechanisms are applied together, we obtain the MINOS-O algorithm.

Figure 8(a) shows the message exchanges of the MINOS-O algorithm for a write and a read using the $\langle \text{Lin}, \text{Synch} \rangle$ model. As shown in the figure, for a client-write, the Coordinator host grabs the RDLock and sends a batched INV message to its SNIC. The SNIC broadcasts the INV message to all the Followers. Afterwards, the SNIC enqueues the update in vFIFO and dFIFO. Later, when all the corresponding ACKs are received, the Coordinator's SNIC sends a batched ACK to the host, which marks the end of the client-write operation. After the corresponding data entry is drained from vFIFO, the SNIC releases the RDlock and broadcasts the VALs to all the Followers.

Figure 8(a) also shows that, in a Follower, when the SNIC receives an INV, a thread grabs the RDLock, enqueues the update in vFIFO and dFIFO, and returns an ACK. Later, when the Follower receives a VAL and the corresponding data entry is drained from vFIFO, the Follower releases the RDLock. In both Coordinator and Follower, a client-read can only proceed when the RDLock is free.

Figure 9 shows the detailed MINOS-O algorithm for the Coordinator and Follower for $\langle \text{Lin}, \text{Synch} \rangle$. The algorithm is organized as Figure 3 with the four MINOS-O optimizations applied.

3.4.4 Supporting Other Persistency Models

Figures 8(b)–(e) show the algorithms for the other persistency models. They are similar to $\langle \text{Lin}, \text{Synch} \rangle$ except for the following differences. First, ACKs and VALs can be for consistency (ACK_C, VAL_C) or persistency (ACK_P, VAL_P). Second, different models have slightly different logic, as discussed in Section 3.2.3 and shown in Figure 4. For example, different algorithms use different conditions to release the RDLock in the Coordinator and in the Follower, and to return to the client.

Note that, in some cases, we have separated the enqueueing to the vFIFO from the

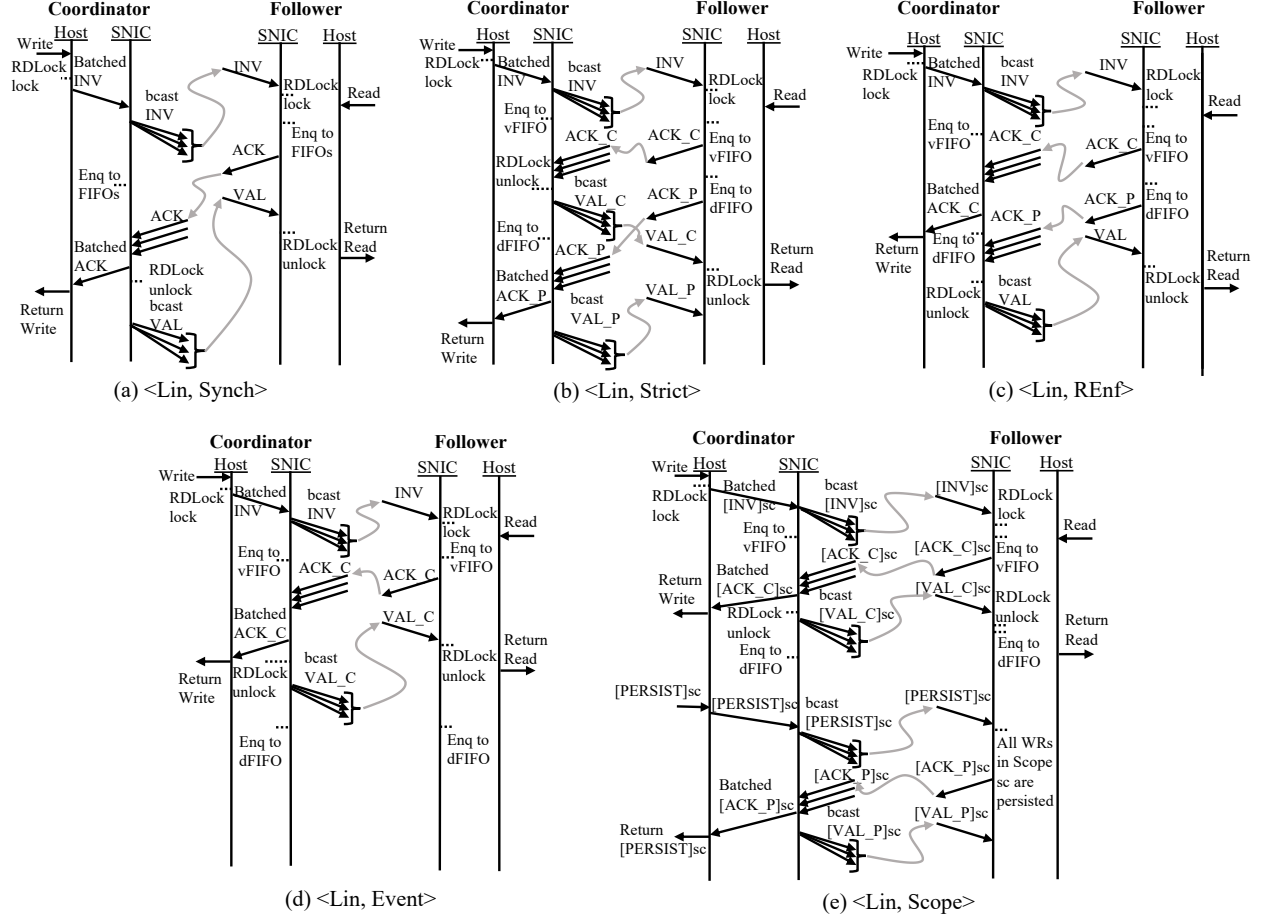


Figure 8: Timeline of the MINOS-O algorithms for consistency and persistency models using SmartNICs (SNICs).

enqueueing to the dFIFO. We do this when only the enqueueing to the vFIFO is in the critical path and, therefore, waiting for the slower enqueueing to the dFIFO in the critical path is unnecessary. For the $[PERSIST]_{sc}$ command of the $\langle \text{Lin}, \text{Scope} \rangle$ model, we do not include the bookkeeping operations in the volatile and non-volatile memory in order to simplify the picture.

3.5 Protocol Verification

To verify the correctness of the MINOS-B and MINOS-O protocols, we use the TLA+ formal specification and verification language, and model-check the protocols in TLC [51]. Our TLA+ work draws upon the Hermes [8] TLA+ design. With TLA+, we specify all the

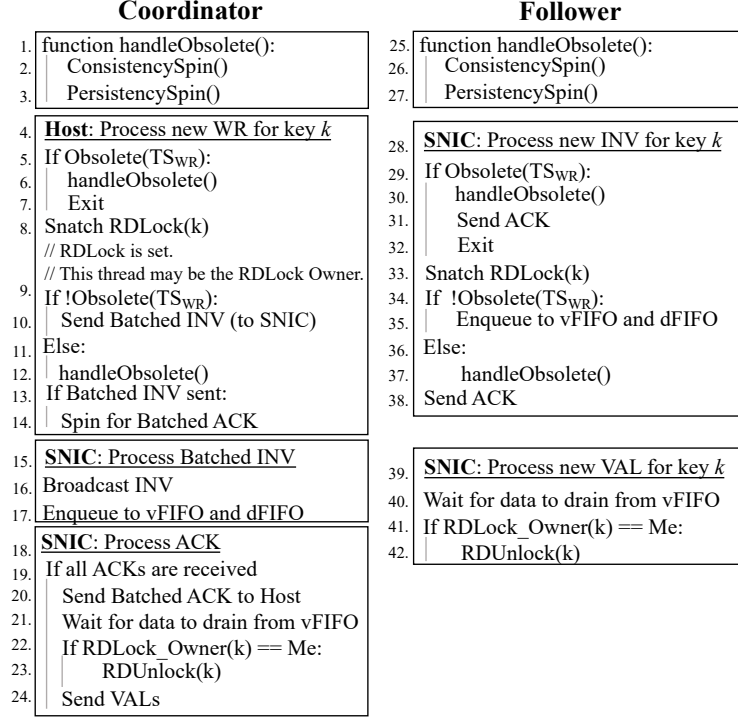


Figure 9: MINOS-O algorithm for $\langle \text{Lin}, \text{Synch} \rangle$.

states and possible actions from all the states, and then check that certain correctness conditions hold. We model-check all $\langle \text{consistency}, \text{persistency} \rangle$ models analyzed.

Table 1 shows the correctness conditions checked for all the $\langle \text{consistency}, \text{persistency} \rangle$ models analyzed. We first check two concurrency conditions: no deadlock and no livelock. Next, for data consistency correctness, we check three invariants related to the values of the volatileTS and glb_volatileTS metadata of records. Next, for data persistency correctness, we check two invariants involving the value of glb_durableTS. Finally, we perform a set of type checks that ensure: a) only legal messages are sent, b) record metadata and locks take only legal values, and c) bookkeeping data in write operations take only legal values.

3.6 Methodology

We evaluate MINOS-B using a distributed machine. Since the proposed MINOS-O hardware does not exist, we evaluate MINOS-O and also MINOS-B (for comparison) using a simulated distributed machine.

1. Concurrency Checks
Absence of deadlocks and livelocks.
2. Consistency Checks
a) When a record is read-unlocked in all nodes, the volatileTS and glb_volatileTS of the record are the same across all nodes.
b) When all ACKs for consistency have been received for a write to a record, the volatileTS of the record is the same across all nodes.
c) When not all ACKs for consistency have been received for a write to a record, the glb_volatileTS of the record is the same across all nodes.
3. Persistency Checks
a) When a record is read-unlocked in all nodes, the glb_durableTS of the record is the same across all nodes.
b) When not all ACKs for persistency have been received for a write to a record, the glb_durableTS of the record is the same across all nodes.
4. Type Checks
a) Each message $\in \{INV, ACK, ACK_C, ACK_P, VAL, VAL_C, VAL_P, [INV]_{sc}, [ACK_C]_{sc}, [ACK_P]_{sc}, [VAL_C]_{sc}, [VAL_P]_{sc}, [PERSIST]_{sc}\}$
b) Record metadata: i) For volatileTS, glb_volatileTS, and glb_durableTS: version $\in \{0 \dots (MAX_VERS-1)\}$, node_id $\in \{0 \dots (MAX_NODES-1)\}$ ii) For RDLock_Owner: version $\in \{-1 \dots (MAX_VERS-1)\}$, node_id $\in \{-1 \dots (MAX_NODES-1)\}$ iii) WRlock $\in \{0, 1\}$
c) Bookkeeping: $\forall$ node_id, $\{RcvdACK_SenderID[node_id], RcvdACK_C_SenderID[node_id],$ and $RcvdACK_P_SenderID[node_id]\} \in \{0 \dots (MAX_NODES-1) \setminus node_id\}$

Table 1: Conditions checked using TLA+ for all the <consistency, persistency> models analyzed.

Distributed Machine. We implement MINOS-B in a 5-node cluster provided by CloudLab [52] with the parameters shown in Table 2. Since we do not have access to a modern persistent memory device, we use information from prior works [16], [53], [54] and assume 1295ns to persist 1KB of data. One way to exchange messages between nodes is to use one-sided Remote Direct Memory Access (RDMA), which is the state-of-the-art paradigm for server-to-server communications in datacenters [55], [56], [57], [58]. However, in our protocol, a message received by a node performs multiple operations before triggering a response to the requester node. Using RDMA would require multiple messages to perform these multiple operations, resulting in performance overhead. Hence, we instead use eRPC [46], [59] for communication, which provides performance similar to RDMA.

This distributed machine was used in the experiments of Section 3.3 and to calibrate

Number of nodes	5
CPU per node	Xeon E5-2450 (5 cores, 2.1 GHz)
Main memory per node	16GB of DRAM (DDR3-1600)
NIC per node	Mellanox MX354A FDR CX3
Emulated NVM per node	1295 ns to persist 1KB of data

Table 2: Distributed machine running MINOS-B.

the parameters of the simulated distributed machine.

Simulated Distributed Machine. The simulator we use is SimGrid [60], [61], which is an accurate and scalable simulator for distributed systems. We model a distributed architecture similar to the CloudLab one with 2, 4, 5 (default), 6, 8, 10, or 16 nodes. The various access latencies of the memory hierarchy of the host are set based on measurements of the CloudLab system. The various overheads and latencies in the SmartNIC are set based on measurements of the BlueField-2 HDR100 100Gb/s SmartNIC [62] of the Thor cluster at the HPC-AI Advisory Council Cluster Center [63]. Other parameters of the SmartNIC and of the communication between SmartNIC and host or between SmartNICs are obtained from the literature [25], [64], [65] or from measurements on the CloudLab system.

Some parameters used in the simulator are shown in Table 3. The synchronization latency is the average latency to perform a compare-and-swap. The table also shows the latencies to send an INV and an ACK message. Further, it shows the time between consecutive messages when sending the same INV to a set of Followers without broadcasting.

In addition to running the MINOS-O algorithms on the simulated distributed architecture, we also run the MINOS-B algorithms on a simulated model of the CloudLab distributed machine. With our parameters, MINOS-B performs similarly in both the real and the simulated machine.

Workloads Used. To support our proposed metadata format (Figure 2), we implement our own key-value store, named MINOS-KV. Requests are generated using a C++ version

Number of Nodes	2, 4, 5 (default), 6, 8, 10, or 16	
Node	Host	SmartNIC
Number of Cores	5	8
Core frequency	2.1 GHz	2 GHz
Synchronization latency	42 ns	105 ns
Communication Link	Latency	BW
PCIe between Host and SmartNIC	500 ns [25]	6.25 GB/s [64]
Network link between SmartNICs	150 ns	7 GB/s [65]
MINOS-O Parameters		
vFIFO & dFIFO latency (wr 1KB)	465 ns and 1295 ns	
vFIFO & dFIFO size	5 and 5 entries	
Send one INV and send one ACK	200 ns and 100 ns	
Time between consecutive msgs	100 ns (with no broadcast support)	

Table 3: Parameters of the simulated system.

of Yahoo! Cloud Serving Benchmark (YCSB) [66], [67]. The back-end in-memory application used is a Hashtable [3].

We use various workloads with different write and read ratios. The database of each node has 100,000 records. The default workload uses a zipfian distribution for keys, has 50% write and 50% read operations, and issues 100,000 requests per node. We use a record size of 1KB, which is the default in YCSB. In some experiments, we use the DeathStar [68] benchmark suite for microservices.

All nodes contain replicas of all records. Therefore, all write operations initiated in a node need to be propagated to all other nodes. Read operations are always local.

3.7 Evaluation

In this section, we compare MINOS-B and MINOS-O under a variety of conditions, evaluate the impact of the MINOS-O optimizations, and perform a sensitivity analysis of some parameters. We compare these two designs as if a MINOS-O SmartNIC was available and a company was able to incorporate it to enhance its baseline design. An alternative comparison would involve starting with an enhanced baseline design that has the same total hardware as MINOS-B. Specifically, since MINOS-O adds 8 additional cores in the SmartNIC, this iso-hardware comparison would enhance MINOS-B with 8 additional cores per host. We leave such an iso-hardware comparison to future work.

3.7.1 Comparing MINOS-B and MINOS-O

Figure 10 compares the latency and throughput of client writes (a) and client reads (b) under various conditions in MINOS-B and MINOS-O. In each chart, the latency is represented with bars and is measured in the left Y-axis, while the throughput is represented with triangles and is measured in the right Y-axis. The bars/triangles are organized in groups corresponding to the different $\langle \text{consistency, persistency} \rangle$ models. For each model, the different bars/triangles correspond to workloads with 20%, 50%, 80%, and 100% of writes or reads. In each figure, the bars and triangles are normalized to MINOS-B with $\langle \text{Lin, Synch} \rangle$ and 50% writes or reads.

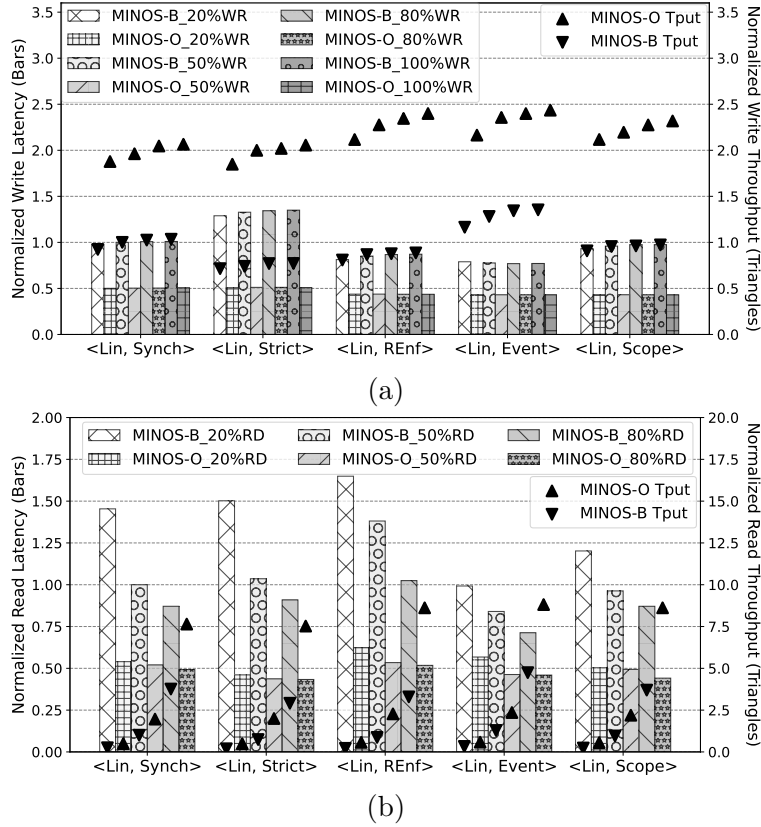


Figure 10: Normalized latency (bars and left Y-axis) and throughput (triangles and right Y-axis) of writes (a) and reads (b) for MINOS-B and MINOS-O for different workloads.

Consider the write transactions first (Figure 10(a)). For all the workloads and $\langle \text{consistency, persistency} \rangle$ models, MINOS-O typically reduces the average write latency

by 2-3x over MINOS-B, and increases the average write throughput by 2-3x over MINOS-B. These are major improvements. Also, MINOS-O is much less sensitive to the persistency model than MINOS-B. In addition, as the fraction of writes increases, MINOS-O's throughput increases, but its latency barely changes. In contrast, MINOS-B's throughput typically improves little with higher fraction of writes. Overall, MINOS-O is a high performance scheme, robust to changes in consistency and load.

Consider now read transactions (Figure 10(b)). The trends are similar. MINOS-O often reduces the average latency by 2x or more over MINOS-B, and increases the average throughput by 2x or more over MINOS-B. For both schemes, as the fraction of reads increases, the read latency decreases and the throughput increases. This is due to the high cost of writes.

On average across models and workloads, MINOS-O's write and read latency are 2.1x and 2.2x lower than MINOS-B's, respectively; MINOS-O's throughput is 2.3x higher than MINOS-B's for both writes and reads.

3.7.2 Comparison for Different Node Counts

Figure 11 compares the latency and throughput of client writes (a) and client reads (b) in MINOS-B and MINOS-O for different node counts (2, 4, 6, 8, and 10). The figure is organized as Figure 10. In each figure, the bars and triangles are normalized to MINOS-B with $\langle \text{Lin}, \text{Synch} \rangle$ and two nodes.

We again see the effectiveness of MINOS-O. As the number of nodes increases, MINOS-O rapidly increases the throughput, while keeping latency increases modest (for writes) or non-existing (for reads). The write latency increases are due to higher contention. In contrast, as the number of nodes increases, MINOS-B increases the latency quickly and is typically unable to improve the throughput.

On average across models and node counts, MINOS-O's write and read latency are 2.3x and 3.1x lower than MINOS-B's; MINOS-O's throughput is 2.4x higher than MINOS-B's

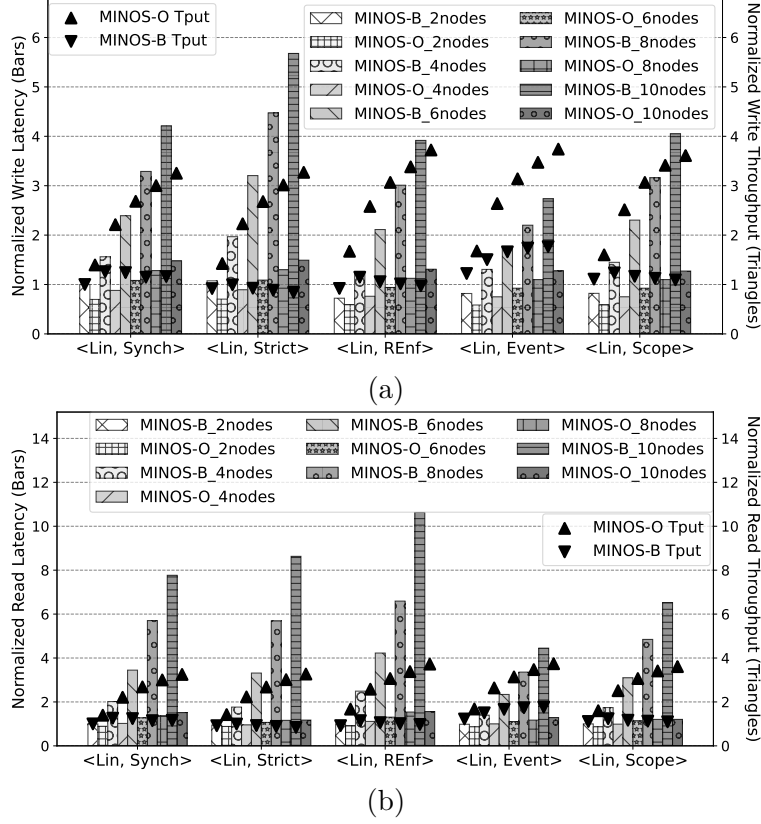


Figure 11: Normalized latency (bars and left Y-axis) and throughput (triangles and right Y-axis) of writes (a) and reads (b) for MINOS-B and MINOS-O for different node counts.

for both writes and reads.

3.7.3 Comparison for Real Applications

We compare the end-to-end latency of running DeathStar [68] functions on MINOS-B and MINOS-O. We evaluate the *Login* function of the *UserService* microservice in the *Social Network* and *Media Microservices* applications. In each SET and GET operation, we invoke our client-write and client-read algorithm, respectively. We assume a node-to-node round-trip latency of $500\mu s$, which has been measured in datacenters [69]. We model a cluster with 16 nodes.

Figure 12 shows the end-to-end latencies for MINOS-B and MINOS-O. The bars are grouped by <consistency, persistency> model. For each model, there are bars for the two functions and for MINOS-B and MINOS-O. The bars are normalized to <Lin, Synch>

MINOS-B and *Social*. From the figure, we see that MINOS-O reduces the end-to-end latency across the board. On average, it reduces the end-to-end latency by 35%.

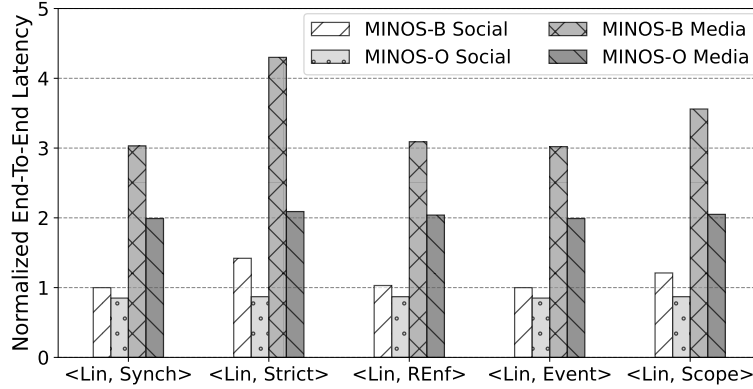


Figure 12: End-to-end latency of real applications.

3.7.4 Evaluating the MINOS-O Optimizations

To evaluate the impact of the MINOS-O optimizations of §3.4.2, we group them into three groups: (i) offloading operations to the SmartNIC plus supporting coherence between host and SmartNIC, and eliminating write locks; (ii) message batching; and (iii) message broadcasting. The first group, called *Combined*, combines three optimizations because applying them separately is sub-optimal. Also note that batching can only be beneficial if Combined is applied first.

Figure 13 compares the average write latency of a workload that issues only client write operations for different architectures: MINOS-B, MINOS-B plus broadcast, MINOS-B plus batching, MINOS-B plus Combined (represented as Offl+Coh+WRLock), MINOS-B plus Combined and broadcast, MINOS-B plus Combined and batching, and MINOS-O. The bars are normalized to MINOS-B and the evaluated model is <Lin, Synch>.

We see that augmenting MINOS-B with broadcast or batching has no noticeable effect. However, augmenting MINOS-B with the Combined optimization is very effective: the write latency goes down by 43.3%. The reasons are the high efficiency of operation execution in the SmartNIC and the reduced number of host-SNIC communications. Taking

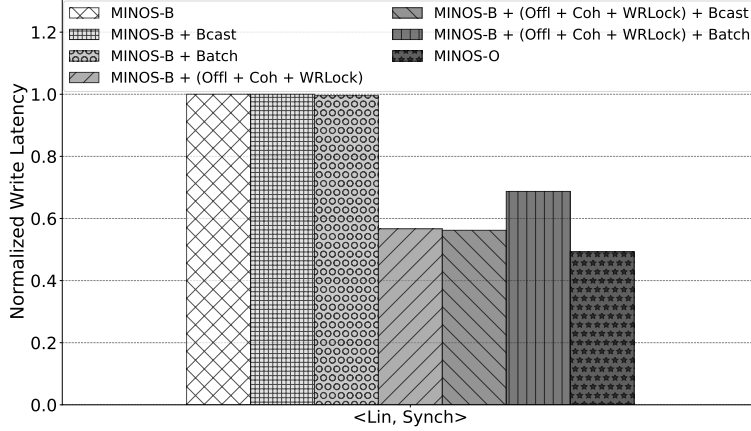


Figure 13: Impact of the MINOS-O optimizations.

this design and adding broadcast barely affects the write latency, as the system is already quite efficient. On the other hand, taking MINOS-B with the Combined optimization and adding batching slows the execution. The reason is that, as a batched message arrives at the SmartNIC, the latter has to unpack it, which adds overhead. However, if we add all the optimizations (creating MINOS-O), batching is beneficial because, with broadcast, the message does not need to be unpacked. Overall, MINOS-O reduces the average write latency by 50.7% over MINOS-B.

3.7.5 Sensitivity Analysis

In this section, we perform a sensitivity analysis of different MINOS-O parameters. We use a workload with 50% writes and 50% reads running with <Lin, Synch>, and measure the average write latency. Figure 14 shows the normalized write latency of MINOS-O as we vary the FIFO size. The figure shows bars for 1, 2, 3, 4, 5, and 100 entries in each of vFIFO and dFIFO. The bars are normalized to the MINOS-O write latency with an unlimited number of FIFO entries. We see that, with 3-5 entries, one attains the same average latency as with an unlimited number of them.

Figure 15 compares the average execution time of a write in MINOS-O and MINOS-B. Specifically, it shows the write transaction speedup of MINOS-O over MINOS-B under

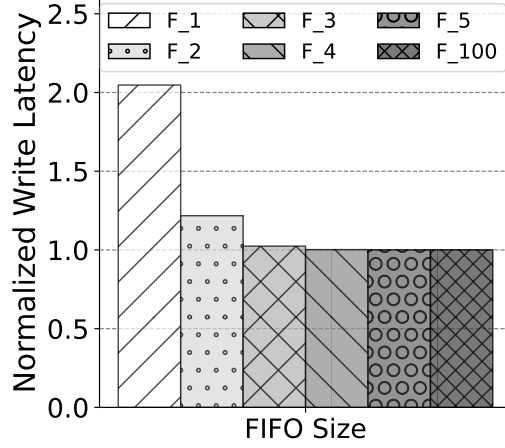


Figure 14: Sensitivity to FIFO size.

different values of persist latency, key distribution, and database size. In the first set of bars, we vary the time it takes to persist a 1KB record from 100ns to 100 μ s. These values are selected to represent current and future durable mediums. For example, today, persisting a 64B cache line to NVM takes about 100 ns [53], and persisting a block to SSD takes about 100 μ s [70]. We can see that MINOS-O speeds-up write transactions in all cases. The speedups increase with the persist latency and are, on average, 2.2x.

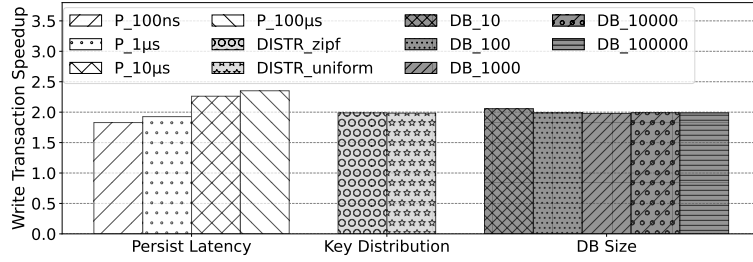


Figure 15: Sensitivity to other parameters.

In the second set of bars, we vary the key distribution between zipfian (default) and uniform. In general, with zipfian, we would expect more conflicts on the same keys, therefore reducing performance. As already mentioned, however, our algorithms can support concurrent and conflicting writes in both MINOS-B and MINOS-O. As a result, the figure shows that MINOS-O delivers a speedup of 2x over MINOS-B in both distributions.

Finally, in the third set of bars, we vary the database size from 10 records to 100K

records. In general, the smaller the database size is, the more the conflicts that are expected. Since both algorithms handle conflicting writes well, the changes as we increase the database size are minimal. On average, MINOS-O delivers a speedup of 2x over MINOS-B.

3.8 Related Work

Consistency and persistency models. There are several key works in this area. Pelley et al. [15] introduce memory persistency models. Ganesan et al. [17] introduce the Read-Enforced persistency model. For consistency models, Hermes [8] proposes a leaderless design for Linearizable consistency. DDP [16], which is closest to our work, introduces a set of combinations of consistency and persistency models and their high-level operation.

NICs in datacenters. NICs can improve performance in datacenters. Caulfield et al. [39] use NICs to enable direct communication between pools of FPGAs. FLOEM [24] introduces a framework to program NIC-accelerated applications. Some works extend RDMA capabilities: Reda et al. [40] support self-modifying RDMA chains, PRISM [41] introduces new primitives, and Hyperloop [42] supports distributed transactions. RAMBDA [71] offloads CPU tasks into a cache-coherent accelerator that can directly interact with RDMA NICs.

Programmable NICs and switches. They are used in datacenters [23], [43]. The NetCache [72] and IncBricks [73] switches cache data in the network, and PMNet [74] also persists data in the network (whether it is a programmable switch or a NIC). Several works optimize/extend existing RDMA primitives using SmartNICs [47], [69], [75], or offload storage operations [76]. In SmartNICs, LineFS [77] supports distributed file systems with support to persist at the client’s side, TURBO [78] load-balances light-tailed RPCs, and Xenic [50] accelerates distributed transactions by maintaining locks and hot data at the NIC. iPipe [25] provides a new programming model to offload distributed applications onto SmartNICs, and SKV [79] offloads key-value stores to SmartNICs while persisting at the

host. MINOS is different from prior works in that it targets offloading both consistency and persistency protocols to the SmartNIC, and uses rigorous definitions of consistency and persistency models.

3.9 Conclusion

To enable high-performance, programmable, and durable distributed systems, this chapter has introduced detailed distributed algorithms for DDP models. The algorithms support Linearizable consistency with five different types of persistency. We call them MINOS-B. Then, to improve performance, we redesigned the algorithms to offload them to a new SmartNIC architecture. The architecture introduces optimizations such as selective data coherence in hardware between host and SmartNIC, message batching, and message broadcasting. The resulting algorithms and architecture are called MINOS-O. Our evaluation of MINOS-O showed that offloading substantially reduces request latency and increases request throughput for various workloads and number of nodes. For example, compared to MINOS-B, MINOS-O reduced the average end-to-end latency of two microservice functions by 35%.

Chapter 4 IASO: Online Incremental Recovery in Leaderless Distributed Systems with Different Persistency Models

4.1 Introduction

Distributed storage applications such as key value stores and databases [1], [2], [3] are central to today’s cloud infrastructure. To provide high performance and availability, these applications replicate data across multiple nodes [4], [6], [7]. Moreover, to enable fault recovery, they continuously persist data to non-volatile storage. To decide when an update in one node should reach the replicas in other nodes, these systems use different memory consistency models [10]; to decide when the replicas in the multiple nodes should be persisted, these systems use different memory persistency models [15], [20].

Most distributed storage systems, such as ZooKeeper [26], FaRM [4], and RAMCloud [7] are *leader-based*. This means that, while client reads can be processed by any of the nodes in the cluster, client writes can only be processed by a single designated node called the *Leader*. As the leader processes a write, it ensures that the replicas in all the nodes are correctly updated and correctly persisted.

A related approach used in transactional [80], [81] and State Machine Replication (SMR) [82] systems relies on support for atomic broadcast. Each client broadcasts each transaction or operation to all the nodes in the cluster, and transactions/operations are totally ordered through a Global ID. Each node then applies each transaction/operation locally. While this approach simplifies recovery, it is costly, as it requires additional software support for the broadcast primitive, usually through consensus [82].

A recent, higher-performance approach is to organize the cluster as a *leaderless* system [8], [9], [20]. In this approach, both client reads and writes can be processed by any individual node in the cluster. A client sends the write request to a single node, which enforces that all the replicas are updated and persisted correctly. Leaderless systems are attractive, as they can attain superior performance and are intrinsically scalable. However,

their implementation is challenging, as the system must support multiple concurrent writes initiated by different nodes, while updating replicas and persisting updates following the correct consistency and persistency requirements.

Recently, in distributed systems, ensuring resilience has become an increasingly growing concern [29], [83], [84], [85], [86]. Both software and hardware failures are becoming more frequent. To recover from faults, there are a number of proposed techniques, which use logging [87], [88], [89], [90], replication [8], [9], erasure coding [91], or a combination of them. However, in today’s environment, it is crucial to deploy light-weight mechanisms that recover a system very quickly and with only a small impact on the live nodes’ throughput.

The most efficient way to recover after the failure of a system component (e.g., a node) is to perform *online incremental recovery*. Online means that the recovery is performed while the workload continues executing; incremental means that the recovery consists of re-applying only the state changes that were missed since the failure. To support online incremental recovery, two mechanisms are required. First, the system needs to keep a record of the updates that were completed since the failure and apply them to the recovering node. Second, these operations need to be supported while the healthy nodes continue processing the reads and writes of the workload.

Unfortunately, no *leaderless* system supports this form of recovery. For example, the state-of-the-art Hermes [8] leaderless system performs online recovery, but recovery requires a full database transfer to the recovering node, which can take a long time. State-of-the-art leader-based systems such as ZooKeeper [26] perform online incremental recovery. However, their recovery works with the simpler, lower-performing *leader-based* protocols.

In addition, recovery in ZooKeeper is not as efficient as it is needed by the increasing resilience concerns of cloud clusters. First, during recovery, the leader node performs a costly access to its log in persistent storage to retrieve the set of updates missed by the recovering node. Second, this resulting set of updates, which is sent to and persisted in the recovering node, can contain redundant entries—i.e., multiple updates to the same record.

Finally, during recovery, the recovering node does not participate in the protocol of the ongoing writes, which further slows down recovery.

To address these limitations, this chapter proposes *IASO, the first online, incremental recovery scheme for leaderless distributed systems*. IASO is a non-transactional system optimized for minimal recovery latency and low throughput impact. In addition to operating in leaderless environments, IASO introduces three techniques for low overhead recovery that could also be used by a state-of-the-art leader-based recovery scheme such as ZooKeeper. First, IASO provides to the recovering node all the updates that the latter missed with very low overhead. We call this approach *Proactive* recovery, in contrast to *Reactive* recovery in ZooKeeper. Second, the provided updates contain no redundant updates—reducing the overheads of transfer and persistency. Finally, during recovery, the recovering node remains active, participating in the protocol of the ongoing writes, which speeds-up execution.

This chapter presents IASO’s algorithms for the combination of Linearizable consistency and five memory persistency models: Synchronous, Strict, ReadEnforced, Eventual, and Scope.

We implement IASO on a 6-node Intel cluster with 7 cores per node. The results show that IASO is very effective. In experiments running TAOBench applications [92] under Linearizable consistency and Synchronous persistency, IASO recovers the cluster after a node failure in 120 milliseconds on average, while reducing the throughput of the running workload by an average of only 8.7%. In contrast, the state-of-the-art recovery scheme for leaderless systems is non-incremental, and needs 360 seconds to recover, reducing the throughput of the system by 16.2% on average. Further, compared to an incremental recovery scheme for a *leader-based* system, IASO reduces the average recovery latency by $20.7\times$ and the throughput degradation by 62.4%.

The contributions of this chapter are as follows:

- IASO, the first online incremental recovery scheme for leaderless distributed systems.

- The IASO design for 1 consistency and 5 persistency models.
- The IASO implementation on a 6-node distributed machine.
- An evaluation of IASO.

4.2 Motivation

To motivate the need for fast recovery after a node failure, we consider the recovery of a state-of-the-art leaderless system like Hermes [8], which involves copying the complete key-value store from a healthy node to a new node that replaces the one that failed. We implement the leaderless Hermes protocol in a 2-node cluster with Xeon E5-2450 CPUs (Section 4.7). We refer to the new node as the *Recovering* node, and the healthy one that provides the data as the *Recoverer* node. We use one-sided RDMA writes to transfer data from recoverer to recovering node.

We consider three environments. In *DRAM-only*, the data is moved from the recoverer’s DRAM memory to the recovering’s DRAM memory. This is the design assumed in Hermes. In this plot, we assume an environment where reading or writing 256GB of DRAM memory with all the related overheads takes about 50 seconds. However, a likely more realistic environment would use persistent memory (PM) like Intel Optane [54]¹ for reliability. In this case, we consider two designs. In *DRAM+PM*, the data is moved from the recoverer’s PM to its DRAM, then sent via RDMA to the recovering’s DRAM, and then persisted to the recovering’s PM. In contrast, in *PM-only*, the data is sent via RDMA directly from the recoverer’s PM to the recovering’s PM. We believe *DRAM+PM* is preferable over *PM-only* because it avoids any inconsistency between DRAM and PM data.

Figure 16 shows the latency of the data transfer, which is the recovery time, as the size of the key-value store changes between 1GB–256GB. As the database size grows, the recovery latency increases. For a 256GB database, *PM-only* takes 254s, while *DRAM+PM*

¹Although Optane is discontinued, other alternatives can be used [93], [94]. For example, Micron demonstrated a 32 Gb (4 GB) NVDIMM device at IEDM [95], Everspin ships standalone STT-MRAM parts, Panasonic and STMicroelectronics continue to advance ReRAM and embedded PCM, and CXL explicitly supports emerging persistent media [94].

takes 360s. These are significant overheads.

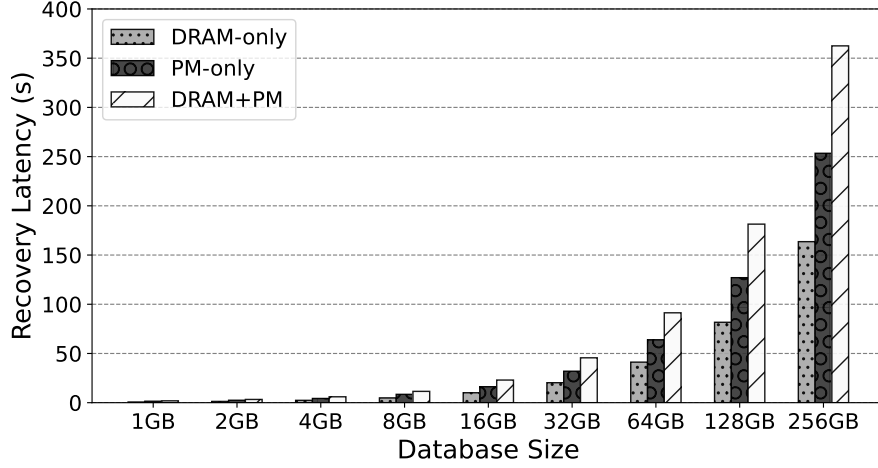


Figure 16: Recovery latency in a state-of-the-art leaderless system.

4.3 Failure Model

Cloud systems are expected to provide high availability. Even brief outages are highly undesirable [96], [97], as they can lead to potential financial losses of millions of dollars[98]. In this chapter, we assume a scenario where a node in the cluster fails or disconnects, but its durable state remains intact and accessible after a short downtime (e.g., seconds to minutes). On regaining access, the durable state can be either in the original failed node, now back online, or in a replacement node.

The first scenario (i.e., durable state in the original node) can be the result of various hardware or software faults [99], [100]. Example hardware faults are internal CPU errors, CPU frequency drops, I/O latency anomalies, overheating, or service exceptions. Example software faults are kernel crashes, firmware issues, application bugs, misconfigurations, memory/resource leaks, or software incompatibilities. This scenario can also be the result of regular fleet operations like rack maintenance or software/BIOS updates. These issues are frequently addressed through reconnection or reboot [99], [100], [101].

The second scenario (i.e., durable state found in a replacement node) appears when hardware replacement is needed to mitigate infrastructure-related errors [102], and may require the involvement of an operator [103], [104]. Storage systems are generally resilient

to failures, as they include technologies like battery-backed DRAMs [105], [106] and eADR [107]. On a node failure, they can be hot-plugged [108], allowing their transfer to a healthy node added to the cluster. Finally, datacenters often utilize dedicated storage servers [109], which ensure that remote storage remains intact and accessible by a replacement node in the event of a compute node failure.

Many of these errors are resolved within minutes. For example, the median restart time due to infrastructure issues is 8.6 minutes [102], while the median downtime following a crash is 11.43 minutes, with about half of these restarts classified as unplanned [110], [111]. Further, thanks to link redundancy, link failures are typically autonomously resolved without the need for an operator within a 5-minute time frame [112].

In these scenarios, (i) the ability of the software to access the durable state again as it was before the failure, and (ii) the short duration of the downtime, provide an opportunity to use high-performance online incremental recovery. The healthy nodes continue executing the application as usual—both while the failed node is down and while a node is recovering.

In cases where the durable state of a node is lost or remains inaccessible for an extended period, incremental recovery is not feasible. Then, the persistent state of the recovering node must be updated with the full copy of the datastore.

We assume that one node in the system runs a Configuration Manager (CM) process that regularly sends heartbeat messages to all the other nodes [26], [113]. If a node fails to respond within a timeout period, a failure is flagged. In line with prior work [8], [26], [114], [115], we assume the CM is fault tolerant. The CM periodically notifies a dedicated distributed service. If the service does not hear from the CM node for a while, it relaunches a new instance of the CM node. This is a well-established approach to detect node failures.

Other types of failures beyond a node crashing or getting disconnected, such as byzantine failures [116] or permanent network partitions, are not supported. We also assume that the system addresses silent data corruption by using existing RAS features and error-correcting codes in DRAM and network.

This chapter primarily assumes a single failure in the system at any given time, which aligns with prior work [117]. However, we extend our discussion to concurrent failures in Section 4.5.

4.4 IASO: Recovery in Leaderless Systems

IASO is a general and high-performance online incremental recovery scheme for leaderless distributed systems. It builds on top of an implementation of MINOS [20] and supports multiple memory persistency models. This section describes IASO.

4.4.1 Timeline of Fault Detection and Recovery

To put the operation of IASO in context, Figure 17 shows the timeline of fault detection and recovery. Before the IASO recovery algorithm executes, two steps take place: fault detection and system reconfiguration to isolate the faulty node. How to perform these steps is not a contribution of this chapter, as they have been detailed elsewhere [4], [5], [8]. Essentially, faults are detected by the CM through timeouts, and reconfiguration is performed by the CM by removing the failed node from the list of live nodes, and broadcasting the new list to all the live nodes.

After reconfiguration ②, the live nodes continue executing the workload, now without communicating with the failed node. This period is called the *Execution with Fewer Nodes* (EFN). Then, at some point, a recovering node R is identified by the CM ③. As indicated in Section 4.3, R may be the node that failed or a new one but, in either case, it contains the durable state of the failed node at the point of failure. After R is ready to recover, the CM changes the configuration again to include R in the list of live nodes, and broadcasts it to all the live nodes ④. The *Recovery time* now starts.

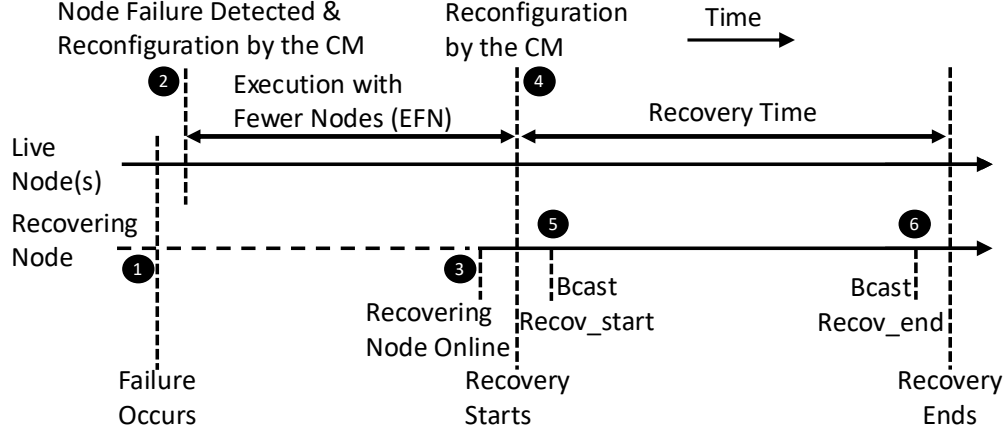


Figure 17: Fault detection, reconfiguration, and recovery.

4.4.2 Key Ideas and Structures for the IASO Recovery

To perform its recovery, IASO relies on extended definitions of the Durability and Visibility points, and on two software structures: the *Partially_persisted* list and the *Missed_updates* buffer. We describe them next.

4.4.2.1 Durability Point

Prior work [20] defines the Durability Point (DP) of an update in a distributed system as the point when the update is pushed to non-volatile storage. In this chapter, we extend the concept by defining a *Local Durability Point* (LDP) for each node. This is the time when the update is pushed to local non-volatile storage in that node—e.g., a non-volatile memory (NVM) log. Also, we define the *Global Durability Point* (GDP) as the time when the update is persisted in all replica nodes. The GDP is observed by different nodes at different times. Since an update has a Coordinator and multiple Follower nodes, we define a *Coordinator GDP* (CGDP) and per-follower *Follower GDP* (FGDP).

Figure 18 shows the transaction for an update as described in 2.1. It corresponds to the $\langle \text{Lin}, \text{Synch} \rangle$ model. We show a single Follower but there are multiple Followers since, in this chapter, we will assume that a record is replicated in all Followers. The figure shows the LDPs, the CGDP, and the FGDPs. The Coordinator reaches its LDP after it has

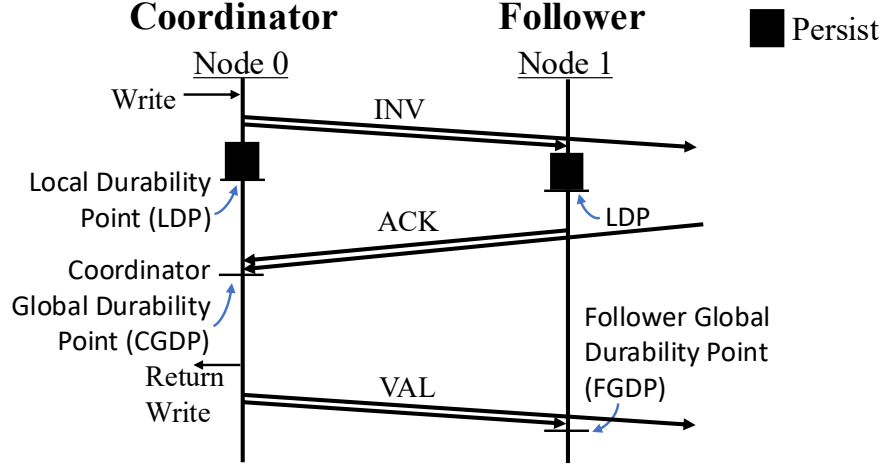


Figure 18: Durability points in an update under <Lin, Sync>.

persisted the update locally. Further, it reaches the CGDP after it has persisted the update locally and has received the ACKs from all nodes. A Follower reaches its LDP after it has persisted the update locally. It reaches the FGDP after it has received the VAL from the Coordinator. Section 4.4.5 describes the LDP, CGDP, and FGDP of the different models considered.

4.4.2.2 Visibility Point

Prior work [20] defines the Visibility Point (VP) of an update as the point when the update becomes available for consumption. It is equivalent to the DP except that VP deals with updates to the *volatile* storage. Hence, we also define a Local Visibility Point (LVP) per node, a Coordinator Global Visibility Point (CGVP), and a per-follower Follower Global Visibility Point (FGVP), in a similar manner. Section 4.4.5 describes the LVP, CGVP, and FGVP of the different models considered.

4.4.2.3 Partially_persisted List

In a Coordinator or Follower node, a *Partially-persisted* update is one that has been persisted in the local NVM log (and, therefore, has reached the LDP), but has not yet reached the CGDP or FGDP, respectively. To identify these updates, we use the *GDP Flag*. Each entry in the local NVM log has a *GDP Flag*, which is set when the update

finally reaches the FGDP (or CGDP).

If a node fails while an update has a clear GDP Flag, the update presents a dilemma during recovery, as the recovering node does not know whether the update reached the GDP in all the other nodes. Hence, during IASO's recovery, the recovering node compiles a list of all the partially-persisted updates in its NVM log. The list is called the *Partially_persisted* list.

4.4.2.4 *Missed_updates* Buffer

While a node is down, all the other nodes continue executing the application. Each of them keeps buffering all the application updates in a volatile buffer in local memory called *Missed_updates*. This buffer will be used during recovery. We allow each live node to generate its own buffer for redundancy and recovery simplicity.

The set of updates in *Missed_updates* is not the same as in the NVM log. First, the NVM log is periodically recycled by pushing its entries to longer-term permanent storage. Second, only updates that had their GDP Flag set go to *Missed_updates*. Finally, in *Missed_updates*, newer updates to a record overwrite earlier updates to the same record.

To prevent excessive DRAM usage, we limit the size of the *Missed_updates* buffer. Once this limit is exceeded, the buffers are discarded and recovery falls back to non-incremental recovery.

4.4.3 IASO Recovery Algorithm

After the CM changes the configuration for the second time, adding the recovering node R , and all the nodes are informed (④ in Figure 17), recovery starts. The idea behind the recovery is simple. R needs to ask a statically determined node (called its *Buddy* node) whether the stores in R 's *Partially_persisted* list were finally globally persisted. Further, R needs to get from its Buddy node the list of updates in the Buddy node's *Missed_updates* buffer and apply them locally. During all this process, R needs to

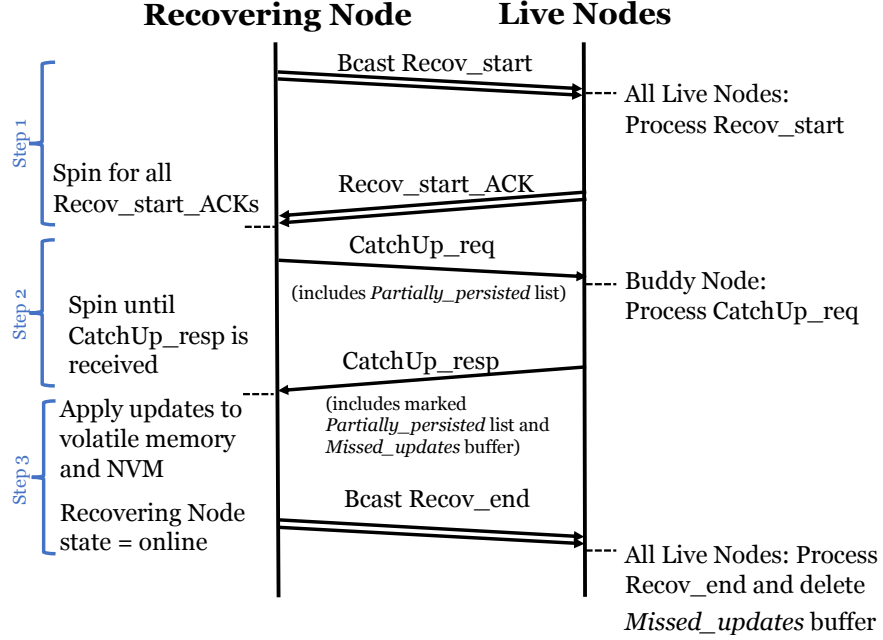


Figure 19: IASO recovery algorithm.

participate in the commit of application updates initiated by other nodes—although R does not accept direct client requests until the recovery is over. Finally, R needs to synchronize with all the other live nodes to initiate ⑤ and to terminate ⑥ the recovery.

Figure 19 depicts the recovery algorithm for the $\langle \text{Lin}, \text{Synch} \rangle$ model. Later, in Section 4.4.5, we consider the other models. Table 5 describes the messages exchanged in the algorithm. The algorithm has three steps.

Model	Reach LVP	Reach CGVP	Reach FGVP	Reach LDP	Reach CGDP	Reach FGDP
$\langle \text{Lin}, \text{Synch} \rangle$	Updated local volatile memory	All ACKs received	VAL received	Updated local NVM	All ACKs received	VAL received
$\langle \text{Lin}, \text{Strict} \rangle$		All ACK-Cs received	VAL_C received		All ACK_Ps received	VAL_P received
$\langle \text{Lin}, \text{REnf} \rangle$			VAL received			VAL received
$\langle \text{Lin}, \text{Event} \rangle$			VAL_C received	—	—	—
$\langle \text{Lin}, \text{Scope} \rangle$		All [ACK_C]sc's received	[VAL_C]sc received	Updated local NVM*	All [ACK_P]sc's received*	[VAL_P]sc received*

Table 4: Times when an update reaches the different visibility and durability points for different models. In the table, “*” does not correspond to the update but, instead, to the PERSIST command that finishes the scope of the update.

Step 1: As soon as the CM completes the second reconfiguration, R *participates* as a

Message	Description
Recov_start	Broadcasted by the recovering node R to all live nodes to initiate the recovery.
Recov_start_ACK	Sent by live nodes to R as a response to Recov_start.
CatchUp_req	Sent by R to the Buddy node to request the updates it has missed. It includes its Partially_persisted list.
CatchUp_resp	Sent by the Buddy node to R . It includes R 's Partially_persisted list marked by the Buddy node and the Buddy node's Missed_updates buffer.
Recov_end	Broadcasted by R to all live nodes to complete the recovery.

Table 5: Messages exchanged in the recovery algorithm.

Follower in the protocol of all the updates initiated by other nodes. However, it cannot yet take any client requests. We say that R acts as a *ShadowReplica* [8].

In this first step, R broadcasts a *Recov_start* message to all live nodes. Then, it uses its local NVM log to compile its Partially_persisted list. In practice, in most cases, the Partially_persisted list is empty. Finally, all live nodes respond with a *Recov_start_ACK* message.

Step 2: R sends a *CatchUp_req* message that includes its Partially_persisted list to its Buddy node. The Buddy node checks the timestamp T_{in} of each entry in such Partially_persisted list against the timestamp T_{curr} of the corresponding record in the Buddy node's volatile datastore state. Three cases are possible. If T_{curr} is equal to T_{in} , the entry in the Partially_persisted list is marked as globally committed. If T_{curr} is less than T_{in} , the entry in the Partially_persisted list is not marked because the update never committed. Finally, if T_{curr} is higher than T_{in} , a later update to the record has committed. Since such update must be in the Missed_updates buffer, there is no need to mark the entry in the Partially_persisted list as committed.

Then, the Buddy node responds to R with a *CatchUp_Resp* message with: 1) the marked Partially_persisted list and 2) the contents of the Buddy node's Missed_updates buffer.

Step 3: When R receives *CatchUp_Resp*, it applies locally the marked updates from the received Partially_persisted list, and the updates from the received Missed_updates buffer.

The updates are applied to both volatile memory and NVM to bring both of them up-to-date.

Then, R changes its state to fully online, which allows it to take client requests. It broadcasts a *Recov_end* message to all live nodes to mark the end of the recovery. When a live node receives the message, it deletes its Missed_updates buffer.

4.4.4 Recovery Issues

In IASO, no committed update is lost. At the time a node fails (① in Figure 17), its local NVM log has an accurate record of what updates have been globally persisted (i.e., the entries with GDP Flag set) or at least locally persisted (i.e., those with GDP Flag clear). In addition, any update initiated from a live node after the node failure but before the first CM-triggered reconfiguration *cannot complete*. The reason is that it requires ACKs from all Followers, including the failed node. The communication with the failed node times out, and the write cannot complete. Eventually, the CM discovers the failure. After reconfiguration, during the EFN period, all updates are saved in the live nodes' Missed_updates buffers.

During recovery, R proceeds to apply the updates received in the *CatchUp_Resp* message from the Buddy node. However, it rejects those that have already become obsolete due to a later update to the same record during its execution as a *ShadowReplica*.

The longer the EFN period is, the larger the Missed_updates buffer in the Buddy node will grow, and the longer it will take for R to process the buffer during the recovery. The duration of the EFN period depends on how quickly an R can be brought online and is orthogonal to IASO.

The time between a node failure and failure detection is likely to be small. This is because it is determined by the timeout of a CM heartbeat to the failed node.

4.4.5 Impact of Consistency-Persistency Models

Table 4 shows when the Local, Coordinator Global, and Follower Global Visibility Points (LVP, CGVP, and FGVP) and Durability Points (LDP, CGDP, and FGDP) are reached for an update in the five <consistency, persistency> models considered. The terminology used follows MINOS [20]. For example, in <Lin, REnf>: (i) a node reaches LVP when it updates its local volatile memory; (ii) a Coordinator reaches CGVP when it receives all the consistency ACKs (ACK_Cs); (iii) a Follower reaches FGVP when it receives the VAL; (iv) a node reaches LDP when it updates its local persistent memory; (v) a Coordinator reaches CGDP when it receives all the persistency ACKs (ACK_Ps); and (vi) a Follower reaches FGDP when it receives the VAL.

Note that <Lin, Event> has no defined durability points for an update, as it persists “eventually”. Moreover, as shown with an asterisk, the durability point conditions for <Lin, Scope> refer not to messages in the update itself but, instead, to messages in the PERSIST that finishes the scope of the update. These visibility/durability point considerations are in effect during normal execution and during ShadowReplica execution. Hence, they are in effect when updating the Missed_updates buffer and when persisting state in the local NVM log.

Type of Update Failure	How the Failure Makes the Update Operation Incomplete	Completing the Update during Execution with Fewer Nodes	What the Recovering Node does during Recovery
Coordinator fails after sending at least 1 INV and before sending all VALs.	At least one Follower received INV but no VAL.	CM reconfigures to remove Coordinator and inform all other nodes. One or more live nodes that had received INV but no VAL take over as Coordinator and replay the update.	Apply the update from the Missed_updates buffer of the Buddy Node (if the update has not become obsolete already).
A Follower fails before sending the ACK to the Coordinator.	The Coordinator does not collect all ACKs.	CM reconfigures to remove Follower and informs all other nodes. The Coordinator replays the update.	Apply the update from the Missed_updates buffer of the Buddy Node (if the update has not become obsolete already).

Table 6: Node failures and recovery from them.

4.4.6 Failure Scenarios

IASO considers two types of failures. The first one includes message loss, long message delays due to high contention, and intermittent network link disconnections; the second one includes a node failing or getting permanently disconnected from the network. As indicated in Section 4.3, other failures like permanent network partitioning are not considered.

To understand how to handle the supported failures, recall that, when a node receives a client read, it satisfies it locally, while when it receives a client write, it involves all the other nodes. If a failure occurs during a read, the read does not complete and the client retries after a timeout using a different replica node. This is trivial and so we do not explore it. Hence, we focus on failures during a write. For simplicity, we use $\langle \text{Lin}, \text{Synch} \rangle$, but similar ideas apply to the other models. We consider the two types of failures.

4.4.6.1 Recovering from Message Loss/Delay and Intermittent Network Disconnections

These failures may cause two types of timeouts (Figure 18). The first one (*INV-ACK*) goes off in a Coordinator if the time from when all the INVs have been sent until when all the corresponding ACKs have been received is over a threshold. The second one (*ACK-VAL*) goes off in a Follower if the time from when it sends an ACK until when it receives the corresponding VAL is over a threshold.

When a Coordinator's *INV-ACK* timeout goes off, the Coordinator rebroadcasts the INVs to all Followers, similarly to Hermes [8]. The process is potentially repeated multiple times. We opt for rebroadcasting over sending messages to only the non-responding nodes to reset the timers in all the Followers. Each message includes the initial request timestamp, which remains unchanged upon replay, facilitating the handling of duplicates.

When a Follower's *ACK-VAL* times out, the Follower resends the ACK as in the previous case. Overall, in both cases, the response will eventually be received.

4.4.6.2 Recovering from Node Failures and Permanent Network Disconnections

These failures are identified through non-responsive CM heartbeats. To understand the IASO recovery, we consider the two possible cases: Coordinator failure (or disconnect) and Follower failure (or disconnect). Table 6 shows, for each of the two failures, (1) how the failure makes the update operation incomplete, (2) how the incomplete operation completes during the execution with fewer nodes, and (3) what the recovering node R does during the recovery.

A *Coordinator failure* occurs when the Coordinator of an update fails after sending at least one INV and before sending all VALs—we ignore the trivial case when a Coordinator fails before sending any of the INVs because, in this case, the client times out and reissues the request to another node. The operation is incomplete because there is at least one *incomplete* Follower, i.e., a Follower that received an INV and no VAL. Once the CM has detected the failure, it reconfigures the system to remove the Coordinator node and informs all the remaining nodes. At this point, an incomplete Follower takes on the role of the Coordinator for this update, and replays the update, carrying the original Coordinator’s timestamp. IASO sets the reaction times of the Followers so that one reacts first. However, it is possible that multiple incomplete Followers react simultaneously, which leads to multiple Followers taking on the role of Coordinator for the same update. This is correct because a node processes requests according to timestamps. Multiple messages with the same timestamp induce the same effect as a single message, as duplicates are obsolete.

A *Follower failure* occurs when a Follower fails before sending the ACK to the Coordinator of an update. The operation is incomplete because the Coordinator does not collect all the ACKs. Once the CM has detected the failure, it reconfigures the system to remove the Follower node and informs all the other nodes. Then, the Coordinator replays the update without including the failed node.

4.4.7 Comparison to ZooKeeper’s Recovery

The IASO recovery is more efficient than the ZooKeeper one described in 2.3. The most important reason is that IASO works for the higher-performing *leaderless* systems while ZooKeeper works for leader-based systems. In addition, IASO provides three advantages.

First, IASO provides *Proactive* recovery—i.e., it provides to the recovering node R all the updates that the latter missed with very low overhead. Indeed, such updates are collected in advance in the Missed_updates DRAM buffer and, on recovery, immediately sent to R by its Buddy node. Second, the Missed_updates buffer contains no redundant updates—only the latest update to each record. This design reduces recovery overhead during the transfer of the updates from the Buddy node to R , and during their application to the persistent storage in R . Finally, during recovery, R remains active, participating in the protocol of ongoing client updates as a shadow replica, which speeds-up execution.

4.5 Concurrent Failures

As described so far, IASO handles single failures; it is assumed that a second failure will not occur until after the system has recovered from the first one. Here, we outline how IASO can be extended to support concurrent failures.

Given the timeline of a node failure shown in Figure 17, we consider two cases: (i) during the EFN time, another node fails, and (ii) during the Recovery time, either the Buddy node, another live node, or the recovering node R fails. If a node fails during the EFN time, the system detects it like in the first failure. Because no update can make progress with a failed node, the CM reconfigures the system again, this time entering an EFN period with two missing nodes. This is shown as EFN2 in Timeline TL2 in Figure 20a.

During EFN2, the live nodes continue execution, augmenting their Missed_updates buffer (which contains the updates performed during the first, interrupted EFN1 period) with the new updates performed during this new EFN2 period. For simplicity, the CM will wait until it can get two working nodes to replace the two failed nodes. At that point, it

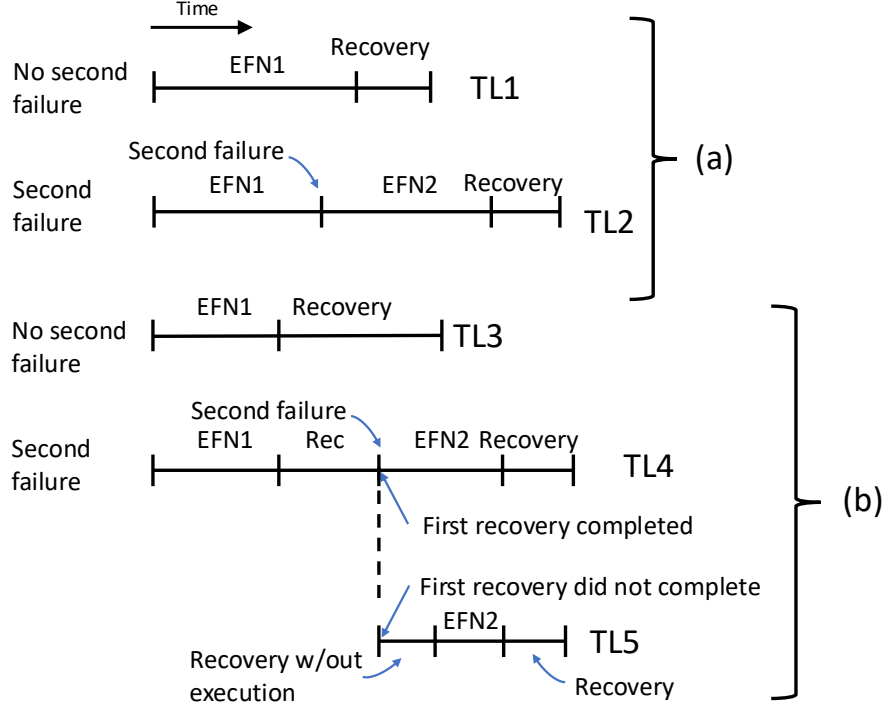


Figure 20: Handling multiple concurrent failures. In the figure, TL_i means timeline i .

will reconfigure the system, and the recovery re-starts, with two recovering nodes. Each of them picks a different Buddy node and, as it gets the Missed_updates from its Buddy node, it only applies those updates that it has missed: the node that had failed earlier applies more updates than the one that failed later. This is shown in Figure 20a.

If a node fails during a recovery period, irrespective of whether it is a Buddy node, R , or any other node, no update can make progress until the CM reconfigures the system. Before the second failure and reconfiguration, R may have made some progress in the recovery process. After the reconfiguration, IASO checks whether R is up and has been able to apply locally all the updates missed since the first fault.

If this is true (Timeline TL4 of Figure 20b), IASO considers the first recovery completed. Then, IASO initiates an EFN2 period while a new node is identified. When the new node is identified and connected, a recovery for that node is performed.

If the first recovery did not complete (Timeline TL5 of Figure 20b), IASO requires the first recovery to be redone. For simplicity, IASO repeats the first recovery without allowing

1. Concurrency Checks Absence of deadlocks and livelocks.
2. Consistency Checks a) The timestamp of the record in volatile memory is the same across all replicas. b) When the Coordinator is about to send a VAL for consistency for a record, the timestamp of the record in volatile memory is the same across all replicas.
3. Persistency Checks a) The timestamp of the record in NVM is the same across all replicas. b) When the Coordinator is about to send a VAL for persistency for a record, the timestamp of the record in NVM is the same across all replicas.

Table 7: Conditions checked using TLA+ for all the models, in the presence of failures.

the live nodes to continue execution. This is reasonable given that having two concurrent failures is relatively rare. In the special case where the second failed node is precisely the node that was trying to recover, IASO waits until a new node can be hot-plugged, and then proceeds to redo the first recovery. Further, in the special case where the second failed node is precisely the Buddy node, a new Buddy node is identified.

Once the first recovery is completed, execution resumes during an EFN2 period while a new node is identified to fill in for the second failed node. When the new node is identified and connected, a recovery for the second failure is performed.

We could instead try to redo the first recovery while the healthy nodes continue executing under an EFN2 period. However, this would potentially require having a double set of Missed_updates buffers per node.

As indicated in Section 4.3, if the CM fails, the Zookeeper relaunched a new instance of the CM.

4.6 Protocol Verification

To ensure the correctness of the IASO protocols, we employ the TLA+ formal specification and verify our models using the TLC model checker [51]. Our implementation builds on the publicly available TLA+ specification of Hermes [8], extending it to incorporate all `<consistency, persistency>` model pairs discussed in this chapter.

First, we ensure *concurrency safety* by confirming the absence of deadlocks and

livelocks across all reachable states. Second, we verify *consistency correctness* by checking that the timestamp of each record in volatile memory is consistent across replicas. Third, we check *persistence correctness* by ensuring that the timestamp of each record in NVM is the same across replicas. All correctness conditions are validated under the fault scenarios of Section 4.3, including node crashes and message loss, ensuring full system recovery.

We executed TLC on a 16-core workstation with 64 GB of memory using breadth-first exploration and full symmetry reduction [51]. For instance, the `<Lin, Synch>` verification completed in 26 minutes, exploring 138 million distinct states with a state-space diameter of 41. These values are representative of the other persistency models.

4.7 Methodology

We evaluate IASO running on a 6-node cluster with 7 cores per node provided by CloudLab [52]. This is the largest cluster we could reliably obtain with the necessary configuration. The cluster parameters are shown in Table 8. One of the nodes is the CM, while the other 5 nodes run the workloads. IASO replicates every record in each of the 5 nodes. This is the default replication degree in past work [8], [20]. In larger machines, one could have multiple 5-node replication groups. The nodes in the cluster do not have modern persistent memories. Hence, we emulate the latency and bandwidth of persistent memory based on prior work [54]. We use an in-memory datastore of 13 GB per node and model one with 256 GB per node as will be described in Section 4.8.1.

When a node receives a protocol message (e.g., INV), it needs to perform a sequence of steps. If we used the popular one-sided RDMA for communication, we would need multiple messages to perform these multiple steps. Consequently, we instead use a Remote Procedure Call (RPC) library named eRPC [46], [59] that provides performance similar to RDMA but with fewer transfers. The CM uses one-sided RDMA for the heartbeat exchange.

Workloads Used. We evaluate IASO with two different benchmarks: the TAOBench

Number of nodes	6
CPU per node	Xeon E5-2450 (7 cores, 2.1 GHz)
In-memory datastore	13GB/node and 256GB/node (modeled)
NIC per node	Mellanox MX354A FDR CX3
Timeouts	2 ms INV-ACK, 2 ms ACK-VAL
Heartbeat period	4 ms

Table 8: Cluster used to run IASO.

application suite [92] and the Yahoo! Cloud Serving Benchmark (YCSB) [66], [67].

TAOBench tests the performance of social graph databases, simulating real-world user interactions. In our experiments, we use four of the provided drivers: MySQL [118], CockroachDB (CRDB) [119], YugabyteDB [120], and Cloud Spanner (Spanner) [121]. The default setup uses a uniform record distribution, and 80% reads and 20% writes. After failure detection, we set the period of execution with fewer nodes (EFN) to 360 seconds.

Because the TAOBench runs take a long time, we also run YCSB. For the YCSB experiments, we use a Hashtable [3] back-end in-memory application. The workload uses a zipfian distribution for records, has 80% read and 20% write operations, and each node issues 3M back-to-back operations. Because YCSB are synthetic benchmarks that issue back-to-back transactions, we set their EFN period to 6 seconds.

Systems Evaluated. We evaluate three leaderless systems that use the Hermes [8] consistency transactions on a MINOS[20] implementation of different persistency models: *Baseline*, *IASO*, and *State-of-the-art*. *Baseline* does not have any recovery support and we assume that it suffers no failure. *IASO* is Baseline plus our proposed incremental recovery mechanism. *State-of-the-art* is Baseline with a nonincremental recovery; on a fault, it copies the full database from one node to another like Hermes. We cannot use Hermes as is because Hermes does not consider any persistency model. In both IASO and State-of-the-art, we model a node failure and subsequent system recovery. The default DDP model used is $\langle \text{Lin}, \text{Synch} \rangle$. We are interested in two metrics: 1) failure recovery latency and 2) loss of system throughput due to the failure.

In Section 4.8.4, we also compare IASO to the ZooKeeper online incremental recovery

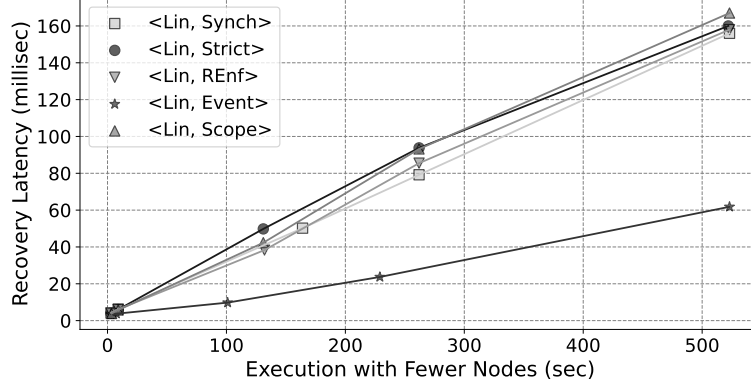


Figure 21: IASO recovery latency as a function of the EFN duration for a 256GB datastore per node.

scheme. The two systems differ not only on the three IASO recovery optimizations described in Section 4.4.7; they also differ in the protocol used: IASO is *leaderless* and ZooKeeper is *leader-based*.

4.8 Evaluation

We first use TAOBench to compare *IASO*'s recovery latency and throughput degradation to *State-of-the-art*. Then, we use the faster YCSB runs to characterize IASO. Finally, we compare IASO to the leader-based ZooKeeper.

4.8.1 Recovery Latency with the TAOBench Suite

Figure 21 shows the recovery latency of IASO as a function of the duration of the execution with fewer nodes (EFN). The figure refers to the average of our four TAOBench drivers, and shows curves for the different persistency models. We model a 256GB datastore per node. Since the machine we use for our experiments only has 16GB per node, we cannot execute the actual datastore. Hence, we use a 13GB datastore per node but emulate a very large datastore by assuming that all the updates performed during the EFN period are to *different keys*. Hence, the Missed_updates buffer adds an entry at every update, and its size grows fast, which increases the recovery time.

We see that, as the EFN period increases, the recovery that follows it takes a longer

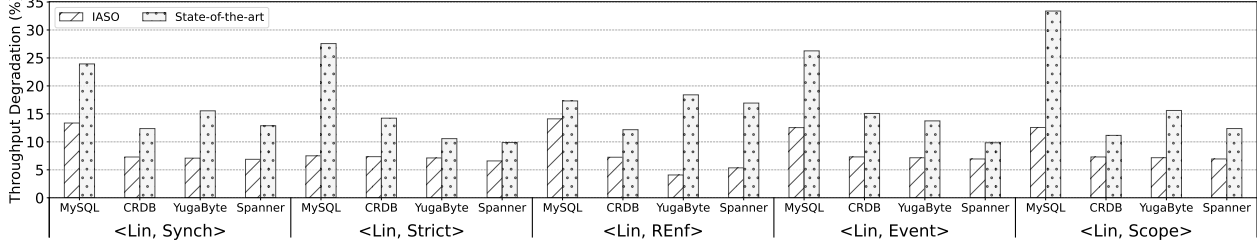


Figure 22: Throughput degradation of *IASO* and *State-of-the-art* over *Baseline* running TAOBench for a 256 GB datastore. Lower values are better.

time in all cases. This is because, the longer a node is offline, the more state the Missed_updates buffers accumulate; then, during recovery, this state is applied to the recovering node. We also see that the recovery latency is similar for all models except <Lin, Event>. The latter has a faster recovery mostly because, in Event, the recovering node applies the updates received from its Buddy node to its persistent memory in the background.

We see that the recovery latency is always small. For example, for an EFN period of 360s, the recovery latency of <Lin, Synch> is about 120ms. This is in contrast to the time it takes for *State-of-the-art* to transfer the database from a recoverer node’s persistent memory (PM) to the recovering node’s PM. From Figure 16, such time is 360 sec for DRAM+PM and 254 sec for PM-only in a 256GB datastore per node. These times are lower bounds of the recovery latency of *State-of-the-art*.

Table 9 shows the size of the per-node Missed_updates buffers for Lin consistency and various persistency models for experiments with 13GB and 256GB datastores. The data is the average across all the TAOBench drivers. We see that, for a 13GB database, the Missed_updates buffers are small. This is because the datastore is small, and many keys are reused during the EFN period. In contrast, for a 256GB datastore, the Missed_updates buffer sizes are larger, and reach 17.7–32.1 MB per node. In addition, the sizes vary across the models, which we attribute to the fact that the EFN period measured in different experiments corresponds to somewhat different sections of the program.

Overall, this is an acceptable cost to enable incremental recovery. If the EFN period was so

Datastore Size	Persistency Model				
	Synch	Strict	REnf	Event	Scope
13GB	2.25 MB	2.25 MB	2.25 MB	2.25 MB	2.26 MB
256GB	19.20 MB	28.86 MB	17.66 MB	18.11 MB	32.14 MB

Table 9: Average per-node Missed_updates buffer size for Lin consistency and various persistency models.

long that the Missed_updates buffer overflowed, the system would revert to recovering by transferring the whole datastore.

4.8.2 Throughput with the TAOBench Suite

Figure 22 shows the throughput degradation of *IASO* and *State-of-the-art* over *Baseline* for the 4 TAOBench drivers and 5 models. *Baseline* has no failure, while *IASO* and *State-of-the-art* suffer and recover from one node failure. The *State-of-the-art* bars use the *DRAM+PM* environment of Fig. 16. We model the 256GB datastore. In the figure, lower bars are better.

If we consider $\langle \text{Lin}, \text{Synch} \rangle$, we see that the average throughput degradation in *IASO* and *State-of-the-art* is 8.7% and 16.2%, respectively. Across all persistency models, the throughput degradation in *IASO* is 4–14%, with an average of 8.1%; the throughput degradation in *State-of-the-art* is 10–34%, with an average of 16.5%. The reason for the difference between *IASO* and *State-of-the-art* is the different recovery latencies in the two systems. In both systems, the application starts with healthy nodes, suffers the loss of a node for 360s, goes through a recovery, and then finishes execution with all nodes healthy. During the EFN and recovery periods, both systems execute with lower throughput. Since the recovery completes much faster in *IASO*, the average throughput degradation in *IASO* is substantially lower than in *State-of-the-art*.

The difference in throughput degradation between *IASO* and *State-of-the-art* remains large across persistency models. In general, weaker models such as $\langle \text{Lin}, \text{Event} \rangle$ should suffer less absolute throughput degradation since, as seen in Sec. 4.8.1, their recovery latency is shorter. However, weaker models also execute faster in *Baseline*, and the loss in

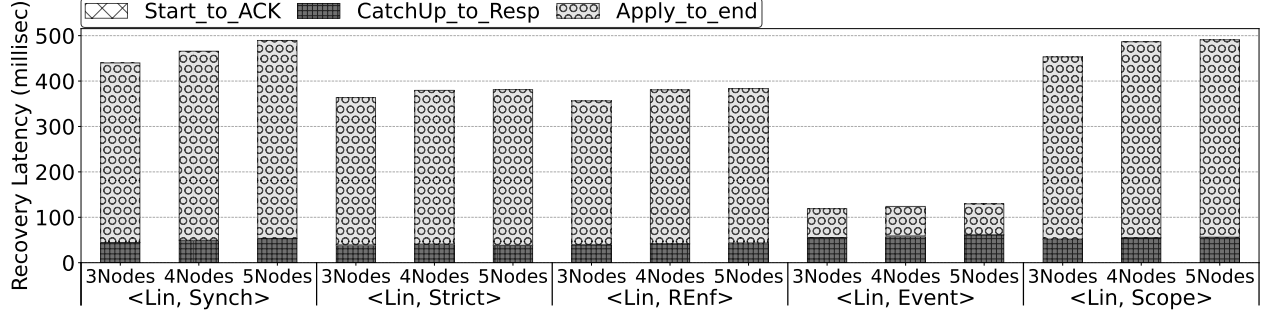


Figure 23: Breakdown of *IASO*'s recovery latency for different persistency models and numbers of nodes.

throughput during the constant-time EFN period causes a higher loss in weaker models.

For these reasons, the percentage throughput degradation is similar across persistency models.

4.8.3 Breakdown of the IASO Recovery Latency

We now characterize *IASO* with faster runs using YCSB. Figure 23 shows the recovery latency for all models and for executions with 5, 4, or 3 nodes. The bars are broken down into the three steps of the recovery shown in Fig. 19: from the beginning to when the recovering node *R* receives all the Recov_start_ACKs (*Start_to_ACK*), from when *R* prepares and sends CatchUp_req until it receives CatchUp_resp (*CatchUp_to_Resp*), and from when *R* starts applying the updates based on CatchUp_resp until the end (*Apply_to_end*).

The Start_to_ACK time is negligible, as it involves exchanging small messages. CatchUp_to_Resp takes about 10-15% of the recovery latency in most models and about 50% in <Lin, Event>. In absolute terms, however, CatchUp_to_Resp is similar in all models, as it involves sending at least one bulky message. Apply_to_end is typically the dominant time by far. The only exception is for the weak <Lin, Event>. Apply_to_end is long because the recovering node *R* persists locally all the entries in the Missed_updates buffer that are not obsolete. In <Lin, Event>, Apply_to_end is short because, under Event persistency, *R* does not apply the updates to the persistent memory in the critical

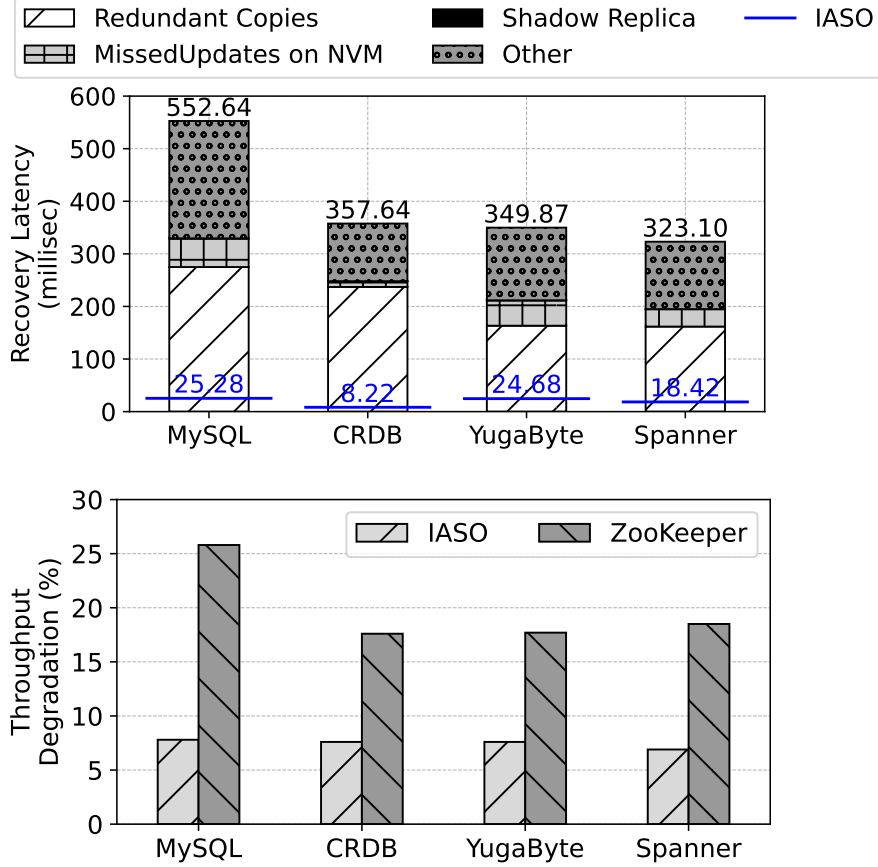


Figure 24: *IASO* vs ZooKeeper for a 13 GB datastore: recovery latency (top) and throughput degradation over the failure-free *Baseline* (bottom).

path.

<Lin, Scope> has a high Apply_to_end time even though it is weaker than some other models. The reason may be because this model needs more synchronization between nodes through [PERSIST]sc commands. The Apply_to_end time tends to increase with the node count due to the higher network activity.

4.8.4 Comparison to the Leader-based ZooKeeper Scheme

Fig. 24 compares *IASO* with <Lin, Synch> and ZooKeeper. We run TAOBench on a 13 GB datastore, as the system suffers a node failure and, after a 360-second EFN period, recovers. The top plot shows the recovery latency of ZooKeeper (in bars) and *IASO* (as horizontal blue lines). The ZooKeeper latency is broken into four components. The first

three correspond to the shortcomings in ZooKeeper that *IASO* eliminates (2.3 and Section 4.4.7), slightly modified for ease of measurement: 1) redundant updates in the leader’s log that are read from non-volatile storage, transferred, and applied to the recovering node’s log (*Redundant Copies*), 2) accessing the leader’s log in non-volatile storage rather than a proactively-prepared DRAM buffer *for the non-redundant copies* (*MissedUpdates on NVM*), and not allowing the recovering node to transition to a shadow replica (*Shadow Replica*). The fourth category is other recovery operations (*Other*).

We see that *IASO* recovers substantially faster than the leader-based ZooKeeper scheme. On average, its recovery latency is $20.7\times$ lower. Consider now the three identified overheads. Manipulating redundant copies in time-consuming non-volatile storage accesses and data transfers is a major overhead in ZooKeeper. Maintaining the log in non-volatile storage for the remaining, non redundant copies is also a large overhead. However, the absence of shadow replica operations incurs minimal additional overhead. In part, this is because these workloads are read-intensive; in write-intensive ones, the overhead will be higher. Finally, the *Other* category is primarily associated with ZooKeeper’s leader-based architecture. It accounts for about 40% of the recovery latency.

The bottom plot shows the throughput degradation experienced by applications over the *Baseline*. On average, *IASO* reduces the throughput degradation by 62.4% over ZooKeeper. While both schemes use on-line incremental recovery, their main throughput differences stem from: (i) *IASO* being leaderless and ZooKeeper being leader-based, and (ii) the two main inefficiencies of the ZooKeeper recovery protocol described above that can be eliminated with some *IASO* techniques.

4.9 Related Work

Many recovery techniques have been proposed. As indicated before, they use logging [87], [88], [89], [90], replication [8], [9], erasure coding [91], or a combination of them. Most of the current systems operate in leader-based mode (e.g., [4], [7], [17], [26], [90]). They use

different consistency and persistency models—e.g., RAMCloud [7] and Skyros [90] support a model similar to $\langle \text{Lin}, \text{Synch} \rangle$; CAD [17] supports a persistency model like REnf.

Several transactional [80], [81] and State Machine Replication (SMR) [82] schemes rely on clients broadcasting each transaction or operation to all the nodes in the cluster, and ordering transactions/operations through a Global ID. For that, they use special software support for a broadcast primitive, usually through consensus [82]. Such support is expensive.

There are only a few systems that operate in the recent, high-performance leaderless mode. They include EPaxos [9], Hermes [8], and MINOS [20]. These schemes attain the highest-performance execution, but make online incremental recovery more complex. In Hermes, recovery after a node failure involves a full database transfer to the node being recovered. Moreover, Hermes does not consider the concept of durability. EPaxos describes a leaderless consensus algorithm, but does not consider recovery. IASO is the first online, incremental recovery scheme for leaderless systems. Its protocol is based on MINOS [20] and is optimized for minimal recovery latency and low throughput impact.

4.10 Discussion

IASO builds on prior work that assumes byte-addressable NVM as the durable layer [20]. Using SSDs instead of NVM is orthogonal to IASO’s protocol. It increases persistency delays due to file I/O, but does not affect correctness or semantics.

Empirical studies show that node failures rarely corrupt the local durable state. Past literature assumes that, in node outages, the local persistent storage remains intact [122], [123]. This observation motivates IASO’s assumption that durable state can be reused after recovery.

4.11 Conclusion

This chapter proposed *IASO*, the first online, incremental recovery scheme for leaderless distributed systems. It introduced algorithms for linearizable consistency and 5 persistency models. On average, IASO recovers a cluster after a node failure in 120 ms, while reducing the throughput by 8.7%. In comparison, a state-of-the-art recovery scheme for leaderless systems needs 360 seconds to recover, and reduces the throughput by 16.2%. Further, compared to a leader-based incremental recovery scheme, IASO reduces the recovery latency by $20.7\times$ and the throughput degradation by 62.4%.

Chapter 5 DIKTAMO: Extending CXL for Resilience to CPU Failures

5.1 Introduction

The latest versions of the Compute Express Link (CXL) standard [19], [27], [28], including CXL 3.0 and beyond, introduce hardware support for distributed compute nodes (CNs) to access shared-data stored in Memory Nodes (MNs) with hardware-enforced cache coherence at the granularity of a cache line. This capability blurs the line between shared-memory multiprocessors and distributed clusters, raising questions about how distributed applications can benefit from shared-memory programming semantics [30], [124], [125], [126], [127], such as sharing data across the cluster through global pointers [31]. However, this same capability complicates a common challenge in distributed computing: tolerance to hardware faults or software crashes. Traditionally, distributed applications such as databases and key-value stores share data at coarse granularities (e.g., records) and handle faults with relatively high overhead software techniques such as data replication or logging. With CXL, a different approach is needed, as data sharing occurs transparently in hardware at cache-line granularity.

The CXL specification describes support for reliability, availability, and serviceability (RAS) [19], [32], [33]. While the support is extensive, it has at least two limitations. First, it does not consider processor (i.e., host) failures; it only considers link, switch, and device (e.g., memory) failures. The second limitation is that, when a component fails, the specification tries to isolate it. There is no attempt to recover the workload to a consistent state.

In reality, handling processor failures is crucial, as CPUs and GPUs are responsible for many of the increasingly frequent failures in datacenters [34]. Moreover, when a node participating in a shared-memory program fails, the dirty data in its caches is lost. Hence, simply isolating the node and continuing execution results in using inconsistent application state. Correct execution requires first the recovery of the application state.

To address these limitations, this chapter extends the specification for CXL 3.0 and beyond to: 1) be resilient to the failure of CNs and 2) correctly recover the application after CN failure. We call the system *DIKTAMO*. DIKTAMO’s changes are mostly focused on the CXL Transaction Layer and minimally affect the CXL Link Layer.

To handle the failure of CNs, DIKTAMO augments the coherence transaction of a write with messages that propagate the update to a small set of other CNs (i.e., the *Replicas*). The replicas save the update in a per-node hardware *Logging Unit*. Such replication ensures resilience to CN failures. Then, at regular intervals, the Logging Units compress the data in their logs and dump it into the memory of Memory Nodes (MNs), which is resilient.

CN failure is detected by minimally extending the CXL Link Layer. Recovery is a distributed software-driven process that involves reading the directory state, reading the logs in the Logging Units, and updating the directory and memory state. The recovery attains a consistent application state, and enables forward progress of the application on the healthy nodes.

We evaluate DIKTAMO using simulations of a cluster of 16 CNs of 4 cores each, and 16 MNs interconnected via a CXL switch. We run parallel engineering applications and the YCSB key-value store. We find that DIKTAMO enables fault-tolerant execution with only 30% slowdown with three replicas (or 26% with two replicas) over the same platform without fault-tolerance support. Further, DIKTAMO is 2.82x faster than ensuring fault tolerance by using write-through writes to MNs. We also evaluate the impact of various aspects of DIKTAMO.

The contributions of this chapter are:

- DIKTAMO, an architecture that extends the CXL specification to make it resilient to processor failures.
- The DIKTAMO cache coherence protocol, which extends a traditional MESI protocol with data replication transactions.
- The DIKTAMO recovery scheme.

- An evaluation of DIKTAMO in a simulated CXL cluster.

5.2 Motivation

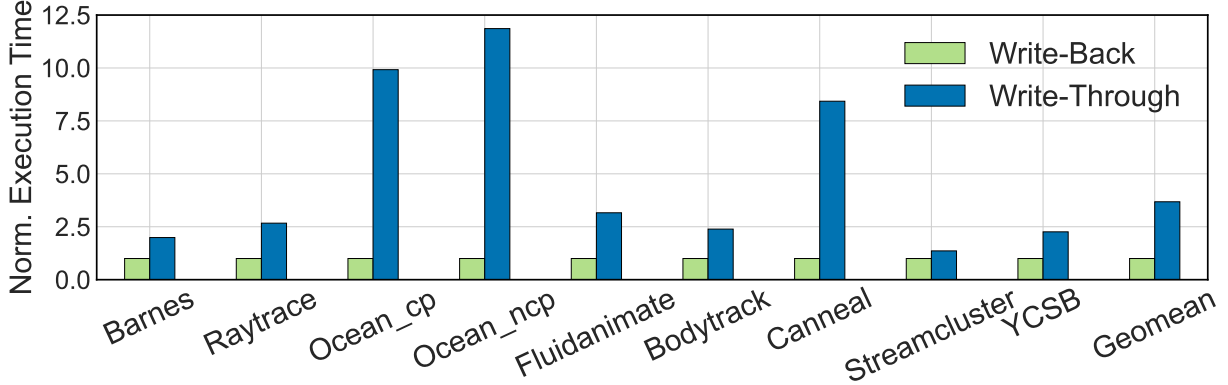


Figure 25: Execution time using a write-back or a write-through protocol. The bars are normalized to the former.

A possible approach to avoid data loss on a CN failure is to always use write-through writes to propagate updates to MNs—while assuming that the MN memory is resilient [29], [30]. However, this approach can introduce substantial performance overheads, especially in systems that support the TSO memory consistency model [128], [129]. In TSO, writes from the same core to different addresses need to be fully serialized. Hence, an update can only be written through to an MN after the previous update has completed its round trip to an MN (i.e., request and acknowledgment).

As an example, Figure 25 considers a CXL-DSM cluster of 16 CNs of 4 cores each, and 16 MNs interconnected via a CXL switch, running several 64-threaded shared-memory parallel applications. The architecture and applications are described in Section 5.6. The figure shows the applications’ execution times when using the MESI write-back protocol (WB) and when using a write through (WT) protocol. The former environment is not resilient, as dirty cached data can be lost on a CN failure. The latter environment is resilient because, like in previous literature [29], [30], we assume that MN memory is fault tolerant and that, as soon as a write updates an MN’s DRAM memory, the update cannot

be lost. MN memory can be made fault-tolerant through techniques like replication, persistency, or the use of a more expensive technologies. How this is done is not relevant to our work in this chapter. We assume the MN memory is safe and *do not penalize the MN memory access latency in any way*.

The bars in the figure are normalized to the WB execution time. We see that WT adds substantial overhead. On average, the execution time of WT is 3.86x longer than WB's. A major reason for WT's longer execution time is that, under TSO, multiple write-through updates from a core cannot overlap with each other.

As we seek to increase the resilience of CXL, we can get inspiration from existing techniques that distributed systems use, such as replication of data records across nodes to survive node loss [4], [6], [7] or data logging to recover old data versions [87], [88]. However, these techniques are typically implemented in software and operate at coarse granularity. In CXL-DSM, where data coherence is maintained at cache-line level and is enforced by hardware, we need solutions that are hardware-based, fine-grained, and low-overhead. In addition, resilient uses of CXL memory [29], [124] typically use transactions, which present different tradeoffs than our target general-purpose environment. Discussion on related work is presented in Section 5.8.

5.3 Overview of DIKTAMO

In this chapter, we address the two limitations of the CXL RAS model described above by extending the CXL specification to handle the failure of a CN and to correctly recover the application state after a CN failure. We call the proposal *DIKTAMO*. DIKTAMO's changes are mostly focused on the CXL Transaction Layer and minimally affect the CXL Link Layer.

5.3.1 Overall Operation

To accomplish its goals, DIKTAMO: 1) adds extra functionality to ordinary execution, 2) extends error detection, and 3) provides error recovery. In this subsection, we overview each of these three components.

1. Ordinary Execution. DIKTAMO augments the CXL Transaction Layer for writes that target the shared CXL memory (i.e., remote writes). Specifically, the transaction of a write performed by a core in a CN is augmented with messages that propagate the update to a small set of other CNs (i.e., the *Replicas*). The replicas save the update in a local hardware *Logging Unit*. The original write cannot commit until all replicas are updated. However, crucially, multiple writes from the same core can overlap their replica update. Then, at regular intervals, the Logging Units compress the data in their logs and dump it to the memory of MNs. As we describe in Section 5.3.2, we consider that the MNs are safe from faults.

Figure 26 shows the messages added to a write ST X ① from a core in CN0 to CXL memory when the system uses two replicas. In addition to the regular coherence protocol transaction for the write itself and before the write can commit, the protocol sends two *Replication (REPL)* messages ② with the update to the two replica nodes. The latter save the update in their log ③ and respond with a *REPL_ACK* message ④ to the originating core. Once the latter has received all the *REPL_ACK*s and the write coherence transaction has completed, the protocol sends two *Validation (VAL)* messages to the replicas ⑤ to notify them that replication was successful. After that, the write commits ⑥. When a log receives the *VAL*, it marks the log entry as validated.

The number of replicas per update (N_r) is a design parameter. It is a small value (e.g., 1–4) that depends on the machine size, trading off fault-tolerance and overhead [2], [8], [26]. To determine which CNs save the replicas for a given cache-line address, the hardware uses a hash function on the line address. Hence, all the updates to a

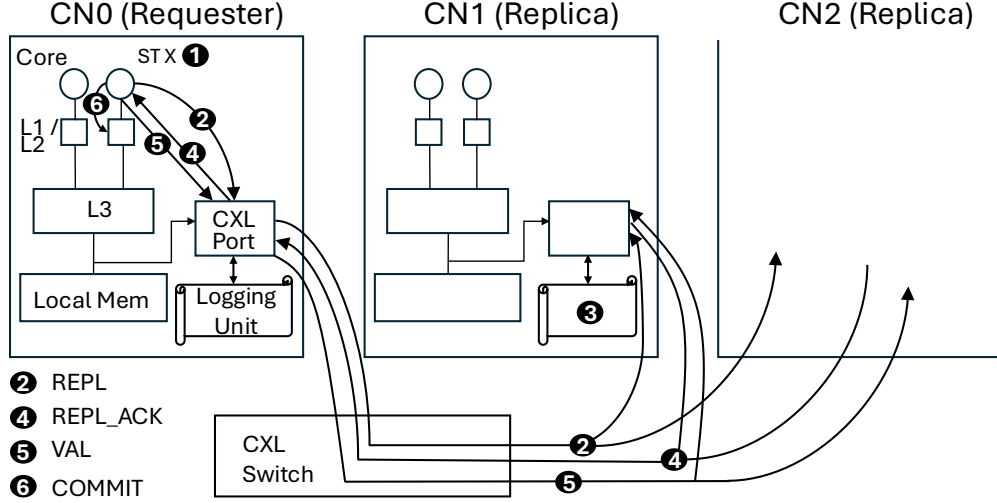


Figure 26: DIKTAMO extensions to the transaction of a remote write, assuming two replicas.

given line address are logged in the same set of N_r nodes.

Loads are unaffected by DIKTAMO. The same stands for writes to the local memory of the CNs.

2. Error Detection. CXL’s Link Layer detects errors in devices, switches, and links.

DIKTAMO uses CXL’s Link Layer for error detection and extends it minimally to detect CN failures. Specifically, each CXL switch includes an extra `Viral_Status` bit for each of the CNs that it is connected to. Additionally, a switch never responds on behalf of a failed CN, since DIKTAMO’s goal is correct execution—not just isolation. Section 5.5.1 describes error detection.

3. Recovery. After a CN error is detected, one of the live cores is informed with a CXL MSI message [19] to act as a Configuration Manager. Such core brings the workload execution to a halt and initiates a distributed software algorithm that disables the faulty CN and recovers the application state to the correct state immediately before the fault. Sections 5.5.2-5.5.4 describe the recovery.

Our main emphasis is on the support for ordinary execution.

5.3.2 Failure Model

The focus of this chapter is on CN failure, leaving link, switch, and device failure handling to the support already present in the CXL specification. For CN failures, we assume the standard *fail-stop* model: a faulty CN halts and ceases participation in the protocol. This model is widely used in practical crash-fault-tolerant (CFT) systems such as RAMCloud [7], FaRM [4], and Hermes [8], as it captures the dominant failure modes in modern data centers (e.g., power loss, kernel panic, or hardware crash).

Byzantine or malicious behavior (e.g., arbitrary message fabrication or equivocation) is out of scope. This assumption is widely consistent with prior work in crash-fault-tolerant replication [4], [7], [8], where the dominant concern is reliability against hardware or software crashes rather than adversarial behavior.

As in prior work [29], [30], we assume that MNs do not suffer faults and that any data that is written to them is safe. MNs are bigger, more expensive servers that have stronger RAS support, possibly based on techniques such as replication, persistent storage, or the use of a more expensive technologies. How such resiliency is accomplished is not relevant to our work in this chapter; we simply assume that the MN memory is safe. We *do not penalize the MN memory access latency in any way*, conservatively assuming that such MN memory resilience is attained for free.

The goal of DIKTAMO is not to attain the highest cluster availability. Rather, it is to recover the program state that existed in caches, directory, and memory right before the CN failure. No committed store can be lost.

5.4 Augmenting Ordinary Execution

This subsection describes the DIKTAMO extensions added to the ordinary execution: the message types, the Logging Unit, the logical timestamps, three variations of the DIKTAMO replication protocol, and the log processing.

5.4.1 DIKTAMO Messages

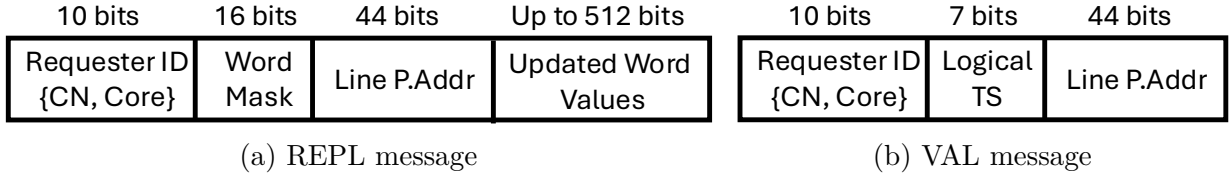


Figure 27: DIKTAMO’s REPL and VAL messages.

DIKTAMO introduces three new protocol messages: *Replication* (REPL), *Replication Acknowledgment* (REPL_ACK), and *Validation* (VAL). The REPL message (Figure 27a) is sent by a core in a requester CN to multiple replica CNs to replicate an update (or a set of coalesced updates to the same cache line). It carries: a) the *Requester ID*, which includes the CN and core IDs, b) a bit mask that shows which words within the line are updated by the store (in this chapter, we assume word granularity, but the design can be modified to use finer granularity), c) the physical address of the line, and d) the updated values of the words. The last field is of varying size, depending on the number of updates that are coalesced. The maximum size is a cache line size.

The REPL_ACK message is sent by the *Logging Unit* of each replica CN back to the requester core. It is sent after the unit has applied the REPL update to its hardware log. Upon receiving all REPL_ACKs, the requester core knows that replication has reached all replicas.

The VAL message (Figure 27b) is sent by the requester core to all replica CNs after it receives all their REPL_ACKs. VAL indicates that the write is committed and so the Logging Unit should record the update as such. VAL carries the Requester ID and the target cache line address. It also carries a logical timestamp (TS), which is used to order the updates as they are logged in the Logging Unit (Section 5.4.3).

5.4.2 Logging Unit

Each CN has a hardware Logging Unit that logs the data of incoming REPL messages and then, at regular intervals, compresses the logged data and sends it to the MNs, where it is saved. The Logging Unit offloads these operations from the local CPUs in each CN, reducing interference. The unit is linked to the CN’s CXL port (Figure 26).

10 bits	7 bits	46 bits	32 bits	1 bit
Requester ID {CN, Core}	Logical TS	Word P.Addr	Updated Word Value	Valid

Figure 28: DIKTAMO log entry.

Figure 28 shows the layout of a log entry. Each entry includes the data of a single store instruction. If a received REPL message carries multiple stores to the same line, then the Logging Unit splits the message into an equal number of log entries. Each entry has the Requester ID, the logical timestamp, the physical address and value of the updated word, and a valid bit. The latter is set only when the unit receives the **VAL** message from the requester.

The Logging Unit’s log is made of DRAM memory for density. To improve access speed, however, incoming REPL and VAL messages update a small SRAM memory called the *SRAM Log Buffer*. Once an entry in the SRAM Log Buffer receives the VAL, its Valid bit is set and the entry is moved to the DRAM log in the background. Persistency of the log is not required, since DIKTAMO employs a replication factor of N_r . This allows the system to tolerate up to N_r-1 CN failures without losing state.

Non-VAled entries correspond to writes that have not yet committed. These entries cannot be sent to the NMs.

5.4.3 Logical Timestamps

The DIKTAMO protocol decouples the allocation of a log entry in the SRAM Log Buffer from the saving of the entry into the DRAM log. The former is performed on reception of a

REPL, while the latter on reception of a VAL. Now, consider the case where the cores in a given CN issue two updates to the same memory location. The VALs are issued in the order of the updates—since they are issued at the point when an update commits. However, the CXL fabric can potentially reorder the two VALs and they can reach the SRAM Log Buffer out of order. Then, the entries would set their Valid bit and move to the DRAM log out-of-order. This would cause the recovery to apply them in the wrong order.

To prevent this problem, we add a logical timestamp to the VAL messages that is stored in the entries of the SRAM Log Buffer. Specifically, each L2 cache line has a 7-bit logical timestamp that is incremented on a write. When a core sends a VAL, it includes the timestamp of the line. In the destination CN, the SRAM Log Buffer saves the VAL’s timestamp in the log entry. However, it only pushes entries from that line into the DRAM log in timestamp order. As entries are pushed into the DRAM log, the timestamp is stripped-out.

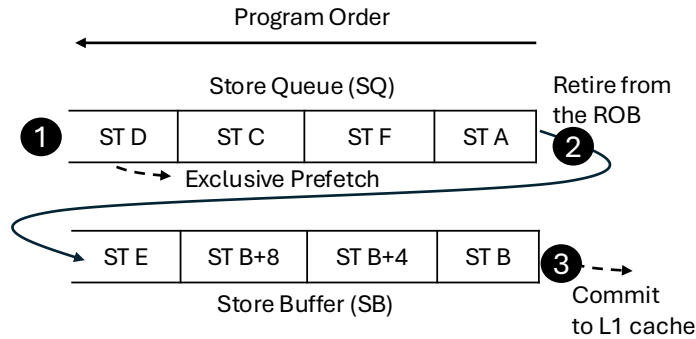


Figure 29: Multiple stores in an out-of-order core with TSO.

5.4.4 Three Variations of DIKTAMO’s Protocol

In general, when a store is ready to commit, it issues a message that invalidates all other cached copies of the line and brings the line in state Exclusive to the local cache. We call this the *Coherence* transaction. DIKTAMO augments the coherence transaction with a *Replication* transaction, which involves sending multiple REPLs, receiving multiple REPL_ACKs, and sending multiple VALs. Only when both the Coherence and the Replication

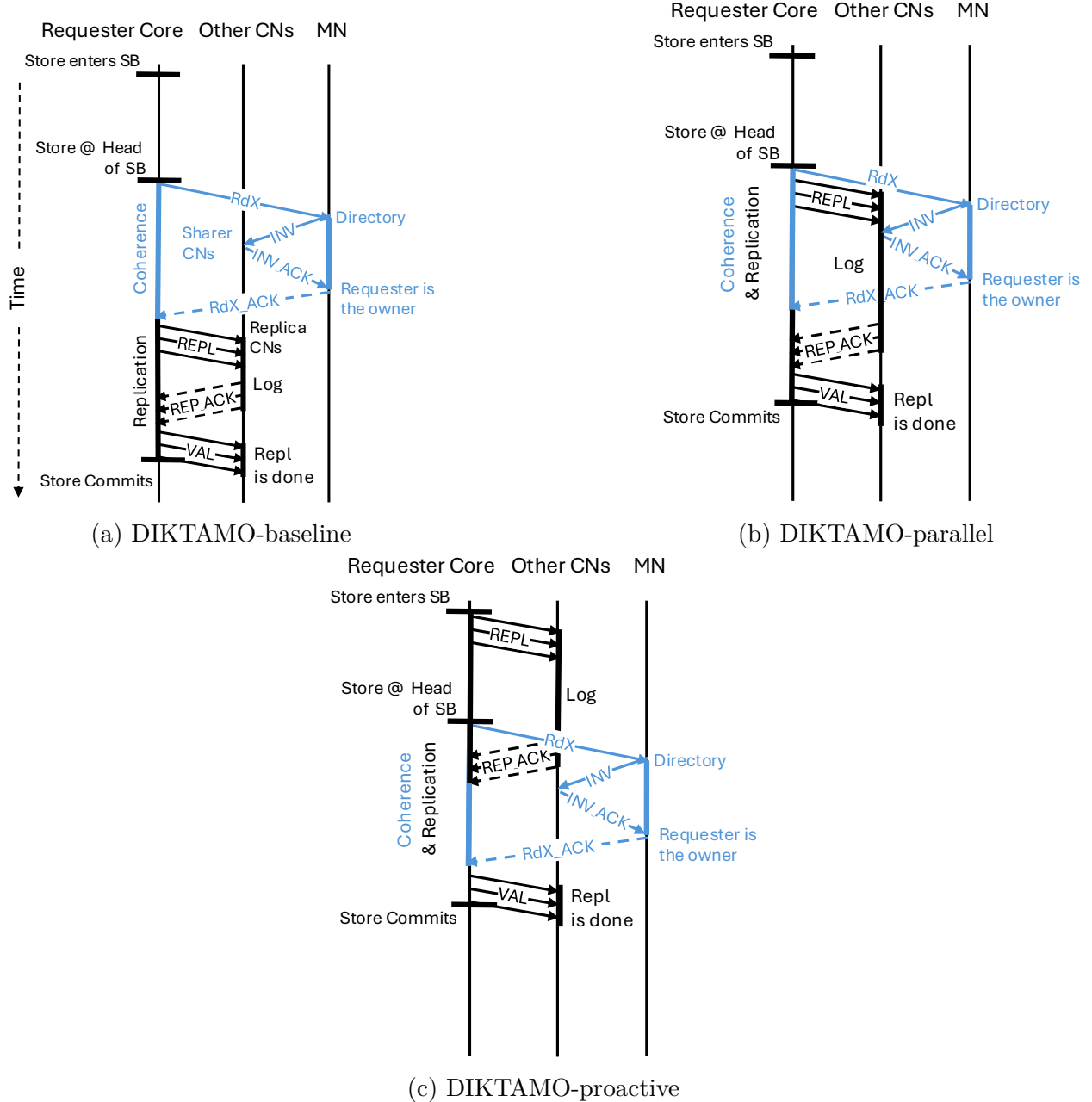


Figure 30: Timeline of the Coherence and Replication transactions of a store in the three variations of the DIKTAMO protocol.

transactions for a store complete can the store commit. In our design, we propose three variations of DIKTAMO that overlap the Coherence and the Replication transactions for a store differently. Before we present them, we describe the life of a store instruction.

1. The life of a Store Instruction. Figure 29 shows multiple store instructions in an

out-of-order core with TSO. There are two logical structures involved: the store queue (SQ) and the store buffer (SB). When a store enters the reorder buffer (ROB), it also enters the SQ. When its target physical address is resolved, an exclusive prefetch is sent to memory to bring the target cache line in Exclusive state to the L1 cache of the core ①. When a store reaches the head of the ROB (and hence of the SQ), it retires into the SB ②. Stores drain from the SB in order, without overlap: a store is merged with memory only when it reaches the head of the SB ③. We will call this point the store commit.

A common optimization is *store coalescing*. Consecutive stores in the SB that target different words of the same cache line and are not interleaved by a store to a different line are coalesced and committed to memory in a single transaction. For instance, in Figure 29, stores ST B, ST B+4, and ST B+8 will be coalesced and merged with memory together. In such cases, DIKTAMO sends a single REPL for all the coalesced stores. In the REPL (Figure 27a), the Word Mask sets the corresponding bits and the Values field includes all the updated word values. This reduces the network traffic of replication. All three DIKTAMO designs support coalescing.

2. First Design: DIKTAMO-baseline. In this design, the Replication transaction of a store starts only after the Coherence transaction has completed. When the Replication transaction completes, the store commits.

Figure 30a shows the timeline of a store from the time it enters the SB in the requester core until it commits. It includes the operations in the requester core, in other CNs, and in the MN that contains the directory. It shows the Coherence transaction in blue and the Replication one in black. We see that, when the store reaches the head of the SB, the Coherence transaction starts. We show the general case where the line is currently cached in Shared state by other nodes. A request for ownership (RdX) is sent, which reaches the directory in an MN. The latter sends invalidations to all sharer CNs, collects their ACK responses, marks the requester as the owner, and responds with the cache line (RdX_ACK).

The Replication transaction starts now. The CN with the requester core sends **REPL** messages to replica CNs (three in the example). Each replica CN logs the update and replies with a **REPL_ACK** to the requester. When the requester receives all the **REPL_ACKs**, it sends the **VALs** to all the replicas and commits the store—i.e., it drains the store from the SB and updates the cache line in L1. On reception of the **VAL**, each replica marks the log entry as valid.

3. Second Design: DIKTAMO-parallel. In this design, the Replication transaction of a store starts at the same time as the Coherence transaction. Both transactions overlap in time. Figure 30b shows the timeline of a store. Note that DIKTAMO-parallel can only send the **VALs** (and immediately commit) when two conditions are met: 1) the originating core has received all the **REPL_ACKs** and 2) the Coherence transaction has completed. The advantage of this design is that, by overlapping the two transactions, the store commits sooner.

4. Third Design: DIKTAMO-proactive. In both DIKTAMO-baseline

and DIKTAMO-parallel, replication is initiated no earlier than when a store reaches the head of the SB. As a result, replication is *serialized* across stores. However, all the necessary information for the replication is known as soon as the store retires and enters the SB (② in Figure 29). To send the **REPLs**, we do not need to wait for the many cycles that a store typically waits in the SB until it reaches the SB head [130].

To avoid waiting in the SB, we introduce DIKTAMO-proactive, where the **REPLs** for a store are sent out as soon as the store retires and enters the SB. Figure 30c shows the store timeline. As before, DIKTAMO-proactive can only send the **VALs** (and immediately commit) when both the **REPL_ACKs** are received and the Coherence transaction has completed.

Sending the **REPLs** earlier (e.g., when the exclusive prefetch is issued) is undesirable because the store is speculative. As a result, the store could be squashed due to a misspeculation, which would lead to two issues: (a) unnecessary network traffic, and (b)

nonVAL-ed entries in logging units that would require a dedicated mechanism to remove. Unlike an exclusive prefetch, a squashed Replication transaction has not even a useful side effect.

DIKTAMO-proactive has two performance advantages. First, it hides the latency of replication behind the existing SB queuing delays. *Second and crucially, it overlaps the REPL-REPL_ACK cycles of multiple retired stores waiting in the SB, delivering faster execution without violating TSO.* To see how, Figure 31 shows the case of stores ST A, ST B, and ST C waiting in the SB. The leftmost chart shows the timeline for DIKTAMO-parallel. Each store waits until it reaches the head of the SB and then performs the Replication transaction in sequence. In contrast, the rightmost chart shows the timeline for DIKTAMO-proactive. It issues REPLs for consecutive stores in order, but allows their REPL-REPL_ACK cycles to fully overlap. DIKTAMO-proactive can support as many outstanding replication requests as the size of the SB, without violating TSO.

5. DIKTAMO-proactive and Store Coalescing. DIKTAMO-baseline

and DIKTAMO-parallel trivially support store coalescing: as the two (or more) oldest stores in the SB are coalesced, only a single REPL message is sent for all of them. However, DIKTAMO-proactive requires some subtle redesign to support coalescing. This is because individual stores send REPLs as soon as they retire and are placed in the SB, while coalescing will potentially merge multiple of them later on, when they are draining from the SB, resulting in a single memory operation. Thus, blindly supporting coalescing in DIKTAMO-proactive would confusingly associate multiple REPLs-REPL_ACKs-VALs transactions with a single memory operation.

Consequently, to support coalescing, we slightly modify DIKTAMO-proactive as follows. When a store *B* is deposited into the SB, it never issues REPLs. If it finds that it is immediately preceded by a store *A* to another word of the same line, it is coalesced with *A*; otherwise it takes no action. When the next store *C* is deposited into the SB, if *C* cannot be coalesced with *B*, *B* issues its REPLs (which carry *B* and all the earlier stores coalesced

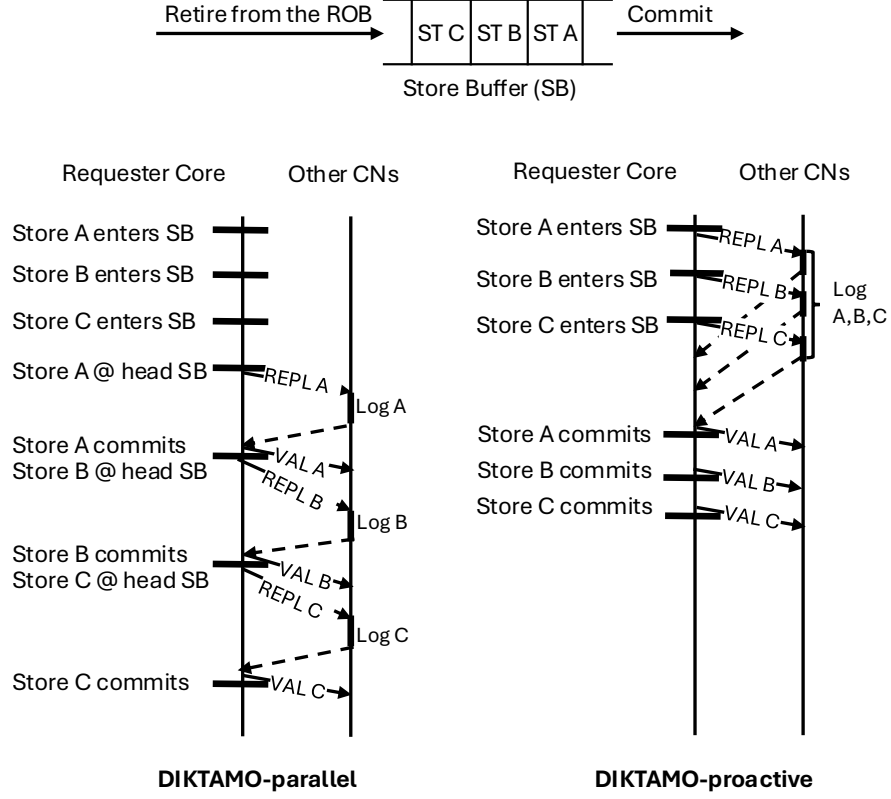


Figure 31: DIKTAMO-proactive overlaps the REPL-REPL_ACK cycle for multiple retired stores waiting in the SB.

with B). Also, if a store B reaches the head of the SB without having sent REPLs, it sends them at this point (carrying B and all the stores coalesced with B). This design supports coalescing, while initiating only a single REPLs-REPL_ACKs-VALs transaction for each commit.

5.4.5 DIKTAMO's Log Processing

At periodic intervals, the Logging Units of all the CNs compress and save the logs in the *background* into the MNs, where the data is safe from faults. Given that there are N_r replicas, groups of N_r nodes have the same data in their logs. We call these groups of CNs *Replica Groups*. To reduce overhead, the Logging Units in a Replica Group divide their work. Specifically, in a Replica Group, each of the Logging Units is in charge of saving only log entries with a range of physical addresses.

The process followed by each individual Logging Unit is as follows. The hardware reads

the DRAM log in sequence, from older to newer entries, extracting the log entries that this particular Logging Unit is in charge of saving (based on the physical address). As these log entries are being extracted, the hardware compresses them to save memory space and sends them in 64-byte messages through the network. For compression, we use gzip [131] with compression level 9, which attains an average compression factor of 5.8x in our experiments. Once a Logging Unit has saved all the log entries it is required to, it synchronizes with the other Logging Units through the MNs and then clears its whole log. Log entries that reach the MNs are maintained as logs for simplicity.

5.5 DIKTAMO and Failures

This subsection describes how DIKTAMO detects a CN failure and outlines the DIKTAMO recovery.

5.5.1 Failure Detection Mechanism

The Link Layer in CXL detects both transient and permanent errors in devices, switches, and links [19]. DIKTAMO reuses this support for all the messages it adds to the protocol. However, since the Link Layer does not support CN failures (2.5), DIKTAMO extends it as follows. First, the CXL specification indicates that CXL switches contain `Viral_Status` bits for various ports to indicate failure conditions in those ports [19]. DIKTAMO extends the switch design to have one `Viral_Status` bit for each CN that is connected to the switch. When the switch detects that a given CN is unresponsive, it sets the corresponding `Viral_Status` bit.

Additionally, the CXL specification indicates that, when a message is sent to a failed device, the device or nearest switch responds with a message with *poisoned* or *viral* data [19]. This is done so that the failed device is isolated. However, in an environment where multiple CNs share data, one cannot afford responding with incorrect data, as it could pollute the state of the running application. Consequently, in DIKTAMO, the switch

connected to a failed CN will not respond at all to requests to the failed node (and the latter will not respond either because of the failure model we assume).

Instead, as soon as a switch detects a failed CN, it sends a Message Signaled Interrupt (MSI) to a core in one of the live CNs to trigger a recovery. An MSI is the standard mechanism used by CXL devices to signal interrupts to a host CPU [19]. They function identically to how they are used in PCI Express (PCIe). The receiver core, which we call the *Configuration Manager* (CM) for this fault, will trigger a coordinated pause of all the CNs, and then initiate a software-driven recovery.

5.5.2 Recovery Protocol Overview

The objective of the software-driven coordinated recovery is to reconstruct the consistent state of the application immediately before the fault and enable forward progress of the application on the remaining nodes. The process involves accessing the directory state in the MNs and identifying the memory lines that the directory records as being present in the caches of the failed CN, either in state **Shared** or **Modified**. For the former, the directory entry must be updated by removing the failed CN as a sharer. For the latter, the logs in the MNs and in the Logging Units must be searched to find the latest update before the crash. Then, such update must be applied to memory, and the directory entry must be changed to indicate that there are no sharers. These operations are done by recovery software. Our main concern is to guarantee correctness and minimize complexity.

Figure 32 shows the messages exchanged by the nodes during recovery. Table 10 describes the messages. These messages can leverage the mailbox interfaces in the CXL RAS API (v0.8) [33], which enable host–device communication for reliability and error-handling events. The recovery process begins with the CM broadcasting an **Interrupt** to all live CNs in the cluster. These messages are directed to all the CPU cores in the CNs and to the per-CN Logging Units. Upon receiving the interrupt, cores and Logging Units complete all outstanding requests/operations, and pause. Then, they

respond with an `InterruptResp`. Once all `InterruptResps` have been received, the CM issues an `InitRecov` to all MNs, prompting software handlers in the directory controllers of the MNs to initiate recovery. The actions followed by these handlers set the directories and memories to a consistent state. They are described in Section 5.5.3.

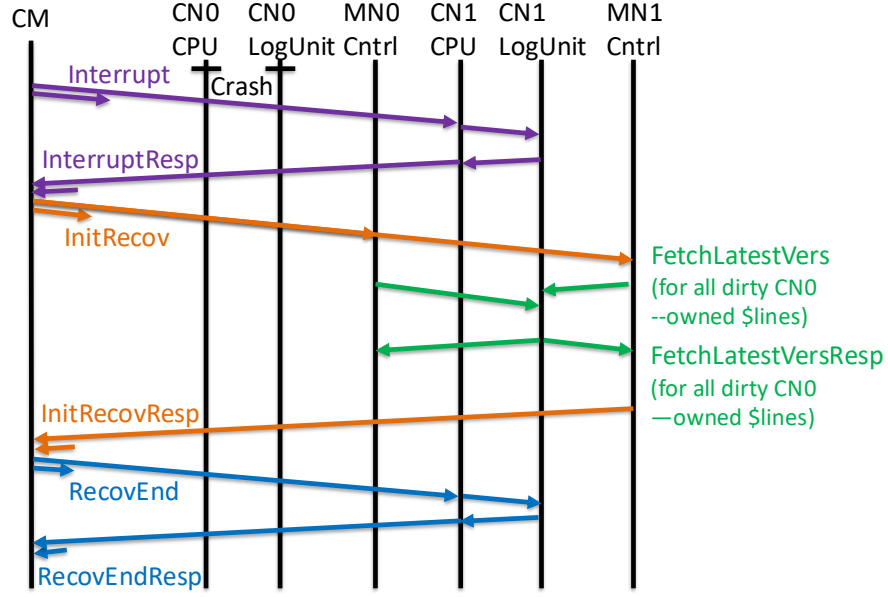


Figure 32: Message exchange during recovery in DIKTAMO.

Once the directory and memory in an MN has been brought to a consistent state, the handler in the directory controller sends an *InitRecovResp* to the CM. Upon the reception of *InitRecovResp* from all live directory controllers, the CM broadcasts a `RecovEnd` to all CNs, signaling the completion of the recovery protocol. Each node responds with a `RecovEndResp` and resumes program execution.

5.5.3 Recovery at the Directory Controller

Each directory controller runs a software handler that repairs the directory state (Algorithm 1). The handler performs two operations. The first one is to identify all the directory entries where the failed CN is a sharer, and remove the failed CN as a sharer. The second operation relates to the directory entries where the failed CN is the owner. After collecting the line addresses corresponding to these directory entries, the handler

Message	Definition / Usage
Interrupt	Sent by the CM to all live CNs to complete outstanding requests and other operations in the cores and in the Logging Units.
InterruptResp	Sent by CNs to confirm completion of outstanding requests and operations.
InitRecov	Issued by the CM to all MNs to initiate directory-level recovery actions by software handlers.
FetchLatestVers	Request from the directory controllers to the Logging Units for the latest logged versions of lines that were dirty in the failed CN's caches.
FetchLatestVersResp	Response with the latest logged versions of the requested cache lines.
InitRecovResp	Issued by handlers in directory controllers to acknowledge completion of directory-level recovery actions.
RecovEnd	Broadcast by CM to all live CNs to signal that recovery is complete and they can resume execution.
RecovEndResp	Sent by CNs to acknowledge completion of recovery and execution resumption.

Table 10: Recovery protocol messages.

sends them with a **FetchLatestVers** to the Logging Units of the relevant replica CNs.

Upon receiving such a request, the handler executed by each Logging Unit is detailed in Section 5.5.4. The handler returns a **FetchLatestVersResp** message. It contains, for each requested address, the sorted list of logged updates to the line (from latest to earliest). When the directory receives the **FetchLatestVersResps** from all replicas, it compares them. Consider one of these addresses. Typically, the latest logged value for the address should be the same in all replica logs. If so, this value is applied to memory. In cases where the CN failure occurred while the replica logs were being updated, it is possible that not all the replica logs have the same latest value. In this case, the latest logged update in all the N_r logs is picked and applied to memory. Finally, if the requested address is not found in the logs of the replica nodes, the log in the MN memory is searched and the latest update

Algorithm 1 Recovery of the Directory State by a Software Handler at the Directory Controller

```
1: Input: Set of dir entries marked as owned or shared by failed CN
2: Output: Corrected directory and memory state
3: For dir entries where the failed CN is a sharer:
4:     Remove the failed CN as sharer
5: For dir entries where the failed CN is the owner:
6:     Collect their addresses
7: Send these addresses with a FetchLatestVers to the Logging Units of the relevant replica CNs
8: Collect all FetchLatestVersResp responses
9: for all directory entries where the failed CN is the owner do
10:     Identify the latest cacheline version from FetchLatestVersResp or from the log in the MN memory
11:     Update memory and mark the directory entry as not shared
12: end for
```

found is applied to memory. In all cases, after the memory location is updated, the directory entry is marked as not shared by any CNs.

5.5.4 Log Traversal Strategy at Replica Nodes

When a Logging Unit receives a **FetchLatestVers** message, a software handler is invoked that performs the algorithm described in Algorithm 2. The handler traverses the log and, for each of the requested addresses, generates a sorted list (from latest in the log to earliest) of all the updates to the address found in the log. A single log traversal generates the lists for all the requested addresses. The lists are then included in a **FetchLatestVersResp** message sent to the directory controller.

5.6 Methodology

Architecture and Simulation Infrastructure. We simulate a distributed system with 16 compute nodes (CNs) of four cores each, and 16 memory nodes (MNs), interconnected

Algorithm 2 Replica Log Traversal by a Software Handler

```
1: for all addresses requested in FetchLatestVers do
2:     Scan the DRAM log in the Logging Unit from latest to earliest
3:     Collect a sorted list of all the versions found
4:     Include the list in FetchLatestVersResp
5: end for
6: Send FetchLatestVersResp to directory controller
```

System Configuration	
Compute Nodes (CN); Memory Nodes (MN)	16 CNs; 16 MNs
CPU Frequency; Logging Unit Frequency	2.4 GHz; 500 MHz
Cores per CN	4 Out-of-Order cores
Load ; Store Queue Size	128 ; 72
Cache Hierarchy & Local Memory (per CN)	
L1 Cache (Size; Associativity; RT Latency)	48 KiB; 12-way; 5 cycles
L2 Cache (Size; Associativity; RT Latency)	512 KiB; 8-way; 13 cycles
L3 Cache (Total Size; Associativity; RT Latency)	8 MiB; 16-way; 36 cycles
Cache line size	64 bytes
Memory Size per Node	512 GB
CXL Memory and Interconnect (per MN)	
DRAM Access Latency	45 ns
CXL Link Bandwidth	160 GB/s [133]
CXL Net RT Latency (includes directory lookup)	200 ns [132]
Logging Unit	
Log Size	SRAM: 4KB (4ns access); DRAM: 18MB
Log Dumping Period	2.5 ms
Replication Factor	$N_r=3$ copies

Table 11: Architecture parameters. RT means round trip.

via high-bandwidth CXL links (Table 11). There is a single CXL switch connecting all CNs to all MNs. Each core is out-of-order and has private L1/L2 caches, while all cores in a CN share an inclusive L3 cache. Logging units operate at 500 MHz. Each Logging Unit has a 4KB SRAM Log Buffer and a 18 MB DRAM log. The latter’s contents are compressed and periodically flushed to the MNs at the end of every Dumping Period. We model CXL using parameters from the literature [132], [133].

We assume that the memories in the MNs have features that make them fault tolerant. As indicated in 2.5, we follow prior work [29], [30] in this assumption. However, we conservatively assume that such features do not penalize in any way the latency of a read or write to the DRAM in the MNs.

We use the Structural Simulation Toolkit (SST) [134] to model the architecture. As of this writing, there are no commercially available systems that support full CXL 3.0 (or later) hardware coherence across multiple nodes, making simulation essential for studying an architecture like ours. Instruction traces of the program execution are collected per core

using Pin [135], and fed into the simulator.

The traces do not have timing information, but include all instruction and data accesses, and synchronizations (lock acquire request, lock release, barrier arrive, and barrier leave). In the simulations, each core advances time by executing its own thread’s instructions. We ensure that the synchronizations are correctly modeled. Specifically, only one thread can be inside a given critical subsection at a time (the others can be spinning), and threads spin on a barrier until all other threads have arrived. We compile the applications with -O3 to eliminate unnecessary local accesses.

Applications. We use a mix of compute- and memory-intensive applications from PARSEC [136] and SPLASH-2 [137], [138], and a custom key-value workload using Yahoo! Cloud Serving Benchmark (YCSB) [66], [67].

From PARSEC, we use *bodytrack*, *fluidanimate*, *streamcluster*, and *canneal*, which are a range of animation, computer vision, engineering, and data mining workloads. From SPLASH-2, we use *raytrace*, *barnes*, *ocean_ncp*, and *ocean_cp*, which are simulation workloads that stress memory-level parallelism with high performance computing and graphics workloads. For YCSB, we use a distributed key-value store backed by a hashtable, inspired by Bigtable [3].

All applications are multi-threaded and are executed with 64 threads running on all cores and fully utilizing the cluster. For each application, we execute 6.4B instructions or more. For the PARSEC and SPLASH-2 applications, we skip the initialization and start the evaluation when the application reaches the parallel subsection or the Region of Interest. We use the Medium size of the applications or larger. We place the shared data on the CXL memory, and the private data of the threads on the local memory of the CNs

For YCSB, the key value store has 1.6M records of size 1KB. The application utilizes a database organized in an array-like format. It performs 6.4 million put or get record operations to the CXL memory. It issues 80% reads and 20% writes with a uniform record access distribution. All accesses in YCSB reference the CXL memory.

Configurations. We evaluate five architecture configurations that affect the behavior of CXL (i.e., remote) memory accesses.

- **Write-Back (WB):** Conventional, non-resilient system where remote stores are committed to local caches without any replication and without simultaneously saving the update in any MN. This configuration not resilient: the failure of a CN results in data loss. It serves as a lower bound in terms of execution time.

- **Write-Through (WT):** Resilient system where remote stores are write through, updating the memory of a MN. As soon as a write updates a MN’s DRAM memory, we assume that the update cannot be lost—since the MN is fault tolerant. We do not penalize the memory access latency in any way. However, note that, under the TSO memory model [128], [129], multiple write-through updates from a core cannot overlap with each other.

- **DIKTAMO-baseline:** Baseline version of our resilient write-back DIKTAMO protocol, where remote stores are replicated to N_r remote CNs after their coherence transactions complete.

- **DIKTAMO-parallel:** Variant that optimizes DIKTAMO-baseline by overlapping the replication and coherence transactions of a store.

- **DIKTAMO-proactive:** Proposed DIKTAMO variant, which improves DIKTAMO-parallel by allowing the initiation of the replication for remote stores before they reach the head of the SB.

All DIKTAMO configurations use the same scheme for selecting replica CNs: a hash function maps each address to a Replica Group. N_r is set to 3, so that every remote store is replicated in three distinct Logging Units across the cluster. This number is *very conservative* (i.e., large) for our small 16-CN machine, given that it is similar to the number used in prior works evaluating datacenter-scale distributed systems [2], [139]. We use this number so we can model the additional traffic that DIKTAMO would add in a large machine.

5.7 Evaluation

5.7.1 Performance

Figure 33 shows the total execution time of our workloads when we run them with the different protocols. We normalize execution time to the *WB* configuration, which has high performance but is intolerant to faults.

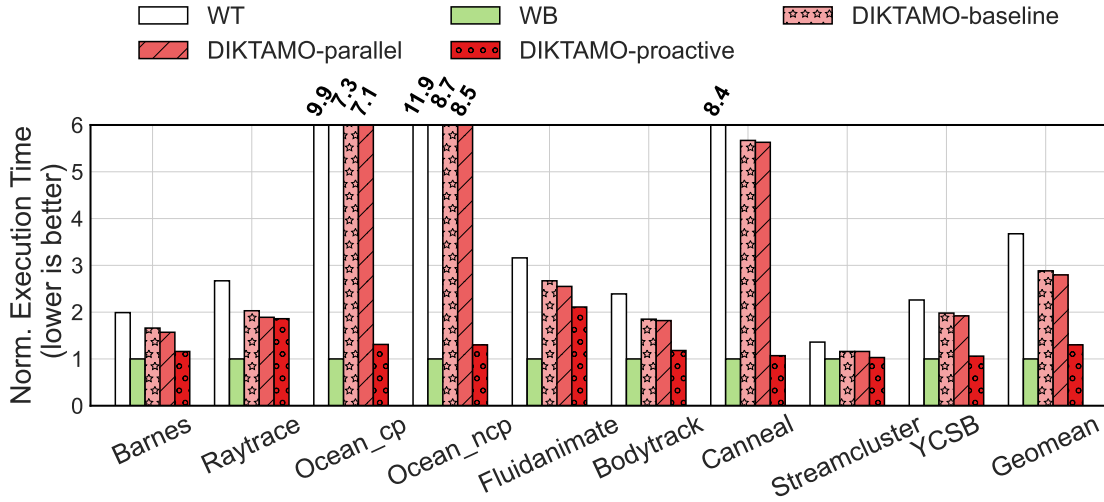


Figure 33: Execution time with different protocols.

We observe that *WT* is prohibitively expensive, especially for write-intensive workloads such as *Ocean-cp* and *Ocean-ncp*. Writing through remote writes can be very slow in TSO, as writes from the same core cannot overlap. As a result, the SB can fill-up, which results in the stall of the core execution. We find that the average execution time of *WT* over *WB* is 3.68x.

DIKTAMO-baseline serializes the Coherence and Replication transactions for a remote store. We see that it is still significantly slower than *WB*. On average, its execution time is 2.88x higher than *WB*. DIKTAMO-baseline penalizes write-intensive applications by slowing down SB draining. However, because of its write-back approach, it is a faster alternative to *WT*.

DIKTAMO-parallel overlaps the Coherence and the Replication transactions for a

remote store. However, we observe that the benefits over DIKTAMO-baseline are minor (only 3% on average). The reason is that the architecture we model is equipped with exclusive prefetching. As soon as a store’s target address is known, a request for ownership is sent to the memory system. Thus, when the store reaches the head of the SB, the Coherence transaction has often completed, exposing the Replication transaction as in DIKTAMO-baseline.

DIKTAMO-proactive proactively initiates the Replication transaction for a remote store while the store is queued in the SB. The advantage of this technique is twofold. First, it overlaps the Replication transaction with the store’s waiting time in the SB. Second, it *enables the overlapping of the Replication transactions for the stores in the SB*. Hence, this design substantially boosts performance. The execution time with DIKTAMO-proactive is on average only 30% longer than *WB*, while making the protocol resilient to failures.

Figure 33 shows that raytrace and fluidanimate do not benefit much from DIKTAMO-proactive. To understand why, Figure 34 shows the fraction of DIKTAMO-proactive REPLs that are sent when the store is at the head of the SB. This is the latest time that DIKTAMO-proactive can send the REPLs. If it sends them at this time, there is no difference between DIKTAMO-parallel and DIKTAMO-proactive. The higher this fraction is, the lower the difference between DIKTAMO-proactive and DIKTAMO-parallel is. We see that raytrace and fluidanimate have a high fraction, which explains their behavior in Figure 33. In these applications, a store often remains in the SB without sending REPLs, with the hope of coalescing with a future store that never appears.

Streamcluster also has a high fraction in Figure 34. However, because the fraction of requests that are remote writes is relatively small, all protocols behave similarly (Figure 33).

5.7.2 Characterization Analysis

In this subsection, we analyze several aspects of DIKTAMO-proactive.

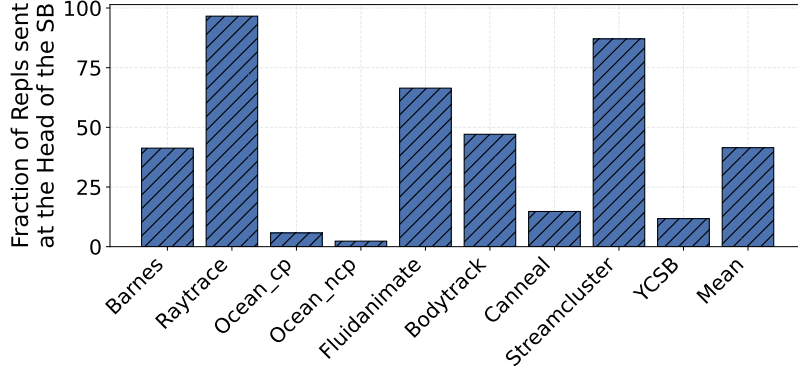


Figure 34: Fraction of REPLs sent by DIKTAMO-proactive when the store is at the head of the SB.

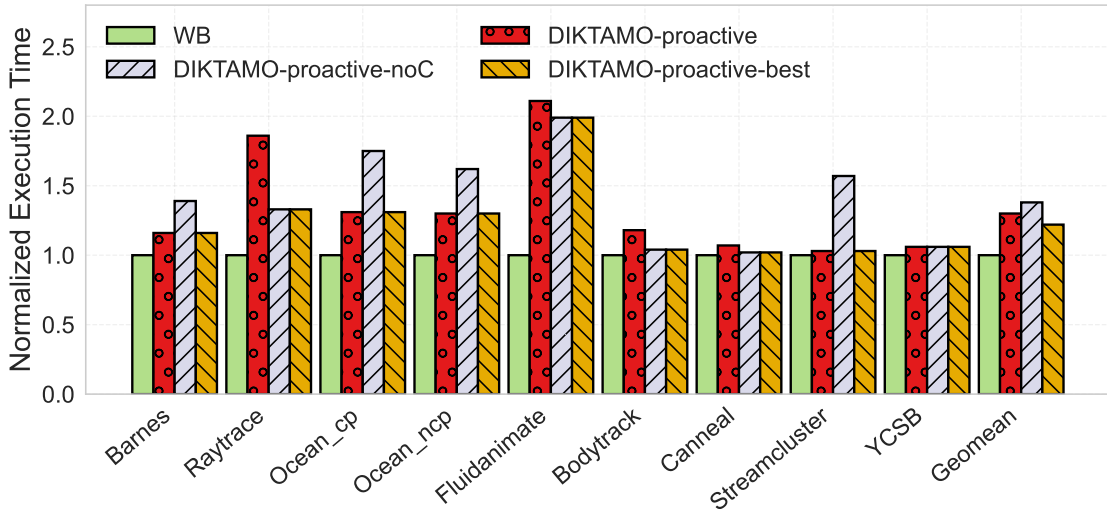


Figure 35: Comparing the execution time of DIKTAMO-proactive to that of a scheme without coalescing (noC).

Impact of Store Coalescing on DIKTAMO-proactive. As discussed in

Section 5.4.4.5, supporting store coalescing in DIKTAMO-proactive required a design that sometimes delays sending REPLs for stores in the SB. Hence, supporting coalescing affects performance in two ways. On the one hand, it can improve performance by combining multiple stores and reducing the number of write transactions. On the other hand, it may delay sending REPLs for some writes, reducing the ability to hide the Replication transaction.

Figure 35 examines whether the positive or the negative effects of supporting coalescing dominates. It repeats Figure 33 for *WB* and DIKTAMO-proactive (which supports

coalescing), and adds two more bars: DIKTAMO-proactive-noC (which does not support coalescing) and DIKTAMO-proactive-best, which is simply the best of DIKTAMO-proactive and DIKTAMO-proactive-noC. Comparing DIKTAMO-proactive and DIKTAMO-proactive-noC shows that DIKTAMO-proactive is only slightly better, and applications vary widely. Some applications like ocean-cp benefit from supporting coalescing, while others like raytrace lose performance. In general, it is hard to correlate Figure 35 with Figure 34: Figure 34 simply shows whether the REPLs were delayed to their latest times; it does not show when the REPLs were sent or whether fruitful coalescing occurred. Still, we see that raytrace, which delayed many REPLs in Figure 34 ends-up hurting from supporting coalescing in Figure 35. Finally, the lower execution time of DIKTAMO-proactive-best suggests that selectively enabling coalescing on and off may be interesting. DIKTAMO-proactive-best’s average overhead over *WB* is only 22%, while DIKTAMO-proactive’s is 30%.

Log Size. Each Logging Unit has a 4KB SRAM Log Buffer for pending transactions and a DRAM buffer to log transactions. To size the DRAM buffer, Figure 36 presents the maximum log size required per CN to buffer the data periodically flushed to MNs in DIKTAMO-proactive. We can see that the needs of the different applications vary widely. Based on this data, we size the DRAM buffer in each Logging Unit to be 18MB.

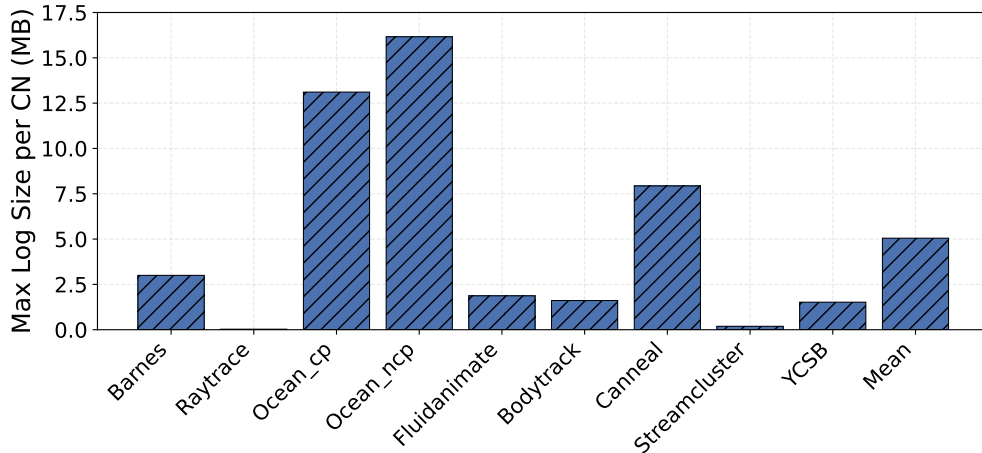


Figure 36: Maximum DRAM log size per CN in DIKTAMO-proactive.

CXL Network Bandwidth Consumption. Figure 37 shows the average CXL network bandwidth consumption by the 16 CNs in DIKTAMO-proactive. For each application, we show the component due to the coherence protocol (including remote reads, writes, invalidations, acknowledgments, REPLs, REPL_ACKs, and VALs), and due to periodically saving the compressed logs. We can see that the former dominates the bandwidth consumption, reaching up to 103 GB/s for the memory-intensive YCSB application. On the other hand, the bandwidth consumption of dumping logs is minimal and below 5 GB/s across all workloads. Hence, the execution time of applications is largely unaffected by the log dump operations.

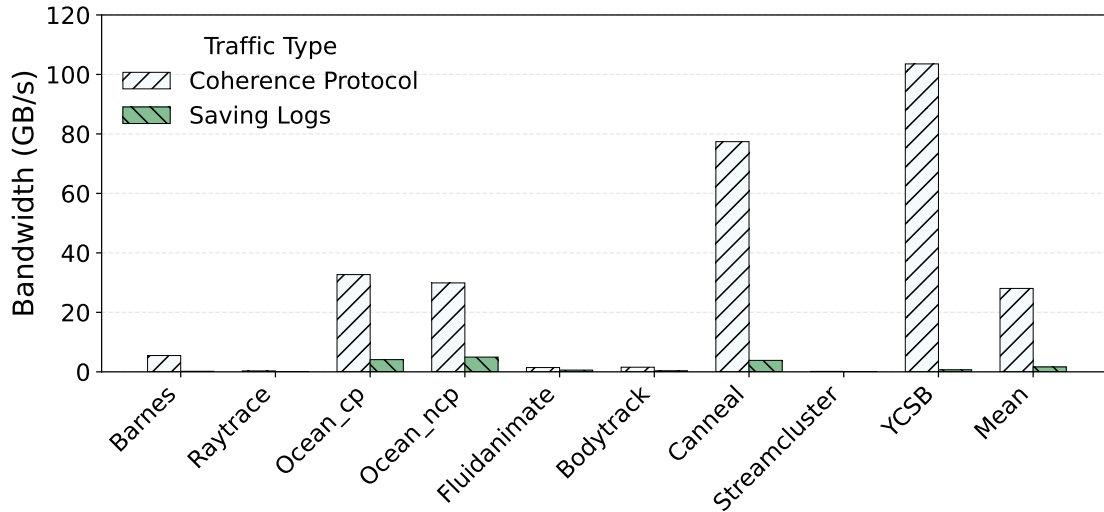


Figure 37: Average CXL network bandwidth consumption by the 16 CNs in DIKTAMO-proactive due to coherence protocol and saving logs.

Lines Owned by a Crashed Node. As part of the recovery, a handler traverses the directory and finds the addresses of all the lines that were Shared or Owned by the caches in the crashed CN. Owned lines can be Dirty or Exclusive, but the directory does not distinguish between them. For the Owned lines, the handler then traverses the logs to recover their value before the crash.

In this experiment, we trigger the crash of CN 0 at 12.5ms of execution in the applications under DIKTAMO-proactive, and count the number of Owned lines. The

directory gives us a number, and then we check the caches of the crashed CN to count those that are indeed Dirty. We label the difference between the two counts as Exclusive, although some of them may have been evicted silently from the caches. Figure 38 shows the result for the applications. For reference, the maximum number of different lines in the caches of a CN is 163K. The figure shows that, on average, less than 29K lines are Owned, although, in YCSB, such number reaches 119K.

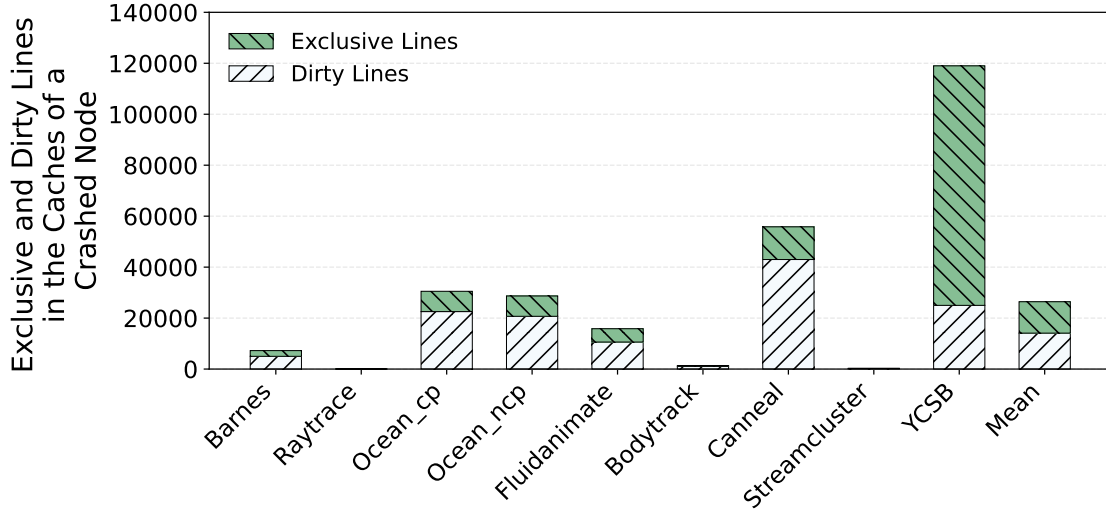


Figure 38: Exclusive and Dirty lines in the caches of a crashed CN in DIKTAMO-proactive.

Remote Write Latency Breakdown. Figure 39 shows the average remote write latency broken down into its components: $ROB \rightarrow ROBHead$ is the time from when the store is deposited into the ROB until it retires; $ROBHead \rightarrow SBHead$ is the time from store retirement until the store reaches the head of the SB. $SBHead \rightarrow CohEnd$ is the additional time it takes for the coherence transaction to complete, bringing the line in state Exclusive to L1; and *Replication Time* is any additional time beyond CohEnd that the write needs to remain in the SB waiting for the Replication transaction to complete. After $SBHead \rightarrow CohEnd$ and *Replication Time*, the write commits. Results are shown for both *WB* and *DIKTAMO-proactive*.

We see that, in *WB*, $ROB \rightarrow ROBHead$ and $ROBHead \rightarrow SBHead$ dominate. As we move to *DIKTAMO-proactive*, $ROB \rightarrow ROBHead$ remains mostly the same, while

$ROBHead \rightarrow SBHead$ increases in all cases and becomes dominant. Because of the need for writes to perform the Coherence transaction, the average write spends more time in the SB. In addition, for the same reason, *Replication Time* appears in several applications. The applications that have the highest increase in average remote write latency are bodytrack and streamcluster.

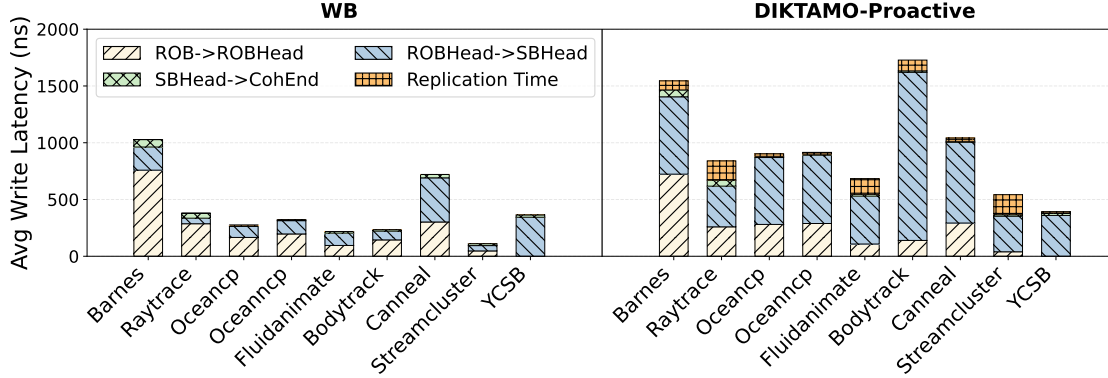


Figure 39: Remote write latency breakdown.

Correlating these write latency increases with the increases in execution time in Figure 33 is not easy, given that not all the remote write latency is visible in the critical path of the application execution, and there are many factors that affect execution time. What is clear is the correlation between the *Replication Time* in Figure 39 and the fraction of REPLs sent at the SB head shown in Figure 34. Applications with a high fraction of REPLs sent at the SB head in Figure 34 (raytrace, streamcluster and, to a lesser extent, fluidanimate, barnes, and bodytrack) have a high *Replication Time* in Figure 39.

5.7.3 Sensitivity Analyses

In this subsection, we perform several sensitivity analyses.

CXL Link Bandwidth. Figure 40 shows the sensitivity of DIKTAMO-proactive and *WB* execution time to CXL link bandwidth. We vary the bandwidth from the default 160 GB/s to 20 GB/s. All bars are normalized to *WB* with 160 GB/s. We show three representative applications and the geometric mean of all applications. Figure 40 shows three different

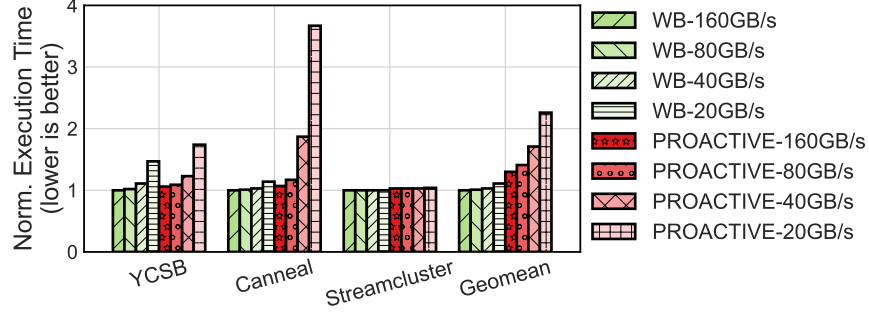


Figure 40: Sensitivity of DIKTAMO-proactive and *WB* execution time to CXL link bandwidth. All bars are normalized to *WB* with 160 GB/s.

behaviors. For YCSB, both *WB* and DIKTAMO-proactive suffer when the available bandwidth becomes limited. For canneal, DIKTAMO’s replication messages congest the network at low bandwidths, while *WB* is unaffected. Finally, for streamcluster, both *WB* and DIKTAMO-proactive are unaffected. On average, a low CXL link bandwidth hurts DIKTAMO-proactive more than *WB*.

Replication Factor. Figure 41 shows the execution time of the applications with DIKTAMO-proactive for different replication factors N_r . All bars are normalized to the default $N_r=3$, a very conservative value (Section 5.6). On average, the execution time increases with N_r . For example, a system with $N_r=4$ takes 2% longer to execute than one with $N_r=3$. Remote write-intensive applications like ocean-cp and ocean-ncp are significantly affected.

Scalability with Number of CNs. Past work suggests that CXL systems with 16 CNs are a good design point [127], [132], and some CXL switches support 32 ports [140] (16 CNs and 16 MNs for us). Figure 42 shows the execution time of DIKTAMO-proactive and *WB* as we change the number of CNs from 4 to 16. The bars are normalized to *WB* for 16 CNs. The figure shows that, across all applications, DIKTAMO-proactive has good scalability. On average, going from 4 to 16 CNs, the execution time decreases by 3.15x and 3.07x for *WB* and DIKTAMO-proactive, respectively.

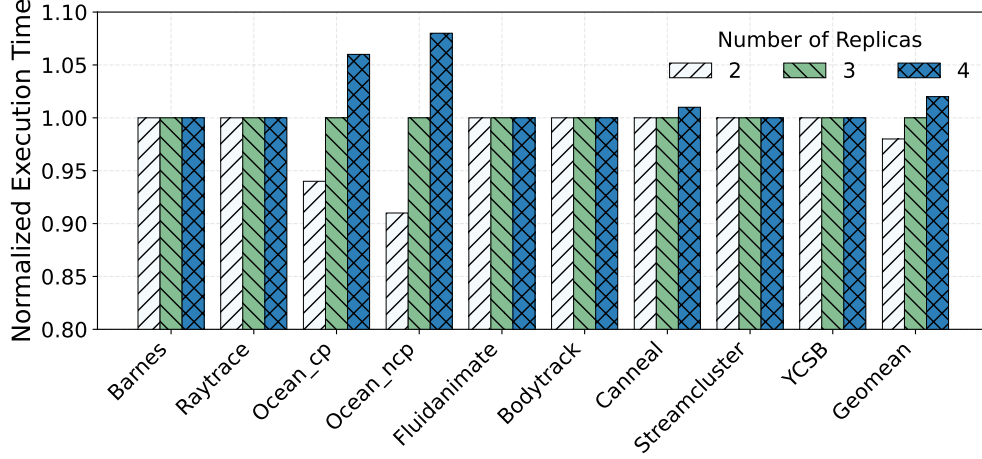


Figure 41: Execution time with DIKTAMO-proactive varying the replication factor N_r . All bars are normalized to $N_r=3$.

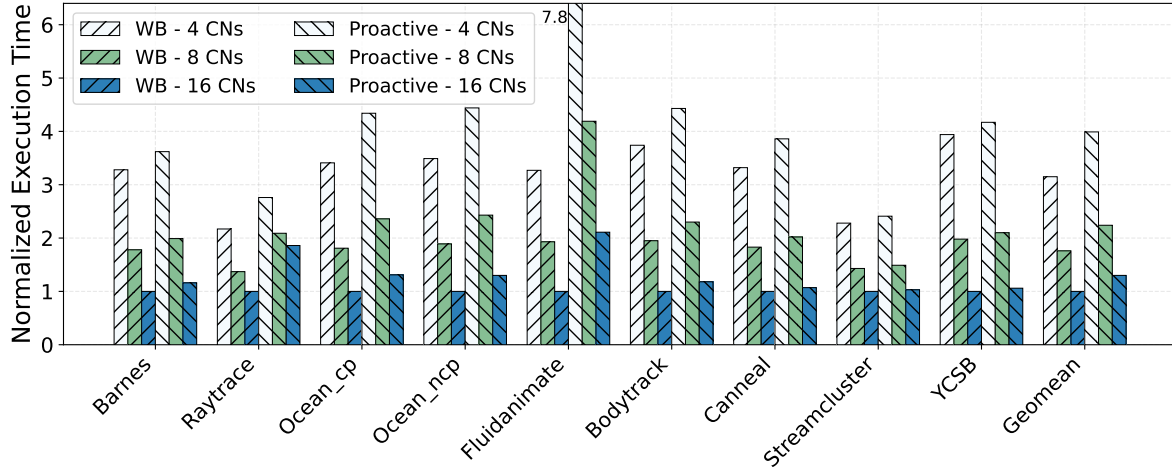


Figure 42: Execution time of DIKTAMO-proactive and WB varying the CN count. Bars are normalized to WB with 16 CNs.

5.8 Related Work

Related uses of CXL. Several systems have leveraged CXL for memory expansion and pooling. CXL-ANNS [141] uses CXL-attached memory to scale approximate nearest neighbor workloads by caching the neighbors in local memory and utilizing prefetching. Memstrata [142] introduces a memory allocator in the hypervisor and the first hardware-managed memory tiering for CXL. Pond [132] addresses memory stranding issues by enabling flexible pooling. StarNUMA [143] employs a CXL-based memory pool to

reduce costly remote accesses in large NUMA systems.

With CXL 3.0, memory sharing has enabled new distributed system abstractions. HydraRPC [127] implements low-latency RPC over shared CXL memory. CXLfork [144] enables fast process forking over CXL shared memory. TrEnv [145] explores the use of shared memory over CXL to accelerate serverless functions.

Tigon [124] and CtXnL [146] are transactional systems over CLX. They both use software cache coherence for the majority of the data in the program and CXL-supported hardware cache coherence for metadata. Both works provide little or no discussion on recovery. Tigon uses conventional database transaction logging and recovers by concurrently restoring the log. CtXnL says it "relies on transaction processing frameworks to ensure data consistency". Hence, we can say that recovery in these systems is software-based and coarse-grained. AptA [29] examines a CXL environment where nodes can fail. Its main emphasis is on a hardware cache coherence protocol for resilience. It uses write-through caches, offloads invalidations from the critical path, and uses a coherence-aware scheduler. It assumes a transactional system and targets a FaaS environment.

In contrast to these three works, DIKTAMO targets general-purpose (not just transactional or application-specific) environments. Unlike AptA, it uses a write back protocol. Unlike all the schemes, it uses a novel hardware-based, fine-grained recovery scheme for performance and generality.

Resilience through replication and coherence protocols. Replication has long been used to achieve both resilience and performance in distributed and shared-memory systems. Works such as FARM [4], PAR [6], and RAMCloud [7] replicate data across multiple nodes for high availability and low-latency access. Other work explores cache-line replication for fault tolerance in processor and memory systems. ICR [147] replicates active L1 lines by overwriting predicted-dead blocks. Replication Cache [148] extends this with a small replica cache storing dirty-line copies on each write. Fernandez-Pascual et al. [149]

ensure at least one valid copy by delaying invalidations until safe transfer, tolerating interconnect faults. Dvé [150] replicates DRAM cache lines across NUMA sockets to provide resilience to memory errors, and maintains coherence among the replicas, but does not handle host-wide failures. In contrast, DIKTAMO logs updates in the context of CXL, enabling recovery from node crashes.

Shared-data reliability and coherence fault-tolerance has been studied in the past for shared memory multiprocessors [149], [151], [152]. Our work in this chapter complements these efforts by providing high-performance fault tolerance in shared-memory distributed CXL systems.

High performance TSO. As an alternative to DIKTAMO, we have considered the WT environment (Figure 33): a write-through protocol with MN memory that is assumed to be fault-free without any extra access latency cost. Such a design is substantially slower than DIKTAMO, in part because, under TSO, multiple write-through updates from a core cannot overlap with each other. Recently, there are two techniques that redesign the cache coherence protocol to allow writes to overlap and still support TSO [153], [154]. Such designs could provide an alternative to DIKTAMO, at the cost of significant hardware protocol changes: hardware linked lists and double transactions, respectively. We leave the analysis to future work.

5.9 Discussion

In this chapter, we place the Logging Units (LUs) next to Compute Nodes (CNs) because we assume that CN to Memory Node (MN) communication is slower than CN to CN communication. An alternative design would be to place the LUs in the MNs. Based on our assumptions, this design would have longer-latency round trip writes to the log than the design in this chapter. However, it would have several advantages. First, it would not need the extra step of regularly sending the compressed logs to the MNs. Second, it would avoid the need to modify the CNs, and the changes would be limited to the MNs—possibly

in a CXL-based module. Finally, since we assume that the MNs are resilient, a write would only need to be logged in a single log, with no replication. Further analysis of this topic is left for future work.

5.10 Conclusion

To address limitations of the CXL specification, this chapter extends it to make it resilient to node failures. We call the system *DIKTAMO*. DIKTAMO augments the coherence transaction of writes to create replicas in the Logging Unit of a few other nodes. Recovery involves using the logs in the Logging Units to bring the directory and memory to a correct state. Our evaluation with 16 4-core nodes shows that DIKTAMO enables fault-tolerant execution with only 30% slowdown with three replicas (or 26% with two replicas) over the same platform with no fault-tolerance support. Further, DIKTAMO is 2.82x faster than ensuring reliability by using write-through writes.

Chapter 6 Future Work

Emerging interconnect technologies such as Compute Express Link (CXL) significantly expand the design space of disaggregated memory systems. While this thesis focuses on specific mechanisms for reliable and efficient distributed and shared-memory architectures, several promising directions remain open. In particular, CXL’s support for shared memory, memory pooling, and fabric-level communication creates opportunities to rethink distributed consistency protocols, memory node failure handling, and large-scale memory architectures. The following subsections outline several future directions that build on these capabilities.

Leveraging CXL for Efficient Leaderless Consistency and Persistency. One promising direction is to explore how emerging interconnects such as Compute Express Link (CXL) can improve the efficiency of leaderless consistency and persistency models. While leaderless designs provide scalability and fault tolerance, they often incur significant computational and network overhead due to extensive coordination and metadata propagation across nodes. These costs can limit their efficiency in large-scale deployments.

CXL’s shared-memory capabilities may help alleviate these limitations. In particular, future systems could place selected protocol metadata in shared CXL memory and access it using cache-coherent semantics, rather than propagating updates through explicit network messages. Such an approach could reduce redundant communication and shift certain coordination tasks off the critical path of operations.

Additionally, exposing key protocol metadata through shared-memory abstractions may simplify the implementation of leaderless systems. By reducing the amount of explicit coordination logic required across nodes, future designs could make consistency and persistency models easier to implement and reason about, while improving overall system efficiency.

Handling Memory Node Failures in CXL-Based Disaggregated Memory

Systems. While DIKTAMO assumes fault-free Memory Nodes (MNs) to focus on failures at the Compute Node (CN) level, real CXL-based disaggregated systems may also experience failures at the MN level. Prior work largely addresses CN failures [29] or considers restricted models of MN failures. For example, MN failures are studied either when MNs store read-only copies—thus avoiding consistency concerns [155]—or when MNs do not share data across nodes [132], [156]. In more realistic disaggregated systems, however, applications may share mutable data (e.g., databases) across memory nodes. In such environments, MN failures must be addressed carefully to (i) avoid data loss, (ii) maintain correct and coherent data, and (iii) ensure forward progress. First, preventing data loss requires replicating data across multiple MNs. A simple approach is primary–backup replication, where each datum has a designated primary location and one or more replicas. Managing replication at the MN level is a natural design choice, as MNs already manage the physical storage of data. An extension of this approach could take advantage of Leaderless replication as covered by this thesis. Second, the system must maintain data correctness and coherence. Since CNs may cache shared data in their local cache hierarchies, the system must track the ownership and sharing state of cachelines. Each MN can maintain a directory that tracks cached copies of its data. To tolerate MN failures, this directory metadata must also be replicated. Finally, the system must ensure forward progress after a failure. This requires timely failure detection —e.g., through a Configuration Manager or the CXL Fabric— and mechanisms to redirect requests to surviving replicas. Replicating both data and directory state enables the system to continue serving requests without indefinite stalls.

Towards Cross-Rack Shared Memory with CXL. CXL is designed primarily to operate within a rack-scale domain [157]. In future architectures that support cross-rack shared memory, it would be valuable to investigate how to efficiently integrate in-rack CXL

communication with cross-rack data transfers. Cross-rack communication could be implemented using technologies such as Remote Direct Memory Access (RDMA). However, several challenges arise, including differences in access granularity (cacheline versus page) and the need to integrate RDMA-based communication with the CXL fabric.

Such an approach would be particularly beneficial for applications that span multiple racks and need to exchange data efficiently. Applications may be distributed across racks for reasons such as storage capacity, load balancing, or resilience to power failures. Storage considerations are especially important, as accessing memory across racks may still be an order of magnitude faster than accessing local storage devices such as SSDs. Moreover, this direction could be extended to support coherence domains that span multiple racks, enabling larger-scale shared-memory systems.

A Potential CXL-Based Alternative to RDMA. Global Integrated Memory (GIM), introduced in CXL 3.1 [158], enables host-to-host, host-to-device, device-to-host, and device-to-device transfers with remote memory access semantics. In essence, it provides functionality conceptually similar to RDMA. A key difference lies in the access granularity (cacheline versus page) and in the configuration process. RDMA typically relies on a queue-pair-based setup with pre-pinned buffers and established virtual-to-physical address translations that are registered with the Network Interface Cards (NICs) of the participating nodes. In contrast, GIM is more transparent: the CXL fabric is inherently aware of the memory regions that participate in GIM, eliminating the need for such explicit setup overhead. Moreover, GIM does not rely on host NICs, but instead operates directly through the CXL fabric. Consequently, a systematic study that evaluates potential performance and total cost of ownership (TCO) advantages, and identifies clear use cases for GIM, is warranted.

Chapter 7 Conclusion

This thesis has presented three contributions aimed at improving the performance and resilience of modern distributed and shared-memory systems in datacenters, addressing complementary challenges in system design: high-performance execution, durability, and fault tolerance.

First, to enable high-performance and programmable distributed systems, we introduced MINOS. First, MINOS-B provides a suite of distributed algorithms designed to support Linearizable consistency with five different types of persistency models for leaderless systems. We further optimized these algorithms by offloading them to a new SmartNIC-based architecture, which we termed MINOS-O. The MINOS-O architecture leverages hardware-level optimizations such as selective data coherence between the host and SmartNIC, message batching, and broadcasting to improve system performance. Our evaluation showed that MINOS-O substantially reduces request latency and increases throughput compared to MINOS-B. Specifically, MINOS-O reduced the average end-to-end latency of two microservice functions by 35%.

Second, we addressed the challenge of resilience in leaderless distributed systems by introducing IASO, an optimized online incremental recovery mechanism. IASO supports Linearizable consistency and multiple persistency models, offering a fast and efficient recovery process. Implemented on a 6-node cluster, IASO demonstrated its effectiveness in recovering a system after a node failure in just 120 milliseconds on average, while reducing the throughput of the running system by only 8.7%. This is a significant improvement compared to a state-of-the-art recovery system, which required 360 seconds to recover and incurred a throughput reduction of 16.2%.

Finally, we addressed the problem of resilience in emerging distributed shared-memory systems enabled by modern interconnects such as Compute Express Link (CXL). We introduced DIKTAMO, a hardware–software co-designed architecture that extends the

CXL coherence protocol to tolerate node failures. DIKTAMO augments write coherence transactions with lightweight, cacheline-granular replication to a small set of peer nodes, where updates are temporarily stored in hardware Logging Units, ensuring that modified data remains recoverable even if a node holding dirty cachelines fails. Recovery leverages these logs to reconstruct correct directory and memory state, enabling consistent application execution without reverting to costly write-through policies. By overlapping replication with store execution, DIKTAMO preserves the performance benefits of write-back caching. Our evaluation shows that DIKTAMO enables fault-tolerant shared memory with only a 30% slowdown (26% with two replicas) compared to a system without fault tolerance, and is $2.82\times$ faster than a write-through baseline.

In summary, the combined results of MINOS, IASO, and DIKTAMO showcase complementary advances across three dimensions of modern systems in datacenters. MINOS improves the performance and programmability of leaderless distributed protocols, IASO enables fast and efficient recovery in distributed systems, and DIKTAMO extends resilience to emerging shared-memory architectures. Together, these contributions provide a foundation for building high-performance, durable, and fault-tolerant systems across both distributed and shared-memory environments.

References

- [1] N. Bronson, Z. Amsden, G. Cabrera, P. Chakka, P. Dimov, H. Ding, J. Ferris, A. Giardullo, S. Kulkarni, H. C. Li, M. Marchukov, D. Petrov, L. Puzar, Y. J. Song, and V. Venkataramani, “TAO: Facebook’s Distributed Data Store for the Social Graph,” in *2013 USENIX Annual Technical Conference, San Jose, CA, USA, June 26-28, 2013*, A. Birrell and E. G. Sirer, Eds., USENIX Association, 2013, pp. 49–60. [Online]. Available: <https://www.usenix.org/conference/atc13/technical-sessions/presentation/bronson>
- [2] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s Highly Available Key-Value Store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*, T. C. Bressoud and M. F. Kaashoek, Eds., ACM, 2007, pp. 205–220. DOI: 10.1145/1294261.1294281 [Online]. Available: <https://doi.org/10.1145/1294261.1294281>
- [3] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A Distributed Storage System for Structured Data,” *ACM Trans. Comput. Syst.*, vol. 26, no. 2, 4:1–4:26, 2008. DOI: 10.1145/1365815.1365816 [Online]. Available: <https://doi.org/10.1145/1365815.1365816>
- [4] A. Dragojevic, D. Narayanan, M. Castro, and O. Hodson, “FaRM: Fast Remote Memory,” in *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2014, Seattle, WA, USA, April 2-4, 2014*, R. Mahajan and I. Stoica, Eds., USENIX Association, 2014, pp. 401–414. [Online]. Available: <https://www.usenix.org/conference/nsdi14/technical-sessions/dragojevi%5C%C4%5C%87>

- [5] A. Dragojevic, D. Narayanan, E. B. Nightingale, M. Renzelmann, A. Shamis, A. Badam, and M. Castro, “No Compromises: Distributed Transactions with Consistency, Availability, and Performance,” in *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, E. L. Miller and S. Hand, Eds., ACM, 2015, pp. 54–70. DOI: 10.1145/2815400.2815425 [Online]. Available: <https://doi.org/10.1145/2815400.2815425>
- [6] R. Alagappan, A. Ganesan, E. Lee, A. Albarghouthi, V. Chidambaram, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Protocol-Aware Recovery for Consensus-Based Storage,” in *2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, D. Malkhi and D. Tsafir, Eds., USENIX Association, 2019. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/alagappan>
- [7] D. Ongaro, S. M. Rumble, R. Stutsman, J. K. Ousterhout, and M. Rosenblum, “Fast crash recovery in RAMCloud,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles 2011, SOSP 2011, Cascais, Portugal, October 23-26, 2011*, T. Wobber and P. Druschel, Eds., ACM, 2011, pp. 29–41. DOI: 10.1145/2043556.2043560 [Online]. Available: <https://doi.org/10.1145/2043556.2043560>
- [8] A. Katsarakis, V. Gavrielatos, M. R. S. Katebzadeh, A. Joshi, A. Dragojevic, B. Grot, and V. Nagarajan, “Hermes: A Fast, Fault-Tolerant and Linearizable Replication Protocol,” in *ASPLOS ’20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, J. R. Larus, L. Ceze, and K. Strauss, Eds., ACM, 2020, pp. 201–217. DOI: 10.1145/3373376.3378496 [Online]. Available: <https://doi.org/10.1145/3373376.3378496>

- [9] S. Tollman, S. J. Park, and J. K. Ousterhout, “EPaxos Revisited,” in *18th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2021, April 12-14, 2021*, J. Mickens and R. Teixeira, Eds., USENIX Association, 2021, pp. 613–632. [Online]. Available: <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [10] H. Attiya and J. L. Welch, “Sequential Consistency versus Linearizability,” *ACM Trans. Comput. Syst.*, vol. 12, no. 2, pp. 91–122, 1994. DOI: 10.1145/176575.176576 [Online]. Available: <https://doi.org/10.1145/176575.176576>
- [11] S. B. Davidson, H. Garcia-Molina, and D. Skeen, “Consistency in a Partitioned Network: A Survey,” *ACM Comput. Surv.*, vol. 17, no. 3, pp. 341–370, Sep. 1985, ISSN: 0360-0300. DOI: 10.1145/5505.5508 [Online]. Available: <https://doi.org/10.1145/5505.5508>
- [12] P. Viotti and M. Vukolic, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Comput. Surv.*, vol. 49, no. 1, 19:1–19:34, 2016. DOI: 10.1145/2926965 [Online]. Available: <https://doi.org/10.1145/2926965>
- [13] W. Vogels, “Eventually Consistent: Building Reliable Distributed Systems at a Worldwide Scale Demands Trade-Offs Between Consistency and Availability,” *Queue*, vol. 6, no. 6, pp. 14–19, Oct. 2008, ISSN: 1542-7730. DOI: 10.1145/1466443.1466448 [Online]. Available: <https://doi.org/10.1145/1466443.1466448>
- [14] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen, “Don’t settle for eventual: scalable causal consistency for wide-area storage with COPS,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles 2011, SOSP 2011, Cascais, Portugal, October 23-26, 2011*, T. Wobber and P. Druschel,

- Eds., ACM, 2011, pp. 401–416. DOI: 10.1145/2043556.2043593 [Online]. Available: <https://doi.org/10.1145/2043556.2043593>
- [15] S. Pelley, P. M. Chen, and T. F. Wenisch, “Memory Persistency,” in *ACM/IEEE 41st International Symposium on Computer Architecture, ISCA 2014, Minneapolis, MN, USA, June 14-18, 2014*, IEEE Computer Society, 2014, pp. 265–276. DOI: 10.1109/ISCA.2014.6853222 [Online]. Available: <https://doi.org/10.1109/ISCA.2014.6853222>
- [16] A. Kokolis, A. Psistakis, B. Reidys, J. Huang, and J. Torrellas, “Distributed Data Persistency,” in *MICRO ’21: 54th Annual IEEE/ACM International Symposium on Microarchitecture, Virtual Event, Greece, October 18-22, 2021*, ACM, 2021, pp. 71–85. DOI: 10.1145/3466752.3480060 [Online]. Available: <https://doi.org/10.1145/3466752.3480060>
- [17] A. Ganesan, R. Alagappan, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, “Strong and Efficient Consistency with Consistency-Aware Durability,” in *18th USENIX Conference on File and Storage Technologies (FAST 20)*, Santa Clara, CA: USENIX Association, Feb. 2020, pp. 323–337, ISBN: 978-1-939133-12-0. [Online]. Available: <https://www.usenix.org/conference/fast20/presentation/ganesan>
- [18] C. Lin, V. Nagarajan, and R. Gupta, “Fence Scoping,” in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2014, New Orleans, LA, USA, November 16-21, 2014*, T. Damkroger and J. J. Dongarra, Eds., IEEE Computer Society, 2014, pp. 105–116. DOI: 10.1109/SC.2014.14 [Online]. Available: <https://doi.org/10.1109/SC.2014.14>
- [19] CXL Consortium, *Compute Express Link (CXL) 3.2 Specification*, https://computeexpresslink.org/wp-content/uploads/2024/11/CXL-Specification_rev3p2_ver1p0_2024October2_evalcopy.pdf, Accessed: 2025-11-05, 2024.

- [20] A. Psistakis, F. Chaix, and J. Torrellas, “MINOS: Distributed Consistency and Persistency Protocol Implementation & Offloading to SmartNICs,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024, Edinburgh, United Kingdom, March 2-6, 2024*, IEEE, 2024, pp. 1–17. DOI: 10.1109/HPCA57654.2024.00076 [Online]. Available: <https://doi.org/10.1109/HPCA57654.2024.00076>
- [21] J. Izraelevitz, H. Mendes, and M. L. Scott, “Linearizability of Persistent Memory Objects Under a Full-System-Crash Failure Model,” in *30th International Symposium on Distributed Computing, DISC 2016, Paris, France, September 27-29, 2016. Proceedings*, C. Gavoille and D. Ilcinkas, Eds., ser. Lecture Notes in Computer Science, vol. 9888, Springer, 2016, pp. 313–327. DOI: 10.1007/978-3-662-53426-7_23 [Online]. Available: https://doi.org/10.1007/978-3-662-53426-7_23
- [22] T. Talpey, *RDMA Persistent Memory Extensions*, 2019. [Online]. Available: https://www.openfabrics.org/wp-content/uploads/209_TTalpey.pdf
- [23] M. Tork, L. Maudlej, and M. Silberstein, “Lynx: A SmartNIC-driven Accelerator-centric Architecture for Network Servers,” in *ASPLOS ’20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, J. R. Larus, L. Ceze, and K. Strauss, Eds., ACM, 2020, pp. 117–131. DOI: 10.1145/3373376.3378528 [Online]. Available: <https://doi.org/10.1145/3373376.3378528>
- [24] P. M. Phothilimthana, M. Liu, A. Kaufmann, S. Peter, R. Bodik, and T. Anderson, “Floem: A Programming System for NIC-Accelerated Network Applications,” in *13th USENIX Symposium on Operating Systems*, ser. OSDI’18, Carlsbad, CA, USA: USENIX Association, 2018, pp. 663–679, ISBN: 9781931971478.

- [25] M. Liu, T. Cui, H. Schuh, A. Krishnamurthy, S. Peter, and K. Gupta, “Offloading Distributed Applications onto SmartNICs using iPipe,” in *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM 2019, Beijing, China, August 19-23, 2019*, J. Wu and W. Hall, Eds., ACM, 2019, pp. 318–333. DOI: 10.1145/3341302.3342079 [Online]. Available: <https://doi.org/10.1145/3341302.3342079>
- [26] P. Hunt, M. Konar, F. P. Junqueira, and B. C. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems,” in *2010 USENIX Annual Technical Conference, Boston, MA, USA, June 23-25, 2010*, P. Barham and T. Roscoe, Eds., USENIX Association, 2010. [Online]. Available: <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [27] D. D. Sharma, “Compute express link (CXL): enabling heterogeneous data-centric computing with heterogeneous memory hierarchy,” *IEEE Micro*, vol. 43, no. 2, pp. 99–109, 2023. DOI: 10.1109/MM.2022.3228561 [Online]. Available: <https://doi.org/10.1109/MM.2022.3228561>
- [28] D. D. Sharma, R. Blankenship, and D. S. Berger, “An introduction to the compute express link (CXL) interconnect,” *ACM Comput. Surv.*, vol. 56, no. 11, 290:1–290:37, 2024. DOI: 10.1145/3669900 [Online]. Available: <https://doi.org/10.1145/3669900>
- [29] A. Patil, V. Nagarajan, N. Nikoleris, and N. Oswald, “Āpta: Fault-tolerant object-granular CXL disaggregated memory for accelerating FaaS,” in *53rd Annual IEEE/IFIP International Conference on Dependable Systems and Network, DSN 2023, Porto, Portugal, June 27-30, 2023*, IEEE, 2023, pp. 201–215. DOI: 10.1109/DSN58367.2023.00030 [Online]. Available: <https://doi.org/10.1109/DSN58367.2023.00030>

- [30] X. Yang, Y. Zhang, H. Chen, F. Li, G. Fan, Y. Kong, B. Wang, J. Fang, Y. Wang, T. Huang, W. Hu, J. Kao, and J. Jiang, “Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases,” in *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, V. Markl, J. M. Hellerstein, and A. Abouzied, Eds., ACM, 2025, pp. 689–702. DOI: 10.1145/3722212.3724460 [Online]. Available: <https://doi.org/10.1145/3722212.3724460>
- [31] M. Zhang, T. Ma, J. Hua, Z. Liu, K. Chen, N. Ding, F. Du, J. Jiang, T. Ma, and Y. Wu, “Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory,” in *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds., ACM, 2023, pp. 658–674. DOI: 10.1145/3600006.3613135 [Online]. Available: <https://doi.org/10.1145/3600006.3613135>
- [32] S. Gurumurthi, A. J. Branover, B. Hornung, V. Michna, M. Natsu, and C. Petersen, “An Overview of Reliability, Availability, and Serviceability (RAS) in Compute Express Link™ 2.0,” CXL Consortium / Compute Express Link™, White Paper, Jan. 2021, Accessed: 2025-11-08. [Online]. Available: https://computeexpresslink.org/wp-content/uploads/2023/12/CXL-RAS-Whitepaper-Post-WG-Revision_FINAL.pdf
- [33] A. Hasbun and D. Walton, “OCP RAS API Revision 0.8,” Open Compute Project, Tech. Rep., Feb. 2023, Accessed: 2025-11-08. [Online]. Available: <https://www.opencompute.org/documents/2022-open-compute-specification-ras-api-v0-8-docx-pdf>
- [34] G. Yu, G. Tan, H. Huang, Z. Zhang, P. Chen, R. Natella, and Z. Zheng, “A Survey on Failure Analysis and Fault Injection in AI Systems,” *CoRR*, vol. abs/2407.00125,

2024. DOI: 10.48550/ARXIV.2407.00125 arXiv: 2407.00125. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.00125>
- [35] Y. Xu, S. Mahar, Z. Liu, M. Shen, and S. Swanson, “Position: CXL Shared Memory Programming: Barely Distributed and Almost Persistent,” *CoRR*, vol. abs/2405.19626, 2024. DOI: 10.48550/ARXIV.2405.19626 arXiv: 2405.19626. [Online]. Available: <https://doi.org/10.48550/arXiv.2405.19626>
- [36] P. Bailis, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Bolt-on causal consistency,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, K. A. Ross, D. Srivastava, and D. Papadias, Eds., ACM, 2013, pp. 761–772. DOI: 10.1145/2463676.2465279 [Online]. Available: <https://doi.org/10.1145/2463676.2465279>
- [37] S. A. Mehdi, C. Littley, N. Crooks, L. Alvisi, N. Bronson, and W. Lloyd, “I Can’t Believe It’s Not Causal! Scalable Causal Consistency with No Slowdown Cascades,” in *14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2017, Boston, MA, USA, March 27-29, 2017*, A. Akella and J. Howell, Eds., USENIX Association, 2017, pp. 453–468. [Online]. Available: <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/mehdi>
- [38] X. Wei, J. Shi, Y. Chen, R. Chen, and H. Chen, “Fast In-Memory Transaction Processing Using RDMA and HTM,” in *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, E. L. Miller and S. Hand, Eds., ACM, 2015, pp. 87–104. DOI: 10.1145/2815400.2815419 [Online]. Available: <https://doi.org/10.1145/2815400.2815419>

- [39] A. M. Caulfield, E. S. Chung, A. Putnam, H. Angepat, J. Fowers, M. Haselman, S. Heil, M. Humphrey, P. Kaur, J. Kim, D. Lo, T. Massengill, K. Ovtcharov, M. Papamichael, L. Woods, S. Lanka, D. Chiou, and D. Burger, “A Cloud-Scale Acceleration Architecture,” in *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*, IEEE Computer Society, 2016, 7:1–7:13. DOI: 10.1109/MICRO.2016.7783710 [Online]. Available: <https://doi.org/10.1109/MICRO.2016.7783710>
- [40] W. Reda, M. Canini, D. Kostic, and S. Peter, “RDMA is Turing complete, we just did not know it yet!” In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, A. Phanishayee and V. Sekar, Eds., USENIX Association, 2022, pp. 71–85. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/reda>
- [41] M. Burke, S. Dharanipragada, S. Joyner, A. Szekeres, J. Nelson, I. Zhang, and D. R. K. Ports, “PRISM: Rethinking the RDMA Interface for Distributed Systems,” in *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, R. van Renesse and N. Zeldovich, Eds., ACM, 2021, pp. 228–242. DOI: 10.1145/3477132.3483587 [Online]. Available: <https://doi.org/10.1145/3477132.3483587>
- [42] D. Kim, A. S. Memaripour, A. Badam, Y. Zhu, H. H. Liu, J. Padhye, S. Raindel, S. Swanson, V. Sekar, and S. Seshan, “Hyperloop: Group-Based NIC-Offloading to Accelerate Replicated Transactions in Multi-Tenant Storage Systems,” in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM 2018, Budapest, Hungary, August 20-25, 2018*, S. Gorinsky and J. Tapolcai, Eds., ACM, 2018, pp. 297–312. DOI: 10.1145/3230543.3230572 [Online]. Available: <https://doi.org/10.1145/3230543.3230572>

- [43] Microsoft, *Microsoft Catapult: Transforming Cloud Computing by Augmenting CPUs with an Interconnected and Configurable Compute Layer Composed of Programmable Silicon*,
<https://www.microsoft.com/en-us/research/project/project-catapult/>, 2018.
- [44] A. Putnam, A. M. Caulfield, E. S. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmaeilzadeh, J. Fowers, G. P. Gopal, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J. Kim, S. Lanka, J. R. Larus, E. Peterson, S. Pope, A. Smith, J. Thong, P. Y. Xiao, and D. Burger, “A reconfigurable fabric for accelerating large-scale datacenter services,” in *ACM/IEEE 41st International Symposium on Computer Architecture, ISCA 2014, Minneapolis, MN, USA, June 14-18, 2014*, IEEE Computer Society, 2014, pp. 13–24. DOI: 10.1109/ISCA.2014.6853195 [Online]. Available: <https://doi.org/10.1109/ISCA.2014.6853195>
- [45] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978. DOI: 10.1145/359545.359563 [Online]. Available: <https://doi.org/10.1145/359545.359563>
- [46] A. Kalia, M. Kaminsky, and D. G. Andersen, “Datacenter RPCs can be General and Fast,” in *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, J. R. Lorch and M. Yu, Eds., USENIX Association, 2019, pp. 1–16. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/kalia>
- [47] A. Ryser, A. Lerner, A. Forencich, and P. Cudré-Mauroux, “D-RDMA: Bringing Zero-Copy RDMA to Database Systems,” in *12th Conference on Innovative Data Systems Research, CIDR 2022, Chaminade, CA, USA, January 9-12, 2022*, www.cidrdb.org, 2022. [Online]. Available: <https://www.cidrdb.org/cidr2022/papers/p77-ryser.pdf>

- [48] Mellanox, *NVIDIA® Mellanox® BlueField® Data Processing Unit (DPU)*, <https://network.nvidia.com/files/doc-2020/pb-bluefield-dpu.pdf>, 2020.
- [49] A. Kalia, M. Kaminsky, and D. G. Andersen, “Design Guidelines for High Performance RDMA Systems,” in *2016 USENIX Annual Technical Conference, USENIX ATC 2016, Denver, CO, USA, June 22-24, 2016*, A. Gulati and H. Weatherspoon, Eds., USENIX Association, 2016, pp. 437–450. [Online]. Available: <https://www.usenix.org/conference/atc16/technical-sessions/presentation/kalia>
- [50] H. N. Schuh, W. Liang, M. Liu, J. Nelson, and A. Krishnamurthy, “Xenic: SmartNIC-Accelerated Distributed Transactions,” in *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, R. van Renesse and N. Zeldovich, Eds., ACM, 2021, pp. 740–755. DOI: 10.1145/3477132.3483555 [Online]. Available: <https://doi.org/10.1145/3477132.3483555>
- [51] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. USA: Addison-Wesley Longman Publishing Co., Inc., 2002, ISBN: 032114306X.
- [52] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, “The Design and Operation of CloudLab,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Jul. 2019, pp. 1–14. [Online]. Available: <https://www.flux.utah.edu/paper/duplyakin-atc19>
- [53] A. van Renen, L. Vogel, V. Leis, T. Neumann, and A. Kemper, “Building Blocks for Persistent Memory,” *VLDB J.*, vol. 29, no. 6, pp. 1223–1241, 2020. DOI:

- 10.1007/s00778-020-00622-9 [Online]. Available:
<https://doi.org/10.1007/s00778-020-00622-9>
- [54] Z. Wang, X. Liu, J. Yang, T. Michailidis, S. Swanson, and J. Zhao, “Characterizing and Modeling Non-Volatile Memory Systems,” in *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*, IEEE, 2020, pp. 496–508. DOI:
 10.1109/MICRO50266.2020.00049 [Online]. Available:
<https://doi.org/10.1109/MICRO50266.2020.00049>
- [55] C. Guo, H. Wu, Z. Deng, G. Soni, J. Ye, J. Padhye, and M. Lipshteyn, “RDMA over Commodity Ethernet at Scale,” in *Proceedings of the ACM SIGCOMM 2016 Conference, Florianopolis, Brazil, August 22-26, 2016*, M. P. Barcellos, J. Crowcroft, A. Vahdat, and S. Katti, Eds., ACM, 2016, pp. 202–215. DOI:
 10.1145/2934872.2934908 [Online]. Available:
<https://doi.org/10.1145/2934872.2934908>
- [56] S. Tsai and Y. Zhang, “LITE Kernel RDMA Support for Datacenter Applications,” in *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, ACM, 2017, pp. 306–324. DOI:
 10.1145/3132747.3132762 [Online]. Available:
<https://doi.org/10.1145/3132747.3132762>
- [57] R. Zambre, M. Grodowitz, A. Chandramowliswaran, and P. Shamis, “Breaking Band: A Breakdown of High-performance Communication,” *CoRR*, vol. abs/2002.02563, 2020. arXiv: 2002.02563. [Online]. Available:
<https://arxiv.org/abs/2002.02563>
- [58] Y. Ajima, T. Kawashima, T. Okamoto, N. Shida, K. Hirai, T. Shimizu, S. Hiramoto, Y. Ikeda, T. Yoshikawa, K. Uchida, and T. Inoue, “The Tofu

- Interconnect D,” in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, 2018, pp. 646–654. DOI: 10.1109/CLUSTER.2018.00090
- [59] A. Kalia, M. Kaminsky, and D. G. Andersen, *eRPC on GitHub*, <https://github.com/erpc-io/eRPC>, 2019.
- [60] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, “Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms,” *J. Parallel Distributed Comput.*, vol. 74, no. 10, pp. 2899–2917, 2014. DOI: 10.1016/j.jpdc.2014.06.008 [Online]. Available: <https://doi.org/10.1016/j.jpdc.2014.06.008>
- [61] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, *SimGrid on GitHub*. <https://github.com/simgrid/simgrid>, 2014.
- [62] NVIDIA, *NVIDIA BlueField 2 DPU*, <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-2-dpu.pdf>, 2021.
- [63] HPC-AI Advisory Council Cluster Center, *HPC-AI Advisory Council Cluster Center*. [Online]. Available: <https://www.hpcadvisorycouncil.com/>
- [64] R. Neugebauer, G. Antichi, J. F. Zazo, Y. Audzevich, S. López-Buedo, and A. W. Moore, “Understanding PCIe Performance for End Host Networking,” in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM 2018, Budapest, Hungary, August 20-25, 2018*, S. Gorinsky and J. Tapolcai, Eds., ACM, 2018, pp. 327–341. DOI: 10.1145/3230543.3230560 [Online]. Available: <https://doi.org/10.1145/3230543.3230560>
- [65] Mellanox, *ConnectX®-3 VPI Single and Dual QSFP+ Port Adapter Card User Manual*, https://network.nvidia.com/pdf/user_manuals/ConnectX-3_VPI_Single_and_Dual_QSFP_Port_Adapter_Card_User_Manual.pdf, 2013.

- [66] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears,
 “Benchmarking Cloud Serving Systems with YCSB,” in *Proceedings of the 1st ACM Symposium on Cloud Computing*, ser. SoCC ’10, Indianapolis, Indiana, USA: Association for Computing Machinery, 2010, pp. 143–154, ISBN: 9781450300360.
 DOI: 10.1145/1807128.1807152 [Online]. Available:
<https://doi.org/10.1145/1807128.1807152>
- [67] J. Ren, *Yahoo! Cloud Serving Benchmark in C++, a C++ version of YCSB*,
<https://github.com/basicthinker/YCSB-C>, 2014.
- [68] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, I. Bahar, M. Herlihy, E. Witchel, and A. R. Lebeck, Eds., ACM, 2019, pp. 3–18. DOI: 10.1145/3297858.3304013 [Online]. Available:
<https://doi.org/10.1145/3297858.3304013>
- [69] E. Amaro, Z. Luo, A. Ousterhout, A. Krishnamurthy, A. Panda, S. Ratnasamy, and S. Shenker, “Remote Memory Calls,” in *HotNets ’20: The 19th ACM Workshop on Hot Topics in Networks, Virtual Event, USA, November 4-6, 2020*, B. Y. Zhao, H. Zheng, H. V. Madhyastha, and V. N. Padmanabhan, Eds., ACM, 2020, pp. 38–44. DOI: 10.1145/3422604.3425923 [Online]. Available:
<https://doi.org/10.1145/3422604.3425923>
- [70] K. Wu, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Towards an Unwritten Contract of Intel Optane SSD,” in *11th USENIX Workshop on Hot Topics in*

- Storage and File Systems, HotStorage 2019, Renton, WA, USA, July 8-9, 2019*, D. Peek and G. Yadgar, Eds., USENIX Association, 2019. [Online]. Available: <https://www.usenix.org/conference/hotstorage19/presentation/wu-kan>
- [71] Y. Yuan, J. Huang, Y. Sun, T. Wang, J. Nelson, D. R. K. Ports, Y. Wang, R. Wang, C. Tai, and N. S. Kim, “RAMBDA: RDMA-driven Acceleration Framework for Memory-Intensive s-scale Datacenter Applications,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*, IEEE, 2023, pp. 499–515. DOI: 10.1109/HPCA56546.2023.10071127 [Online]. Available: <https://doi.org/10.1109/HPCA56546.2023.10071127>
- [72] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, and I. Stoica, “NetCache: Balancing Key-Value Stores with Fast In-Network Caching,” in *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, ACM, 2017, pp. 121–136. DOI: 10.1145/3132747.3132764 [Online]. Available: <https://doi.org/10.1145/3132747.3132764>
- [73] M. Liu, L. Luo, J. Nelson, L. Ceze, A. Krishnamurthy, and K. Atreya, “IncBricks: Toward In-Network Computation with an In-Network Cache,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi’an, China, April 8-12, 2017*, Y. Chen, O. Temam, and J. Carter, Eds., ACM, 2017, pp. 795–809. DOI: 10.1145/3037697.3037731 [Online]. Available: <https://doi.org/10.1145/3037697.3037731>
- [74] K. Seemakhupt, S. Liu, Y. Senevirathne, M. Shahbaz, and S. M. Khan, “PMNet: In-Network Data Persistence,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18*,

- 2021, IEEE, 2021, pp. 804–817. DOI: 10.1109/ISCA52012.2021.00068 [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00068>
- [75] B. Li, Z. Ruan, W. Xiao, Y. Lu, Y. Xiong, A. Putnam, E. Chen, and L. Zhang, “KV-Direct: High-Performance In-Memory Key-Value Store with Programmable NIC,” in *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, ACM, 2017, pp. 137–152. DOI: 10.1145/3132747.3132756 [Online]. Available: <https://doi.org/10.1145/3132747.3132756>
- [76] Y. Mu, K. Yao, Y. Li, Z. Li, T. Sun, L. Lu, J. He, and M. Huang, “SOSP: A SmartNIC-Based Offloading Framework for Cloud Storage Pooling,” in *2022 9th International Conference on Wireless Communication and Sensor Networks (ICWCSN)*, ser. icWCSN 2022, Dalian, China: Association for Computing Machinery, 2022, pp. 25–28, ISBN: 9781450396219. DOI: 10.1145/3514105.3514110 [Online]. Available: <https://doi.org/10.1145/3514105.3514110>
- [77] J. Kim, I. Jang, W. Reda, J. Im, M. Canini, D. Kostic, Y. Kwon, S. Peter, and E. Witchel, “LineFS: Efficient SmartNIC Offload of a Distributed File System with Pipeline Parallelism,” in *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, R. van Renesse and N. Zeldovich, Eds., ACM, 2021, pp. 756–771. DOI: 10.1145/3477132.3483565 [Online]. Available: <https://doi.org/10.1145/3477132.3483565>
- [78] H. Seyedroudbari, S. Vanavasam, and A. Daglis, “TURBO: SmartNIC-enabled Dynamic Load Balancing of s-scale RPCs,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*, IEEE, 2023, pp. 1045–1058. DOI:

- 10.1109/HPCA56546.2023.10071135 [Online]. Available:
<https://doi.org/10.1109/HPCA56546.2023.10071135>
- [79] S. Sun, R. Zhang, M. Yan, and J. Wu, “SKV: A SmartNIC-Offloaded Distributed Key-Value Store,” in *IEEE International Conference on Cluster Computing, CLUSTER 2022, Heidelberg, Germany, September 5-8, 2022*, IEEE, 2022, pp. 1–11. DOI: 10.1109/CLUSTER51413.2022.00016 [Online]. Available:
<https://doi.org/10.1109/CLUSTER51413.2022.00016>
- [80] B. Kemme, A. Bartoli, and O. Babaoglu, “Online reconfiguration in replicated databases based on group communication,” in *2001 International Conference on Dependable Systems and Networks*, IEEE, 2001, pp. 117–126.
- [81] R. Jiménez-Peris, M. Patiño-Martínez, and G. Alonso, “Non-intrusive, parallel recovery of replicated data,” in *21st IEEE Symposium on Reliable Distributed Systems, 2002. Proceedings.*, IEEE, 2002, pp. 150–159.
- [82] O. M. Mendizabal, F. L. Dotti, and F. Pedone, “High performance recovery for parallel state machine replication,” in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, IEEE, 2017, pp. 34–44.
- [83] C. D. Martino, Z. T. Kalbarczyk, R. K. Iyer, F. Baccanico, J. Fullop, and W. Kramer, “Lessons Learned from the Analysis of System Failures at Petascale: The Case of Blue Waters,” in *44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2014, Atlanta, GA, USA, June 23-26, 2014*, IEEE Computer Society, 2014, pp. 610–621. DOI: 10.1109/DSN.2014.62 [Online]. Available: <https://doi.org/10.1109/DSN.2014.62>
- [84] D. Tiwari, S. Gupta, J. H. Rogers, D. Maxwell, P. Rech, S. S. Vazhkudai, D. Oliveira, D. Londo, N. DeBardeleben, P. O. A. Navaux, L. Carro, and A. S. Bland, “Understanding GPU errors on large-scale HPC systems and the implications for system design and operation,” in *21st IEEE International*

- Symposium on High Performance Computer Architecture, HPCA 2015, Burlingame, CA, USA, February 7-11, 2015*, IEEE Computer Society, 2015, pp. 331–342. DOI: 10.1109/HPCA.2015.7056044 [Online]. Available: <https://doi.org/10.1109/HPCA.2015.7056044>
- [85] F. Cappelletto, A. Geist, W. Gropp, S. Kale, B. Kramer, and M. Snir, “Toward Exascale Resilience: 2014 update.” *Supercomput. Front. Innov.*, vol. 1, no. 1, pp. 5–28, 2014. DOI: 10.14529/JSFI140101 [Online]. Available: <https://doi.org/10.14529/jsfi140101>
- [86] B. Liu, C. Scott, M. Tariq, A. D. Ferguson, P. Gill, R. Alimi, O. Alipourfard, D. Arulkannan, V. Beauregard, P. Conner, P. B. Godfrey, X. Lin, J. Ong, M. Patel, A. Sabaa, A. Singh, A. Smirnov, M. Verma, P. V. Viswanadham, and A. Vahdat, “CAPA: An Architecture For Operating Cluster Networks With High Availability,” in *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, L. Vanbever and I. Zhang, Eds., USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/liu-bingzhe>
- [87] W. Zheng, S. Tu, E. Kohler, and B. Liskov, “Fast Databases with Fast Durability and Recovery Through Multicore Parallelism,” in *11th USENIX Symposium on Operating Systems Design and Implementation, OSDI '14, Broomfield, CO, USA, October 6-8, 2014*, J. Flinn and H. Levy, Eds., USENIX Association, 2014, pp. 465–477. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng%5C_wenting
- [88] C. Yao, D. Agrawal, G. Chen, B. C. Ooi, and S. Wu, “Adaptive Logging: Optimizing Logging and Recovery Costs in Distributed In-memory Databases,” in *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference*

- 2016, San Francisco, CA, USA, June 26 - July 01, 2016, F. Özcan, G. Koutrika, and S. Madden, Eds., ACM, 2016, pp. 1119–1134. DOI: 10.1145/2882903.2915208 [Online]. Available: <https://doi.org/10.1145/2882903.2915208>
- [89] Y. Wu, W. Guo, C. Chan, and K. Tan, “Fast Failure Recovery for Main-Memory DBMSs on Multicores,” in *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, S. Salihoglu, W. Zhou, R. Chirkova, J. Yang, and D. Suciu, Eds., ACM, 2017, pp. 267–281. DOI: 10.1145/3035918.3064011 [Online]. Available: <https://doi.org/10.1145/3035918.3064011>
- [90] A. Ganesan, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Exploiting Nil-Externality for Fast Replicated Storage,” in *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, R. van Renesse and N. Zeldovich, Eds., ACM, 2021, pp. 440–456. DOI: 10.1145/3477132.3483543 [Online]. Available: <https://doi.org/10.1145/3477132.3483543>
- [91] J. C. W. Chan, Q. Ding, P. P. C. Lee, and H. H. W. Chan, “Parity logging with reserved space: towards efficient updates and recovery in erasure-coded clustered storage,” in *Proceedings of the 12th USENIX conference on File and Storage Technologies, FAST 2014, Santa Clara, CA, USA, February 17-20, 2014*, B. Schroeder and E. Thereska, Eds., USENIX, 2014, pp. 163–176. [Online]. Available: <https://www.usenix.org/conference/fast14/technical-sessions/presentation/chan>
- [92] A. Cheng, X. Shi, A. N. Kabcenell, S. Lawande, H. Qadeer, J. Chan, H. Tin, R. Zhao, P. Bailis, M. Balakrishnan, N. Bronson, N. Crooks, and I. Stoica, “TAOBench: An End-to-End Benchmark for Social Networking Workloads,” *Proc. VLDB Endow.*, vol. 15, no. 9, pp. 1965–1977, 2022. DOI:

- 10.14778/3538598.3538616 [Online]. Available:
<https://www.vldb.org/pvldb/vol15/p1965-cheng.pdf>
- [93] P. Desnoyers, I. F. Adams, T. Estro, A. Gandhi, G. Kuenning, M. P. Mesnier, C. A. Waldspurger, A. Wildani, and E. Zadok, “Persistent Memory Research in the Post-Optane Era,” in *Proceedings of the 1st Workshop on Disruptive Memory Systems, DIMES 2023, Koblenz, Germany, 23 October 2023*, ACM, 2023, pp. 23–30. DOI: 10.1145/3609308.3625268 [Online]. Available:
<https://doi.org/10.1145/3609308.3625268>
- [94] A. Sainio, J. Handy, and T. Coughlin, *Emerging Memories: Branch Out*, Webinar presentation (PDF), Live: January 22, 2024; On-demand at SNIA, Jan. 2024. [Online]. Available: https://www.snia.org/sites/default/files/event-calendar/2024-0122_SNIA_CMSI_Emerging_Memories_Webinar.pdf
- [95] N. Ramaswamy, A. Calderoni, J. Zahurak, G. Servalli, A. Chavan, S. Chhajed, M. Balakrishnan, M. Fischer, M. Hollander, D. P. Ettisserry, A. Liao, K. Karda, M. Jerry, M. Mariani, A. Visconti, B. R. Cook, B. D. Cook, D. Mills, A. Torsi, C. Mouli, E. Byers, M. Helm, S. Pawlowski, S. Shiratake, and N. Chandrasekaran, “NVDRAM: A 32Gb Dual Layer 3D Stacked Non-volatile Ferroelectric Memory with Near-DRAM Performance for Demanding AI Workloads,” in *2023 International Electron Devices Meeting (IEDM)*, 2023, pp. 1–4. DOI: 10.1109/IEDM45741.2023.10413848
- [96] E. Brewer, *Spanner, truetime & the cap theorem*, Accessed: 2024-07-17, 2017. [Online]. Available: <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45855.pdf>
- [97] M. Burrows, “The chubby lock service for loosely-coupled distributed systems,” in *7th Symposium on Operating Systems Design and Implementation (OSDI '06), November 6-8, Seattle, WA, USA*, B. N. Bershad and J. C. Mogul, Eds., USENIX

- Association, 2006, pp. 335–350. [Online]. Available:
<http://www.usenix.org/events/osdi06/tech/burrows.html>
- [98] H. Liu, S. Lu, M. Musuvathi, and S. Nath, “What bugs cause production cloud incidents?” In *Proceedings of the Workshop on Hot Topics in Operating Systems, HotOS 2019, Bertinoro, Italy, May 13-15, 2019*, ACM, 2019, pp. 155–162. DOI: 10.1145/3317550.3321438 [Online]. Available:
<https://doi.org/10.1145/3317550.3321438>
- [99] M. Ma, Y. Liu, Y. Tong, H. Li, P. Zhao, Y. Xu, H. Zhang, S. He, L. Wang, Y. Dang, S. Rajmohan, and Q. Lin, “An empirical investigation of missing data handling in cloud node failure prediction,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, A. Roychoudhury, C. Cadar, and M. Kim, Eds., ACM, 2022, pp. 1453–1464. DOI: 10.1145/3540250.3558946 [Online]. Available:
<https://doi.org/10.1145/3540250.3558946>
- [100] X. Li, G. Yu, P. Chen, H. Chen, and Z. Chen, “Going through the life cycle of faults in clouds: Guidelines on fault handling,” in *IEEE 33rd International Symposium on Software Reliability Engineering, ISSRE 2022, Charlotte, NC, USA, October 31 - Nov. 3, 2022*, IEEE, 2022, pp. 121–132. DOI: 10.1109/ISSRE55969.2022.00022 [Online]. Available: <https://doi.org/10.1109/ISSRE55969.2022.00022>
- [101] B. Schroeder and G. A. Gibson, “A large-scale study of failures in high-performance computing systems,” *IEEE Trans. Dependable Secur. Comput.*, vol. 7, no. 4, pp. 337–351, 2010. DOI: 10.1109/TDSC.2009.4 [Online]. Available:
<https://doi.org/10.1109/TDSC.2009.4>
- [102] Q. Hu, Z. Ye, Z. Wang, G. Wang, M. Zhang, Q. Chen, P. Sun, D. Lin, X. Wang, Y. Luo, Y. Wen, and T. Zhang, “Characterization of large language model

- development in the datacenter,” in *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, L. Vanbever and I. Zhang, Eds., USENIX Association, 2024, pp. 709–729. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/hu>
- [103] H. S. Gunawi, R. O. Suminto, R. Sears, C. Golliher, S. Sundararaman, X. Lin, T. Emami, W. Sheng, N. Bidokhti, C. McCaffrey, D. Srinivasan, B. Panda, A. Baptist, G. Grider, P. M. Fields, K. Harms, R. B. Ross, A. Jacobson, R. Ricci, K. Webb, P. Alvaro, H. B. Runesha, M. Hao, and H. Li, “Fail-slow at scale: Evidence of hardware performance faults in large production systems,” *ACM Trans. Storage*, vol. 14, no. 3, 23:1–23:26, 2018. DOI: 10.1145/3242086 [Online]. Available: <https://doi.org/10.1145/3242086>
- [104] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. S. E. Ng, and Y. Wang, “GEMINI: fast failure recovery in distributed training with in-memory checkpoints,” in *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds., ACM, 2023, pp. 364–381. DOI: 10.1145/3600006.3613145 [Online]. Available: <https://doi.org/10.1145/3600006.3613145>
- [105] R. Kateja, A. Badam, S. Govindan, B. Sharma, and G. Ganger, “Viyojit: Decoupling battery and DRAM capacities for battery-backed DRAM,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*, ACM, 2017, pp. 613–626. [Online]. Available: <https://dl.acm.org/citation.cfm?id=3080236>
- [106] M. A. Alshboul, P. Ramrakhiani, W. Wang, J. Tuck, and Y. Solihin, “BBB: simplifying persistent programming using battery-backed buffers,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA*

- 2021, Seoul, South Korea, February 27 - March 3, 2021, IEEE, 2021, pp. 111–124.
DOI: 10.1109/HPCA51647.2021.00019 [Online]. Available:
<https://doi.org/10.1109/HPCA51647.2021.00019>
- [107] T. Zhou, Y. Du, F. Yang, X. Liao, and Y. Lu, “Efficient atomic durability on eadr-enabled persistent memory,” in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques, PACT 2022, Chicago, Illinois, October 8-12, 2022*, A. Klöckner and J. Moreira, Eds., ACM, 2022, pp. 124–134.
DOI: 10.1145/3559009.3569676 [Online]. Available:
<https://doi.org/10.1145/3559009.3569676>
- [108] Premio Inc., *What is a hot swap drive? how does it work*, Accessed: 2024-07-17, 2022. [Online]. Available: <https://premioinc.com/blogs/blog/what-is-a-hot-swap-drive-how-does-it-work>
- [109] S. McAllister, F. Kazhamiaka, D. S. Berger, R. Fonseca, K. Frost, A. Ogus, M. Sah, R. Bianchini, G. Amvrosiadis, N. Beckmann, and G. R. Ganger, “A call for research on storage emissions,” in *Proceedings of HotCarbon 2024*, 2024. [Online]. Available: <https://hotcarbon.org/assets/2024/pdf/hotcarbon24-final126.pdf>
- [110] M. Kalyanakrishnam, Z. Kalbarczyk, and R. K. Iyer, “Failure Data Analysis of a LAN of Windows NT based Computers,” in *The Eighteenth Symposium on Reliable Distributed Systems, SRDS 1999, Lausanne, Switzerland, October 19-22, 1999, Proceedings*, IEEE Computer Society, 1999, pp. 178–187. DOI: 10.1109/RELDIS.1999.805094 [Online]. Available: <https://doi.org/10.1109/RELDIS.1999.805094>
- [111] L. A. Barroso, U. Hölzle, and P. Ranganathan, *The Datacenter as a Computer: Designing Warehouse-Scale Machines, Third Edition* (Synthesis Lectures on Computer Architecture). Morgan & Claypool Publishers, 2018, ISBN:

- 978-3-031-00633-3. DOI: 10.2200/S00874ED3V01Y201809CAC046 [Online]. Available: <https://doi.org/10.2200/S00874ED3V01Y201809CAC046>
- [112] P. Gill, N. Jain, and N. Nagappan, “Understanding network failures in data centers: measurement, analysis, and implications,” in *Proceedings of the ACM SIGCOMM 2011 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, Toronto, ON, Canada, August 15-19, 2011*, S. Keshav, J. Liebeherr, J. W. Byers, and J. C. Mogul, Eds., ACM, 2011, pp. 350–361. DOI: 10.1145/2018436.2018477 [Online]. Available: <https://doi.org/10.1145/2018436.2018477>
- [113] M. K. Aguilera, N. Ben-David, R. Guerraoui, V. J. Marathe, A. Xygkis, and I. Zablatchi, “Microsecond consensus for microsecond applications,” in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, USENIX Association, 2020, pp. 599–616. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/aguilera>
- [114] J. Terrace and M. J. Freedman, “Object Storage on CRAQ: High-Throughput Chain Replication for Read-Mostly Workloads,” in *Proceedings of the 2009 USENIX Annual Technical Conference, USENIX ATC 2009, San Diego, CA, USA, June 14-19, 2009*, G. M. Voelker and A. Wolman, Eds., USENIX Association, 2009.
- [115] R. van Renesse and F. B. Schneider, “Chain Replication for Supporting High Throughput and Availability,” in *6th Symposium on Operating System Design and Implementation (OSDI 2004), San Francisco, California, USA, December 6-8, 2004*, E. A. Brewer and P. Chen, Eds., USENIX Association, 2004, pp. 91–104. [Online]. Available: <http://www.usenix.org/events/osdi04/tech/renesse.html>

- [116] K. Driscoll, B. Hall, H. Sivicrona, and P. Zumsteg, “Byzantine fault tolerance, from theory to reality,” in *International Conference on Computer Safety, Reliability, and Security*, Springer, 2003, pp. 235–248.
- [117] B. Liskov and J. Cowling, “Viewstamped Replication Revisited,” MIT, Tech. Rep. MIT-CSAIL-TR-2012-021, Jul. 2012.
- [118] *MySQL*, <https://mysql.com>, (last accessed on 11/14/2023), Oracle., 2023.
- [119] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “CockroachDB: The Resilient Geo-Distributed SQL Database,” in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, Eds., ACM, 2020, pp. 1493–1509. DOI: 10.1145/3318464.3386134 [Online]. Available: <https://doi.org/10.1145/3318464.3386134>
- [120] *Yugabyte DB*, <https://www.yugabyte.com>, Yugabyte, 2022.
- [121] *Cloud Spanner*, <https://cloud.google.com/spanner>, Google, 2022.
- [122] R. Alagappan, A. Ganesan, J. Liu, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Fault-Tolerance, Fast and Slow: Exploiting Failure Asynchrony in Distributed Systems,” in *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, A. C. Arpaci-Dusseau and G. Voelker, Eds., USENIX Association, 2018, pp. 390–408. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/alagappan>
- [123] G. Hodgkins, Y. Xu, S. Swanson, and J. Izraelevitz, “Zhuque: Failure is Not an Option, it’s an Exception,” in *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, J. Lawall

- and D. Williams, Eds., USENIX Association, 2023, pp. 833–849. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/hodgkins>
- [124] Y. Huang, H. Chen, N. Ni, Y. Sun, V. Chidambaram, D. Tang, and E. Witchel, “Tigon: A Distributed Database for a CXL Pod,” in *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025*, L. Zhou and Y. Zhou, Eds., USENIX Association, 2025, pp. 109–128. [Online]. Available: <https://www.usenix.org/conference/osdi25/presentation/huang-yibo>
- [125] S. Mahar, E. Hajyjasini, S. Lee, Z. Zhang, M. Shen, and S. Swanson, “Telepathic Datacenters: Fast RPCs using Shared CXL Memory,” *CoRR*, vol. abs/2408.11325, 2024. DOI: 10.48550/ARXIV.2408.11325 arXiv: 2408.11325. [Online]. Available: <https://doi.org/10.48550/arXiv.2408.11325>
- [126] S. Wang, B. Hindman, and I. Stoica, “In Reference to RPC: It’s Time to Add Distributed Memory,” in *HotOS ’21: Workshop on Hot Topics in Operating Systems, Ann Arbor, Michigan, USA, June, 1-3, 2021*, S. Angel, B. Kasikci, and E. Kohler, Eds., ACM, 2021, pp. 191–198. DOI: 10.1145/3458336.3465302 [Online]. Available: <https://doi.org/10.1145/3458336.3465302>
- [127] T. Ma, Z. Liu, C. Wei, J. Huang, Y. Zhuo, H. Li, N. Zhang, Y. Guan, D. Niu, M. Zhang, and T. Ma, “HydraRPC: RPC in the CXL Era,” in *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, S. Bagchi and Y. Zhang, Eds., USENIX Association, 2024, pp. 387–395. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/ma>
- [128] P. Sewell, S. Sarkar, S. Owens, F. Z. Nardelli, and M. O. Myreen, “x86-TSO: A Rigorous and Usable Programmer’s Model for x86 Multiprocessors,” *Commun.*

- ACM, vol. 53, no. 7, pp. 89–97, 2010. DOI: 10.1145/1785414.1785443 [Online]. Available: <https://doi.org/10.1145/1785414.1785443>
- [129] D. L. Weaver and T. Germond, *The SPARC Architecture Manual: Version 9*. Prentice Hall, 1994. [Online]. Available: <https://api.semanticscholar.org/CorpusID:58276639>
- [130] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, and H. Li, “Systematic CXL Memory Characterization and Performance Analysis at Scale,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds., ACM, 2025, pp. 1203–1217. DOI: 10.1145/3676641.3715987 [Online]. Available: <https://doi.org/10.1145/3676641.3715987>
- [131] P. Deutsch, “GZIP file format specification version 4.3,” *RFC*, vol. 1952, pp. 1–12, 1996. DOI: 10.17487/RFC1952 [Online]. Available: <https://doi.org/10.17487/RFC1952>
- [132] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini, “Pond: CXL-Based Memory Pooling Systems for Cloud Platforms,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, T. M. Aamodt, N. D. E. Jerger, and M. M. Swift, Eds., ACM, 2023, pp. 574–587. DOI: 10.1145/3575693.3578835 [Online]. Available: <https://doi.org/10.1145/3575693.3578835>
- [133] Micron Corporation, *CXL Memory Expansion: A Closer Look on Actual Platform*, 2024. [Online]. Available: <https://>

- [//www.micron.com/content/dam/micron/global/public/products/white-paper/cxl-memory-expansion-a-close-look-on-actual-platform.pdf](http://www.micron.com/content/dam/micron/global/public/products/white-paper/cxl-memory-expansion-a-close-look-on-actual-platform.pdf)
- [134] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. D. Kersey, R. A. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. CooperBalls, and B. L. Jacob, “The Structural Simulation Toolkit,” *SIGMETRICS Perform. Evaluation Rev.*, vol. 38, no. 4, pp. 37–42, 2011. DOI: 10.1145/1964218.1964225 [Online]. Available: <https://doi.org/10.1145/1964218.1964225>
- [135] Intel, *Pin - A Dynamic Binary Instrumentation Tool*, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-dynamic-binary-instrumentation-tool.html>
- [136] C. Bienia, S. Kumar, J. P. Singh, and K. Li, “The PARSEC Benchmark Suite: Characterization and Architectural Implications,” in *17th International Conference on Parallel Architectures and Compilation Techniques, PACT 2008, Toronto, Ontario, Canada, October 25-29, 2008*, A. Moshovos, D. Tarditi, and K. Olukotun, Eds., ACM, 2008, pp. 72–81. DOI: 10.1145/1454115.1454128 [Online]. Available: <https://doi.org/10.1145/1454115.1454128>
- [137] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, “The SPLASH-2 Programs: Characterization and Methodological Considerations,” in *Proceedings of the 22nd Annual International Symposium on Computer Architecture, ISCA '95, Santa Margherita Ligure, Italy, June 22-24, 1995*, D. A. Patterson, Ed., ACM, 1995, pp. 24–36. DOI: 10.1145/223982.223990 [Online]. Available: <https://doi.org/10.1145/223982.223990>
- [138] C. Bienia, S. Kumar, and K. Li, “PARSEC vs. SPLASH-2: A quantitative comparison of two multithreaded benchmark suites on Chip-Multiprocessors,” in *4th International Symposium on Workload Characterization (IISWC 2008), Seattle, Washington, USA, September 14-16, 2008*, D. Christie, A. Lee, O. Mutlu, and

- B. G. Zorn, Eds., IEEE Computer Society, 2008, pp. 47–56. DOI: 10.1109/IISWC.2008.4636090 [Online]. Available: <https://doi.org/10.1109/IISWC.2008.4636090>
- [139] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop Distributed File System,” in *IEEE 26th Symposium on Mass Storage Systems and Technologies, MSST 2012, Lake Tahoe, Nevada, USA, May 3-7, 2010*, M. G. Khatib, X. He, and M. Factor, Eds., IEEE Computer Society, 2010, pp. 1–10. DOI: 10.1109/MSST.2010.5496972 [Online]. Available: <https://doi.org/10.1109/MSST.2010.5496972>
- [140] XConn Technologies, *XC50256 CXL 2.0 Switch Chip*, Product datasheet, “Fully compliant with CXL 2.0/1.1 and PCIe Gen5, up to 32 ports with bifurcation, 256 lanes, 2048 GB/s switching capacity.”, 2024. [Online]. Available: <https://www.xconn-tech.com/product>
- [141] J. Jang, H. Choi, H. Bae, S. Lee, M. Kwon, and M. Jung, “CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search,” in *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, J. Lawall and D. Williams, Eds., USENIX Association, 2023, pp. 585–600. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/jang>
- [142] Y. Zhong, D. S. Berger, C. A. Waldspurger, R. Wee, I. Agarwal, R. Agarwal, F. Hady, K. Kumar, M. D. Hill, M. Chowdhury, and A. Cidon, “Managing Memory Tiers with CXL in Virtualized Environments,” in *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, A. Gavrilovska and D. B. Terry, Eds., USENIX Association, 2024,

- pp. 37–56. [Online]. Available:
<https://www.usenix.org/conference/osdi24/presentation/zhong-yuhong>
- [143] A. Cho and A. Daglis, “StarNUMA: Mitigating NUMA Challenges with Memory Pooling,” in *57th IEEE/ACM International Symposium on Microarchitecture, MICRO 2024, Austin, TX, USA, November 2-6, 2024*, IEEE, 2024, pp. 997–1012. DOI: 10.1109/MICRO61859.2024.00077 [Online]. Available:
<https://doi.org/10.1109/MICRO61859.2024.00077>
- [144] C. Alverti, S. Psomadakis, B. Ocalan, S. Jaiswal, T. Xu, and J. Torrellas, “CXLfork: Fast Remote Fork over CXL Fabrics,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds., ACM, 2025, pp. 210–226. DOI: 10.1145/3676641.3715988 [Online]. Available: <https://doi.org/10.1145/3676641.3715988>
- [145] J. Huang, M. Zhang, T. Ma, Z. Liu, S. Lin, K. Chen, J. Jiang, X. Liao, Y. Shan, N. Zhang, M. Lu, T. Ma, H. Gong, and Y. Wu, “TrEnv: Transparently Share Serverless Execution Environments Across Different Functions and Nodes,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSOP 2024, Austin, TX, USA, November 4-6, 2024*, E. Witchel, C. J. Rossbach, A. C. Arpaci-Dusseau, and K. Keeton, Eds., ACM, 2024, pp. 421–437. DOI: 10.1145/3694715.3695967 [Online]. Available:
<https://doi.org/10.1145/3694715.3695967>
- [146] Z. Wang, Y. Chen, C. Li, Y. Guan, D. Niu, T. Guan, Z. Du, X. Wei, and G. Sun, “CTXNL: A Software-Hardware Co-designed Solution for Efficient CXL-Based Transaction Processing,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*,

- Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds., ACM, 2025, pp. 192–209. DOI: 10.1145/3676641.3716244 [Online]. Available: <https://doi.org/10.1145/3676641.3716244>
- [147] W. Zhang, S. Gurumurthi, M. T. Kandemir, and A. Sivasubramaniam, “ICR: In-Cache Replication for Enhancing Data Cache Reliability,” in *2003 International Conference on Dependable Systems and Networks (DSN 2003), 22-25 June 2003, San Francisco, CA, USA, Proceedings*, IEEE Computer Society, 2003, pp. 291–300. DOI: 10.1109/DSN.2003.1209939 [Online]. Available: <https://doi.org/10.1109/DSN.2003.1209939>
- [148] W. Zhang, “Replication Cache: A Small Fully Associative Cache to Improve Data Cache Reliability,” *IEEE Trans. Computers*, vol. 54, no. 12, pp. 1547–1555, 2005. DOI: 10.1109/TC.2005.202 [Online]. Available: <https://doi.org/10.1109/TC.2005.202>
- [149] R. Fernández-Pascual, J. M. García, M. E. Acacio, and J. Duato, “A Low Overhead Fault Tolerant Coherence Protocol for CMP Architectures,” in *13th International Conference on High-Performance Computer Architecture (HPCA-13 2007), 10-14 February 2007, Phoenix, Arizona, USA*, IEEE Computer Society, 2007, pp. 157–168. DOI: 10.1109/HPCA.2007.346194 [Online]. Available: <https://doi.org/10.1109/HPCA.2007.346194>
- [150] A. Patil, V. Nagarajan, R. Balasubramonian, and N. Oswald, “Dvé: Improving DRAM Reliability and Performance On-Demand via Coherent Replication,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*, IEEE, 2021, pp. 526–539. DOI: 10.1109/ISCA52012.2021.00048 [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00048>

- [151] J. Chapin, M. Rosenblum, S. Devine, T. Lahiri, D. Teodosiu, and A. Gupta, “Hive: Fault Containment for Shared-Memory Multiprocessors,” in *Proceedings of the Fifteenth ACM Symposium on Operating System Principles, SOSP 1995, Copper Mountain Resort, Colorado, USA, December 3-6, 1995*, M. B. Jones, Ed., ACM, 1995, pp. 12–25. DOI: 10.1145/224056.224059 [Online]. Available: <https://doi.org/10.1145/224056.224059>
- [152] M. Banâtre, A. Gefflaut, P. Joubert, C. Morin, and P. A. Lee, “An Architecture for Tolerating Processor Failures in Shared Memory Multiprocessors,” *IEEE Trans. Computers*, vol. 45, no. 10, pp. 1101–1115, 1996. DOI: 10.1109/12.543705 [Online]. Available: <https://doi.org/10.1109/12.543705>
- [153] P. Ekemark, Y. Yao, A. Ros, K. Sagonas, and S. Kaxiras, “TSOPER: Efficient Coherence-Based Strict Persistency,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2021, Seoul, South Korea, February 27 - March 3, 2021*, IEEE, 2021, pp. 125–138. DOI: 10.1109/HPCA51647.2021.00021 [Online]. Available: <https://doi.org/10.1109/HPCA51647.2021.00021>
- [154] B. Ocalan, C. Alverti, S. Jaiswal, A. Psistakis, D. Koufaty, S. Mahar, S. Swanson, and J. Torrellas, “PhasedStore: Supporting High-Performance Write-Through Cache-Coherence Protocols Under TSO,” in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2026, pp. 1–14. DOI: 10.1109/HPCA68181.2026.11408509
- [155] L. Ke, X. Zhang, B. Lee, G. E. Suh, and H. S. Lee, “DisaggRec: Architecting Disaggregated Systems for Large-Scale Personalized Recommendation,” *CoRR*, vol. abs/2212.00939, 2022. DOI: 10.48550/ARXIV.2212.00939 arXiv: 2212.00939. [Online]. Available: <https://doi.org/10.48550/arXiv.2212.00939>

- [156] Y. Zhou, H. M. G. Wassel, S. Liu, J. Gao, J. Mickens, M. Yu, C. Kennelly, P. Turner, D. E. Culler, H. M. Levy, and A. Vahdat, “Carbink: Fault-Tolerant Far Memory,” in *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, M. K. Aguilera and H. Weatherspoon, Eds., USENIX Association, 2022, pp. 55–71. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/zhou-yang>
- [157] Z. Wang, Y. Guo, K. Lu, J. Wan, D. Wang, T. Yao, and H. Wu, “Rcmp: Reconstructing RDMA-Based Memory Disaggregation via CXL,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 1, 15:1–15:26, 2024. DOI: 10.1145/3634916 [Online]. Available: <https://doi.org/10.1145/3634916>
- [158] CXL Consortium, *Cxl 3.1 webinar presentation (february 2024)*, https://computeexpresslink.org/wp-content/uploads/2024/03/CXL_3.1-Webinar-Presentation_Feb_2024.pdf, Accessed: 2025-03-09, 2024.